華中科技大學

课 程 实 验 报 告

课程名称： 数据结构实验

专业班级 计算机类 2510 班

学　　号 U202515003

姓　　名 王皓轩

指导教师 杜冰倩

报告日期 2026 年 6 月 3 日

计算机科学与技术学院

目　　录

1　基于顺序存储结构的线性表实现 1
1.1　问题描述 1
1.2　系统设计 3
1.3　系统实现 27
1.4　系统测试 27
1.5　实验小结 43
2　基于二叉链表的二叉树实现 46
2.1　问题描述 46
2.2　系统设计 49
2.3　系统实现 51
2.4　系统测试 76
2.5　实验小结 88
3　课程的收获和建议 91
3.1　基于顺序存储结构的线性表实现 91
3.2　基于链式存储结构的线性表实现 91
3.3　基于二叉链表的二叉树实现 92
3.4　基于邻接表的图实现 93
参考文献 95
附录 A 基于顺序存储结构线性表实现的源程序 96
附录 B 基于链式存储结构线性表实现的源程序 114
附录 C 基于二叉链表二叉树实现的源程序 132
附录 D 基于邻接表图实现的源程序 151

1 基于顺序存储结构的线性表实现

1.1 问题描述

1.1.1 实验目的

通过实验达到：（1）加深对线性表的概念、基本运算的理解；（2）熟练掌握线性表的逻辑结构与物理结构的关系；（3）物理结构采用顺序表，熟练掌握顺序表基本运算的实现。

1.1.2 线性表运算定义

依据最小完备性和常用性相结合的原则，以函数形式定义了线性表的初始化表、销毁表、清空表、判定空表、求表长和获得元素等 12 种基本运算，具体运算功能定义如下：

（1）初始化表：函数名称是 InitList(L)；初始条件是线性表 L 不存在；操作结果是构造一个空的线性表；

（2）销毁表：函数名称是 DestroyList(L)；初始条件是线性表 L 已存在；操作结果是销毁线性表 L；

（3）清空表：函数名称是 ClearList(L)；初始条件是线性表 L 已存在；操作结果是将 L 重置为空表；

（4）判定空表：函数名称是 ListEmpty(L)；初始条件是线性表 L 已存在；操作结果是若 L 为空表则返回 TRUE, 否则返回 FALSE；

（5）求表长：函数名称是 ListLength(L)；初始条件是线性表已存在；操作结果是返回 L 中数据元素的个数；

（6）获得元素：函数名称是 GetElem(L,i,e)；初始条件是线性表已存在，

1≤i≤ListLength(L)；操作结果是用 e 返回 L 中第 i 个数据元素的值；

（7）查找元素：函数名称是 LocateElem(L,e,compare())；初始条件是线性表已存在；操作结果是返回 L 中第 1 个与 e 满足关系 compare（）关系的数据元素的位序，若这样的数据元素不存在，则返回值为 0；

（8）获得前驱：函数名称是 PriorElem(L,cur_e,pre_e)；初始条件是线性表 L 已存在；操作结果是若 cur_e 是 L 的数据元素，且不是第一个，则用 pre_e 返回它的前驱，否则操作失败，pre_e 无定义；

（9）获得后继：函数名称是 NextElem(L,cur_e,next_e)；初始条件是线性表 L 已存在；操作结果是若 cur_e 是 L 的数据元素，且不是最后一个，则用 next_e 返回它的后继，否则操作失败，next_e 无定义；

（10）插入元素：函数名称是 ListInsert(L,i,e)；初始条件是线性表 L 已存在，1≤i≤ListLength(L)+1；操作结果是在 L 的第 i 个位置之前插入新的数据元素 e。

（11）删除元素：函数名称是 ListDelete(L,i,e)；初始条件是线性表 L 已存在且非空，1≤i≤ListLength(L)；操作结果：删除 L 的第 i 个数据元素，用 e 返回的值；

（12）遍历表：函数名称是 ListTraverse(L,visit())，初始条件是线性表 L 已存在；操作结果是依次对 L 的每个数据元素调用函数 visit()。

附加功能：

（1）最大连续子数组和：函数名称是 MaxSubArray(L); 初始条件是线性表 L 已存在且非空，请找出一个具有最大和的连续子数组（子数组最少包含一个元素），操作结果是其最大和；

（2）和为 K 的子数组：函数名称是 SubArrayNum(L,k); 初始条件是线性表 L 已存在且非空, 操作结果是该数组中和为 k 的连续子数组的个数；

（3）顺序表排序：函数名称是 sortList(L)；初始条件是线性表 L 已存在；操作结果是将 L 由小到大排序；

（4）实现线性表的文件形式保存：其中，①需要设计文件数据记录格式，以高效保存线性表数据逻辑结构 (D,R) 的完整信息；②需要设计线性表文件保存和加载操作合理模式。附录 B 提供了文件存取的参考方法。

注：①保存到文件后，可以由一个空线性表加载文件生成线性表。

（5）实现多个线性表管理：设计相应的数据结构管理多个线性表的查找、添加、移除等功能。

注：附加功能（5）实现多个线性表管理应能创建、添加、移除多个线性表，单线性表和多线性表的区别仅仅在于线性表管理的个数不同，多线性表管理应可自由切换到管理的每个表，并可单独对某线性表进行单线性表的所有操作。

1.1.3 实验任务

采用顺序表作为线性表的物理结构，实现 1.2 小节的运算。其中 ElemType 为数据元素的类型名，具体含义可自行定义，其它有关类型和常量的定义和引用

详见文献 [1] 的 p10。

构造一个具有菜单的功能演示系统。其中，在主程序中完成函数调用所需实参值的准备和函数执行结果的显示，并给出适当的操作提示显示。附录 A 提供了简易菜单的框架（供参考）。

1.2 系统设计

1.2.1 系统结构设计

本项目搭建了支持多线性表管理的交互式控制台系统，通过可视化功能菜单完成各类操作。系统集成的核心功能包含：线性表的创建初始化、销毁释放、清空重置、空表判定、长度统计、指定位置元素获取、关键字查找、元素前驱与后继查询、元素插入与删除、全表遍历输出；同时拓展实现了最大连续子数组和计算、和为定值 K 的子数组统计、线性表升序排序；支持将线性表数据持久化保存至本地文件、从文件读取恢复数据。

系统支持同时创建多张独立线性表，可对每张表自定义命名，实现多表之间的灵活切换，实时查看所有已创建线性表的状态与数据。

整体架构上，在程序头部完成常量、自定义数据结构与类型的定义，实现各基础功能与拓展功能的函数封装；主函数搭建交互菜单界面，通过用户指令调用对应函数，完成整套系统的逻辑调度与功能执行。

1.2.2 数据结构设计

本实验采用动态顺序存储结构实现线性表，基于连续内存空间存储数据，借助动态内存分配实现存储空间自动扩容；同时设计多线性表统一管理结构，实现多张独立顺序表的创建、切换、删除与维护，整体结构分层清晰，可拓展性强。

1) 常量与基础类型定义

首先定义操作状态宏常量，用于函数返回值区分执行结果，包含操作成功、失败、不可行、内存溢出等状态标识；统一将存储元素定义为整型；设置顺序表初始容量与扩容增量，为动态顺序表提供基础参数。

```
// 状态常量定义
#define TRUE        1       // 逻辑真
#define FALSE       0       // 逻辑假
```

```
#define OK          1       // 操作成功
#define ERROR       0       // 操作出错
#define INFEASIBLE  -1      // 操作不可行
#define OVERFLOW    -2      // 内存分配失败

typedef int status;         // 函数返回状态类型
typedef int ElemType;       // 数据元素类型（int）

// 顺序表容量参数
#define LIST_INIT_SIZE 100  // 初始存储空间大小
#define LISTINCREMENT  10   // 扩容增量
```

2) 动态顺序表结构定义

采用动态分配的顺序表结构，结构体包含三个核心成员：动态数组指针指向连续的元素存储空间；length 记录当前有效元素个数；listsize 记录当前已分配的总容量。当插入元素导致空间不足时，通过 realloc 完成内存扩容，解决静态顺序表容量固定、空间利用率低的问题。

```
// 动态顺序表结构体定义
typedef struct{
    ElemType *elem;     // 数据元素存储空间基址
    int length;         // 当前有效元素个数
    int listsize;       // 当前分配的存储容量
}SqList;
```

3) 多线性表管理结构定义

为实现多表并行管理，设计全局管理结构体 LISTS，内部嵌套结构体数组，每个单元包含自定义表名与对应顺序表实例，最多可管理 10 张独立线性表；通过成员变量记录已创建表的总数。全局定义该结构体统一调度所有线性表，配合全局下标标记当前操作对象，实现多表灵活切换、查找与删除。

```
// 多线性表管理结构体
typedef struct{
    struct {
        char name[30];      // 线性表名称
        SqList L;           // 对应顺序表实体
```

```
    } elem[10];                 // 最多管理10张表
    int length;                 // 当前已创建表数量
}LISTS;

LISTS Lists;                    // 全局多表管理器
int current = 0;                // 当前正在操作的表下标
```

4) 结构逻辑与复杂度说明

整体采用双层嵌套架构：底层动态顺序表实现单表的增删改查、遍历、排序、文件读写、最大子数组和等核心功能；上层多表管理器完成多表的命名、切换、批量销毁等调度功能。动态顺序表可实现随机访问，按位获取元素时间复杂度为 $O(1)$；插入、删除需移动元素，时间复杂度为 $O(n)$；多表管理通过数组索引快速定位，切换效率高。该结构既满足基础线性表功能，又拓展了持久化存储、多实例管理等需求，适配本系统全部功能设计。

1.2.3 函数算法设计

各函数实现思想如下：

1.Status InitList(SqList *L)

输入：顺序表指针 L

输出：状态码（OK, OVERFLOW, INFEASIBLE）

算法思想描述：为顺序表动态分配初始内存，并设置初始状态。若表已初始化则返回不可行，防止内存泄漏。

算法处理步骤：

(1) 检查 L->elem 是否不为 NULL，如果是则返回 INFEASIBLE。

(2) 调用 malloc 分配大小为 LIST_INIT_SIZE 的内存空间给 L->elem。

(3) 如果分配失败，返回 OVERFLOW。

(4) 将 L->length 设为 0，L->listsize 设为 LIST_INIT_SIZE。

(5) 返回 OK。

时间复杂度：O(1)

空间复杂度：O(LIST_INIT_SIZE)

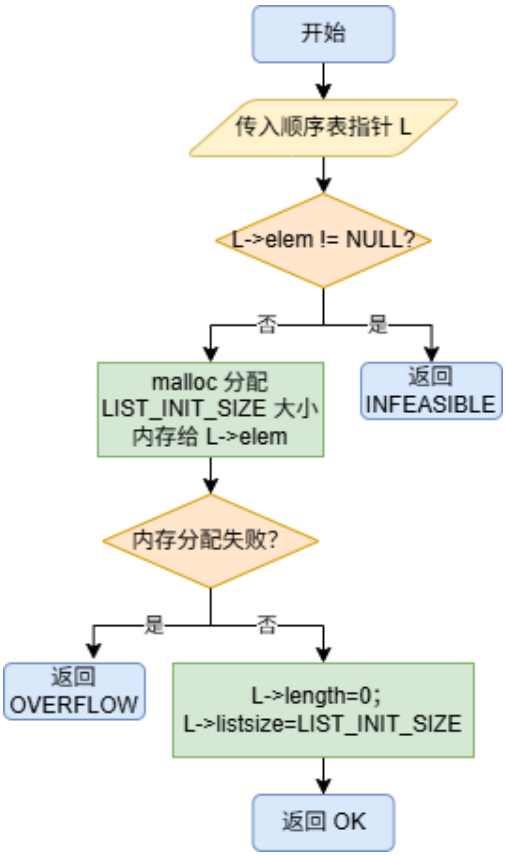


图 1-1　InitList 的程序流程图

2.Status DestroyList(SqList *L)

输入：顺序表指针 L

输出：状态码（OK, INFEASIBLE）

算法思想描述：释放顺序表动态分配的内存空间，并将相关字段重置为初始状态，防止悬空指针。

算法处理步骤：

(1) 检查 L->elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 调用 free 释放 L->elem 指向的内存空间。

(3) 将 L->elem 设为 NULL。

(4) 将 L->length 和 L->listsize 设为 0。

(5) 返回 OK。

时间复杂度：O(1)

空间复杂度：O(1)

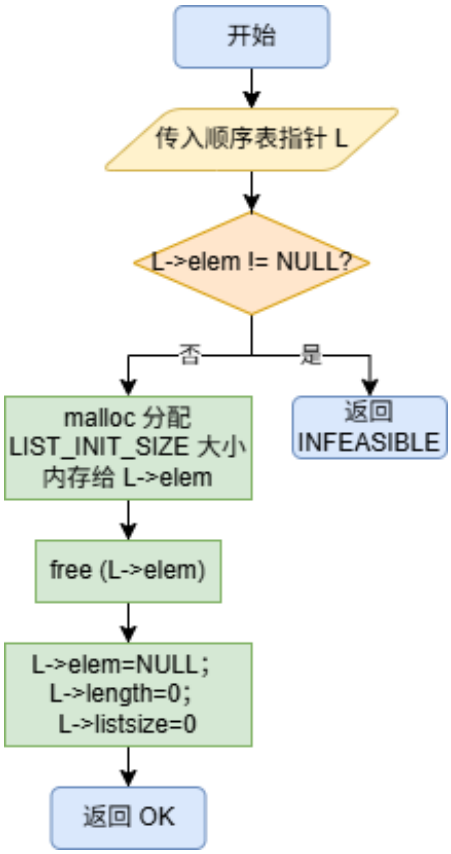


图 1-2　DestroyList 的程序流程图

3.Status ClearList(SqList *L)

输入：顺序表指针 L

输出：状态码（OK, INFEASIBLE）

算法思想描述：清空顺序表中的元素，仅将长度设为 0，不释放内存空间，以便复用。

算法处理步骤：

(1) 检查 L->elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 将 L->length 设为 0。

(3) 返回 OK。

时间复杂度：O(1)

空间复杂度：O(1)

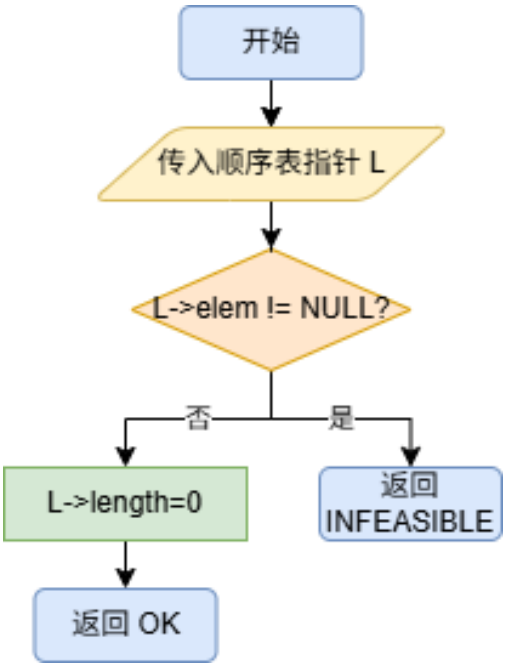


图 1-3　ClearList 的程序流程图

4.Status ListEmpty(SqList L)

输入：顺序表 L

输出：状态码（TRUE, FALSE, INFEASIBLE）

算法思想描述：判断顺序表是否为空表。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 判断 L.length 是否等于 0。

(3) 若等于 0，返回 TRUE；否则返回 FALSE。

时间复杂度：O(1)

空间复杂度：O(1)

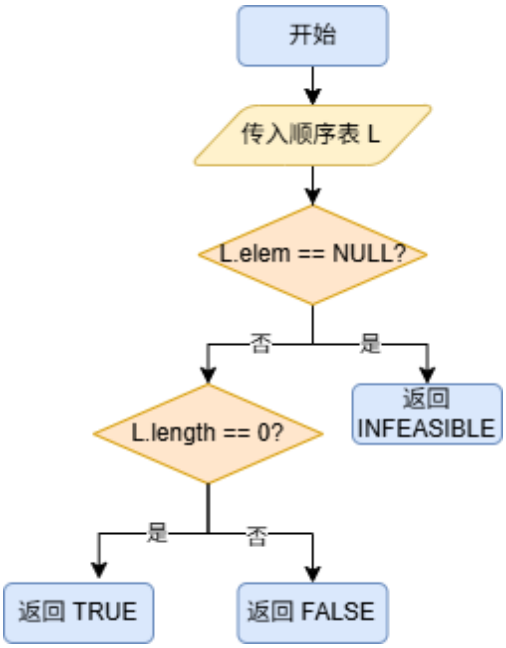


图 1-4　ListEmpty 的程序流程图

5.Status ListLength(SqList L)

输入：顺序表 L

输出：表长度或状态码（INFEASIBLE）

算法思想描述：获取顺序表中当前元素的个数。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 返回 L.length。

时间复杂度：O(1)

空间复杂度：O(1)

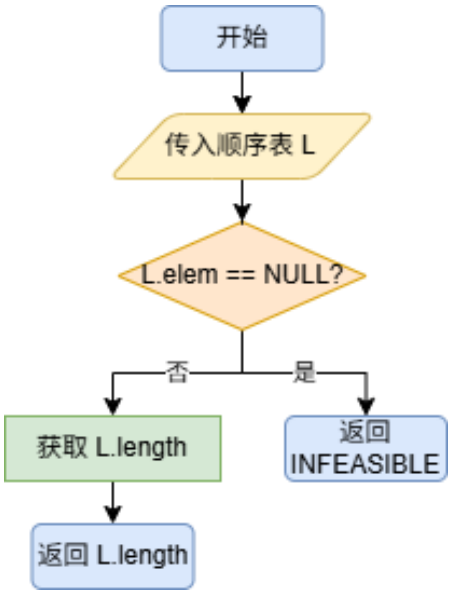


图 1-5　ListLength 的程序流程图

6.Status GetElem(SqList L, int i, ElemType *e)

输入：顺序表 L，位置 i，元素指针 e

输出：状态码（OK, ERROR, INFEASIBLE）

算法思想描述：获取顺序表中第 i 个元素的值，并将其存入 e 指向的变量。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 检查 i 是否满足 $1 \le i \le L.length$。若不满足，返回 ERROR。

(3) 将 L.elem[i-1] 的值赋给 *e。

(4) 返回 OK。

时间复杂度：O(1)

空间复杂度：O(1)

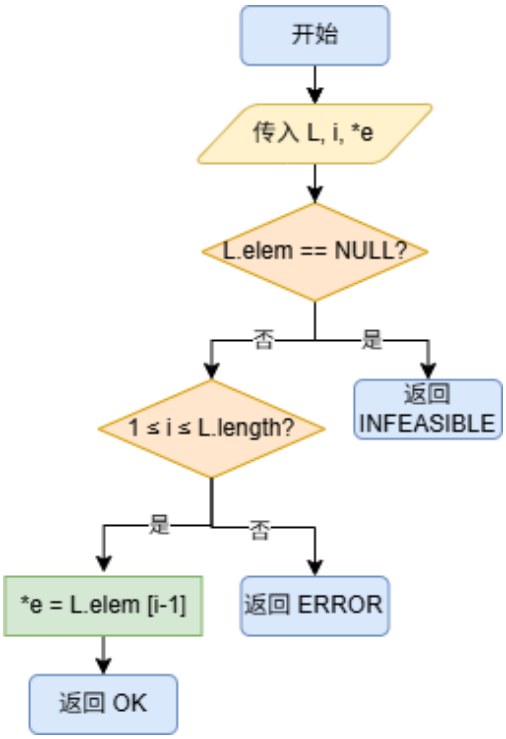


图 1-6　GetElem 的程序流程图

7.int LocateElem(SqList L, ElemType e)

输入：顺序表 L，元素 e

输出：元素位序（1 到 L.length）或 0（未找到），INFEASIBLE

算法思想描述：在顺序表中查找第一个值为 e 的元素，并返回其位序。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 初始化 i = 0。

(3) 当 i < L.length 且 L.elem[i] != e 时，执行 i++。

(4) 如果 i 等于 L.length，表示未找到，返回 0。

(5) 否则，返回 i + 1。

时间复杂度：O(n)

空间复杂度：O(1)

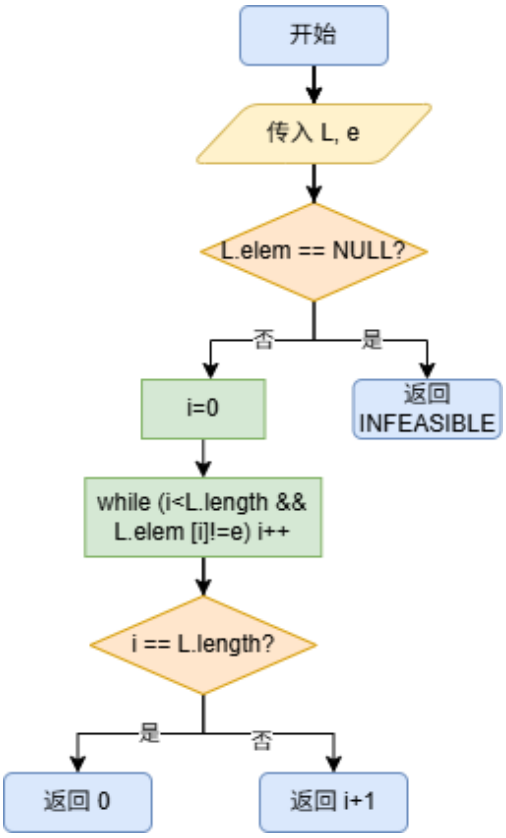


图 1-7　LocateElem 的程序流程图

8.Status PriorElem(SqList L, ElemType e, ElemType *pre_e)

输入：顺序表 L，元素 e，前驱元素指针 pre_e

输出：状态码（OK, ERROR, INFEASIBLE）

算法思想描述：求元素 e 的前驱元素。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 遍历查找 e 的位置 i。

(3) 如果 i == 0（e 是第一个元素）或 i >= L.length（未找到），返回 ERROR。

(4) 将 L.elem[i-1] 的值赋给 *pre_e。

(5) 返回 OK。

时间复杂度：O(n)

空间复杂度：O(1)

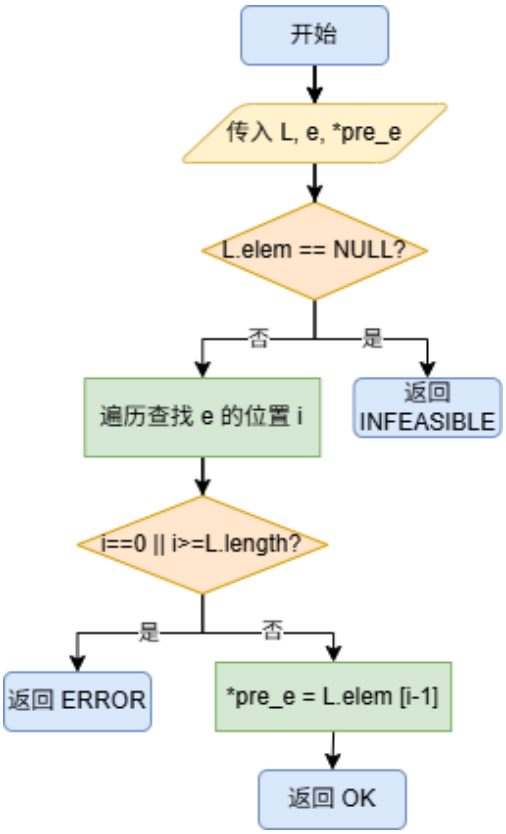


图 1-8　PriorElem 的程序流程图

9.Status NextElem(SqList L, ElemType e, ElemType *next_e)

输入：顺序表 L，元素 e，后继元素指针 next_e

输出：状态码（OK, ERROR, INFEASIBLE）

算法思想描述：求元素 e 的后继元素。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 遍历查找 e 的位置 i。

(3) 如果 i >= L.length - 1（e 是最后一个元素或未找到），返回 ERROR。

(4) 将 L.elem[i+1] 的值赋给 *next_e。

(5) 返回 OK。

时间复杂度：O(n)

空间复杂度：O(1)

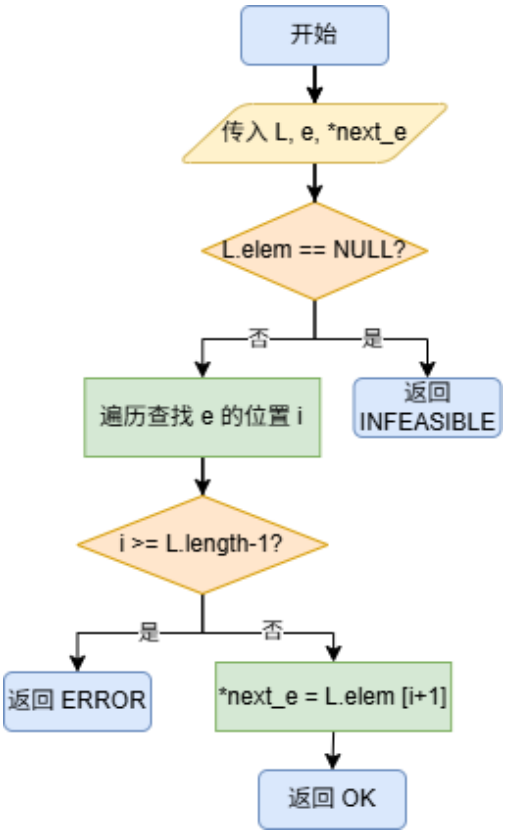


图 1-9　NextElem 的程序流程图

10.Status ListInsert(SqList *L, int i, ElemType e)

输入：顺序表指针 L，插入位置 i，元素 e

输出：状态码（OK, ERROR, OVERFLOW, INFEASIBLE）

算法思想描述：在顺序表的第 i 个位置插入元素 e。如果表满，则动态扩容。

算法处理步骤：

(1) 检查 L->elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 检查 i 是否满足 $1 \le i \le L->length + 1$。若不满足，返回 ERROR。

(3) 如果 L->length >= L->listsize，调用 realloc 扩容，增加 LISTINCREMENT。若失败，返回 OVERFLOW。

(4) 从 L->length - 1 到 i - 1，将元素依次后移一位。

(5) 将 e 插入到 L->elem[i-1] 位置。

(6) L->length 加 1。

(7) 返回 OK。

时间复杂度：O(n)

空间复杂度：O(1)

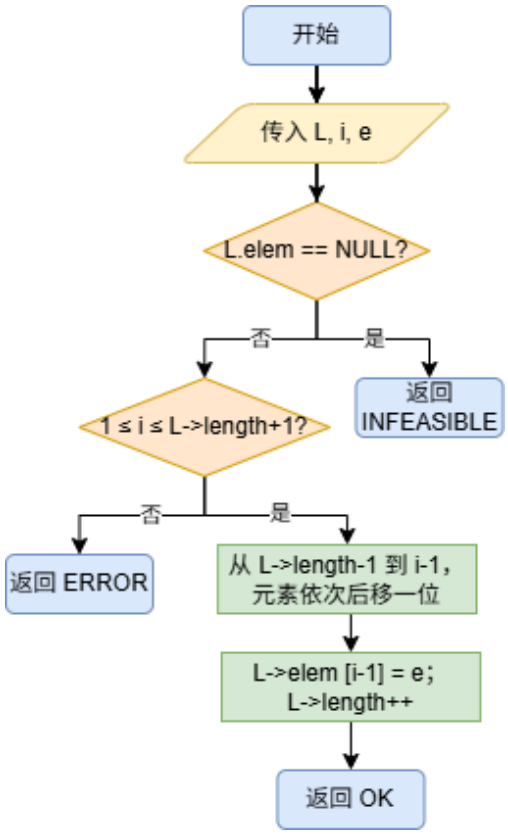


图 1-10　ListInsert 的程序流程图

11.Status ListDelete(SqList *L, int i, ElemType *e)

输入：顺序表指针 L，删除位置 i，被删元素指针 e

输出：状态码（OK, ERROR, INFEASIBLE）

算法思想描述：删除顺序表第 i 个位置的元素，并将其值存入 e。

算法处理步骤：

(1) 检查 L->elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 检查 i 是否满足 1 ≤ i ≤ L->length。若不满足，返回 ERROR。

(3) 将被删元素 L->elem[i-1] 存入 *e。

(4) 从 i 到 L->length - 1，将元素依次前移一位。

(5) L->length 减 1。

(6) 返回 OK。

时间复杂度：O(n)

空间复杂度：O(1)

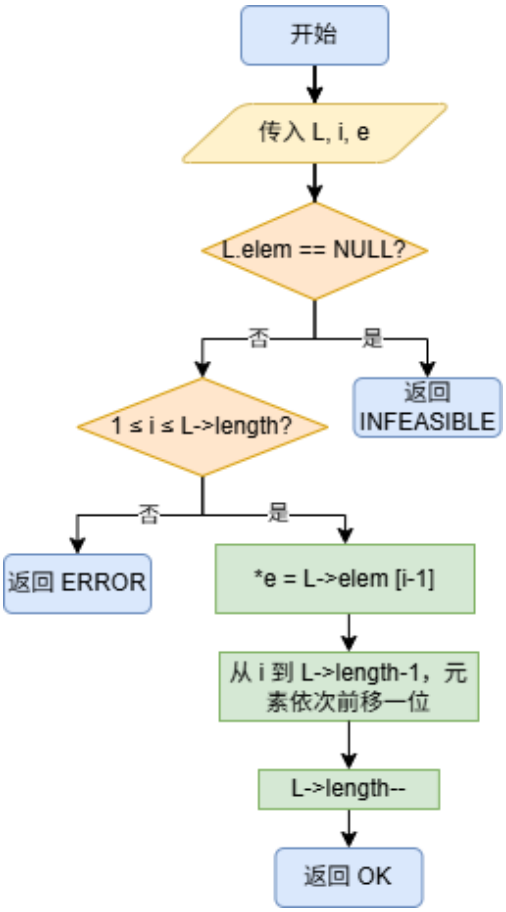


图 1-11　ListDelete 的程序流程图

12.Status ListTraverse(SqList L)

输入：顺序表 L

输出：状态码（OK, INFEASIBLE）

算法思想描述：遍历顺序表，按序打印所有元素。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，如果是则返回 INFEASIBLE。

(2) 若 L.length == 0，打印” 空线性表”。

(3) 否则，从 0 到 L.length - 1 遍历 L.elem，打印每个元素，元素间以空格分隔。

(4) 打印换行符。

(5) 返回 OK。

时间复杂度：O(n)

空间复杂度：O(1)

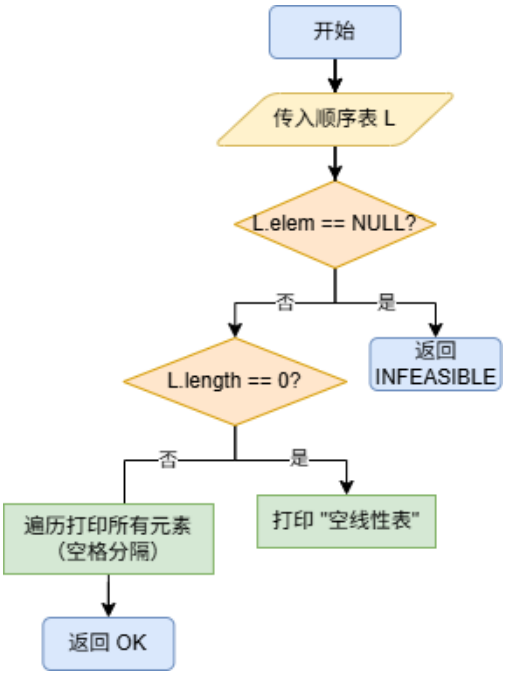


图 1-12　ListTraverse 的程序流程图

13.int MaxSubArray(SqList L)

输入：顺序表 L

输出：最大子数组和或状态码（INFEASIBLE）

算法思想描述：使用 Kadane 算法求最大子数组和。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL 或 L.length 是否为 0，若是则返回 INFEASIBLE。

(2) 初始化 maxSum = L.elem[0]，curSum = L.elem[0]。

(3) 从 i = 1 到 L.length - 1 执行循环：

a. curSum = (curSum > 0) ? curSum + L.elem[i] : L.elem[i]。

b. maxSum = (maxSum > curSum) ? maxSum : curSum。

(4) 返回 maxSum。

时间复杂度：O(n)

空间复杂度：O(1)

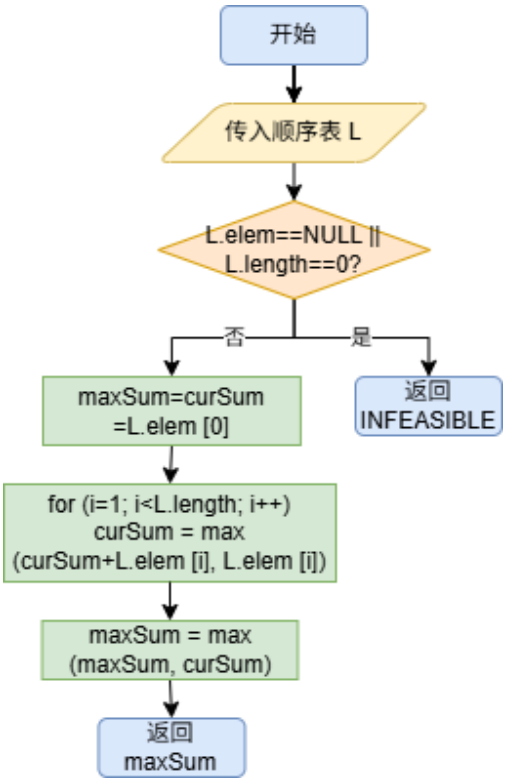


图 1-13　MaxSubArray 的程序流程图

14.int SubArrayNum(SqList L, int k)

输入：顺序表 L，整数 k

输出：子数组个数或状态码（INFEASIBLE）

算法思想描述：枚举所有连续子数组，统计和为 k 的个数。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL 或 L.length 是否为 0，若是则返回 INFEASIBLE。

(2) 初始化计数器 cnt = 0。

(3) 从 i = 0 到 L.length - 1 执行外层循环：

a. 初始化 sum = 0。

b. 从 j = i 到 L.length - 1 执行内层循环：

i. sum += L.elem[j]。

ii. 如果 sum == k，则 cnt++。

(4) 返回 cnt。

时间复杂度：$O(n^2)$

空间复杂度：O(1)

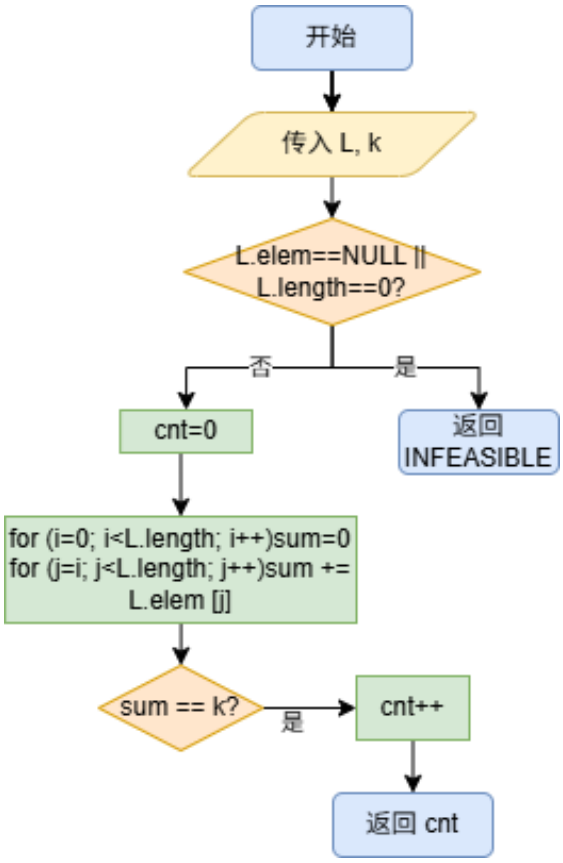


图 1-14　SubArrayNum 的程序流程图

15.void sortList(SqList *L)

输入：顺序表指针 L

输出：无

算法思想描述：使用冒泡排序算法将顺序表元素按升序排列。

算法处理步骤：

(1) 检查 L->elem 是否为 NULL 或 L->length ≤ 1，若是则直接返回。

(2) 从 i = 0 到 L->length - 2 执行外层循环：

a. 从 j = 0 到 L->length - i - 2 执行内层循环：

i. 如果 L->elem[j] > L->elem[j+1]，则交换这两个元素。

时间复杂度：O(n²)

空间复杂度：O(1)

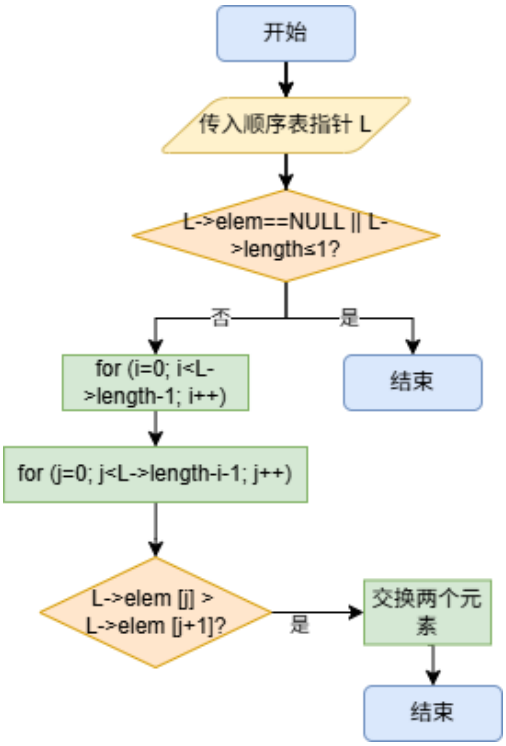


图 1-15　sortList 的程序流程图

16.Status SaveList(SqList L, char FileName[])

输入：顺序表 L，文件名 FileName[]

输出：状态码（OK, INFEASIBLE）

算法思想描述：将顺序表的数据以二进制形式保存到文件中。

算法处理步骤：

(1) 检查 L.elem 是否为 NULL，若是则返回 INFEASIBLE。

(2) 以写入二进制模式（”wb”）打开文件 FileName。若打开失败，返回 INFEASIBLE。

(3) 将 L.length 写入文件。

(4) 将 L.elem 指向的数组内容写入文件。

(5) 关闭文件。

(6) 返回 OK。

时间复杂度：O(n)

空间复杂度：O(1)

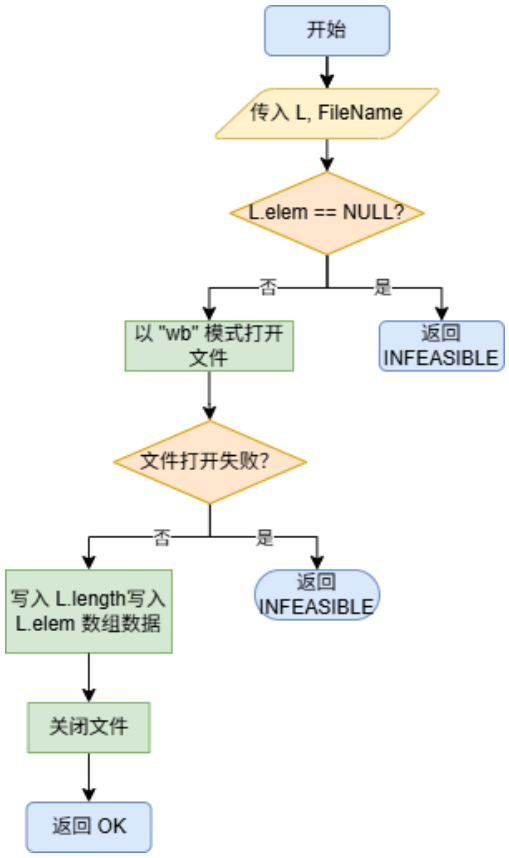


图 1-16　SaveList 的程序流程图

17.Status LoadList(SqList *L, char FileName[])

输入：顺序表指针 L，文件名 FileName[]

输出：状态码（OK, INFEASIBLE）

算法思想描述：从文件中读取二进制数据，重建顺序表。

算法处理步骤：

(1) 如果 L->elem 不为 NULL，先释放旧内存并将指针置空。

(2) 以读取二进制模式（”rb”）打开文件 FileName。若打开失败，返回 INFEASIBLE。

(3) 从文件读取长度到 len。

(4) 根据 len 为 L->elem 分配内存。

(5) 设置 L->length 和 L->listsize 为 len。

(6) 从文件读取数据到 L->elem。

(7) 关闭文件。

(8) 返回 OK。

时间复杂度：O(n)

空间复杂度：O(n)

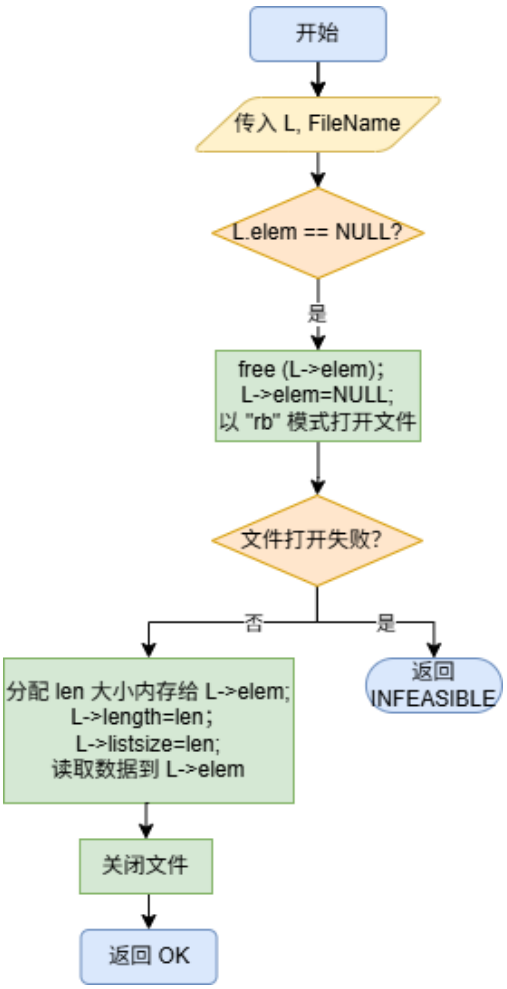


图 1-17　LoadList 的程序流程图

18.Status AddList(LISTS *Lists, char ListName[])

输入：表管理器指针 Lists，表名 ListName[]

输出：状态码（OK, ERROR）

算法思想描述：在表管理器中添加一张新的空表。

算法处理步骤：

(1) 检查 Lists->length 是否已达到 10（容量上限），若是则返回 ERROR。

(2) 将 ListName 复制到 Lists->elem[Lists->length].name。

(3) 将 Lists->elem[Lists->length].L.elem 设为 NULL，长度和容量设为 0。

(4) Lists->length 加 1。

(5) 返回 OK。

时间复杂度：O(1)

空间复杂度：O(1)

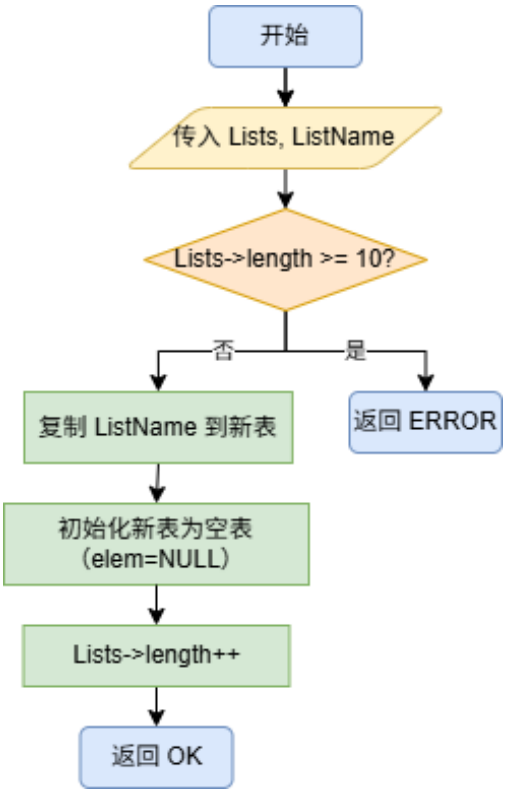


图 1-18　AddList 的程序流程图

19.Status RemoveList(LISTS *Lists, char ListName[])

输入：表管理器指针 Lists，表名 ListName[]

输出：状态码（OK, ERROR）

算法思想描述：在表管理器中删除指定名称的表及其所有数据。

算法处理步骤：

(1) 从 i = 0 到 Lists->length - 1 遍历查找名字为 ListName 的表：

a. 如果找到：

i. 如果表名是" 默认表"，打印提示并返回 ERROR。

ii. 调用 DestroyList 销毁该表的顺序表。

iii. 从 j = i 到 Lists->length - 2，将后续表前移一位。

iv. Lists->length 减 1。

v. 如果 i == current，则将 current 设为 0；否则如果 i < current，current 减 1。

vi. 返回 OK。

(2) 如果遍历结束未找到，返回 ERROR。

时间复杂度：O(m × n) （m 为表数量，n 为被删表的长度）

空间复杂度：O(1)

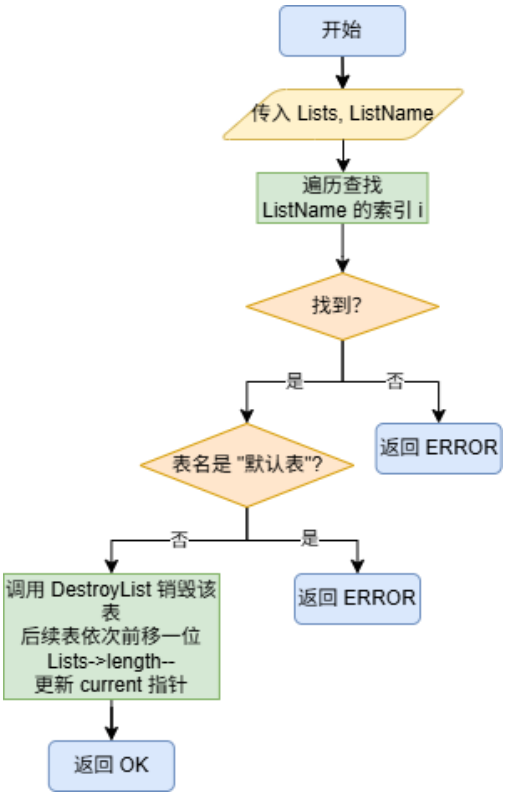


图 1-19　RemoveList 的程序流程图

20.int LocateList(LISTS Lists, char ListName[])

输入：表管理器 Lists，表名 ListName[]

输出：索引值（0 到 Lists.length - 1）或 -1（未找到）

算法思想描述：在表管理器中查找指定名称表的索引。

算法处理步骤：

(1) 从 i = 0 到 Lists.length - 1 遍历：

a. 如果 strcmp(ListName, Lists.elem[i].name) == 0，则返回 i。

(2) 若遍历结束未找到，返回 -1。

时间复杂度：O(m)

空间复杂度：O(1)

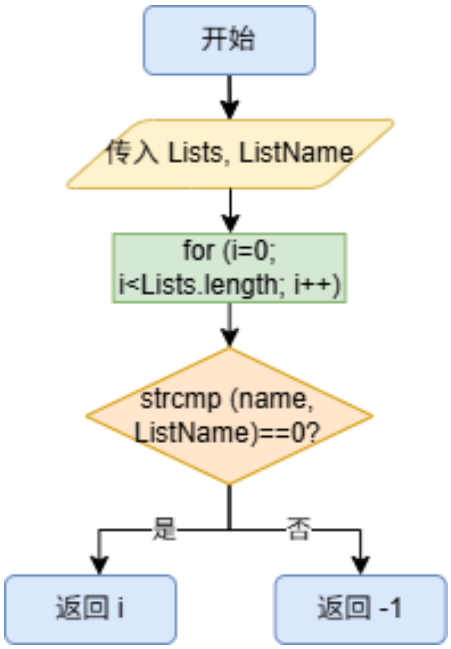


图 1-20　LocateList 的程序流程图

21.Status SwitchList(LISTS *Lists, char *name)

输入：表管理器指针 Lists，表名 name

输出：状态码（OK, ERROR）

算法思想描述：将当前操作表切换为指定名称的表。

算法处理步骤：

(1) 调用 LocateList 查找 name 的索引 idx。

(2) 如果 idx < 0，返回 ERROR。

(3) 将全局变量 current 设为 idx。

(4) 返回 OK。

时间复杂度：O(m)

空间复杂度：O(1)

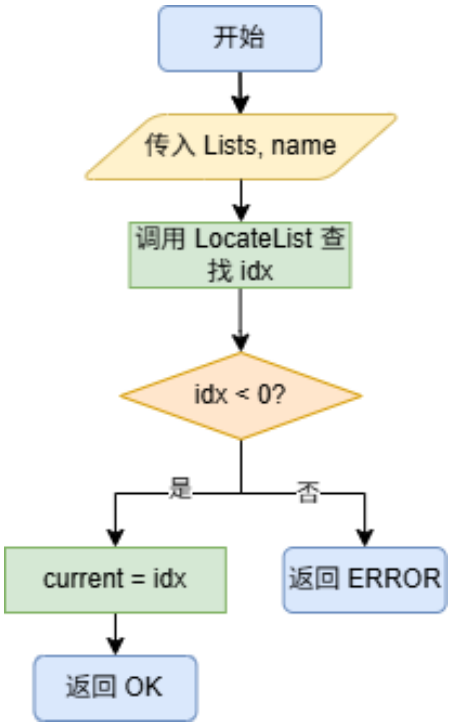


图 1-21　SwitchList 的程序流程图

22.void ShowAllLists(LISTS Lists)

输入：表管理器 Lists

输出：无

算法思想描述：打印所有表及其数据。

算法处理步骤：

(1) 打印表头和分隔线。

(2) 如果 Lists.length == 0，打印” 暂无任何线性表” 并返回。

(3) 从 i = 0 到 Lists.length - 1 遍历：

a. 打印表名。

b. 如果 Lists.elem[i].L.elem == NULL，打印” 未初始化”。

c. 否则，如果 Lists.elem[i].L.length == 0，打印” 空线性表”。

d. 否则，遍历打印该表的所有元素。

e. 如果 i == current，打印 ← 当前操作。

f. 打印换行。

(4) 打印结束分隔线。

时间复杂度：O(m × n)

空间复杂度：O(1)

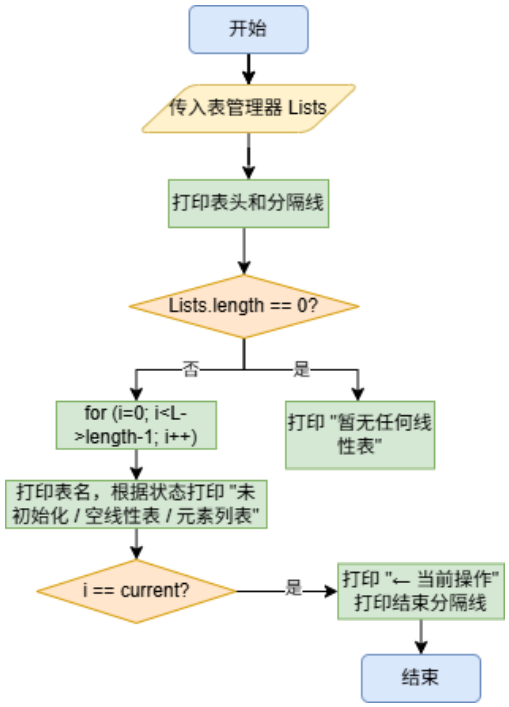


图 1-22　ShowAllLists 的程序流程图

1.3　系统实现

1.3.1　开发环境与工具

本实验采用 Windows 11 家庭版（版本 25H2）作为操作系统，使用 Visual Studio Code 作为代码编辑与集成开发环境。编译器选用 MSYS2 项目提供的 GCC 15.2.0（MinGW-w64 工具链），该编译器支持完整的 C99 标准，并在 Windows 平台下具有良好的兼容性与优化能力。编译时通过 -std=c99 显式指定语言标准，确保代码的可移植性与规范一致性。程序调试主要依赖 VS Code 内置的终端和 GDB 调试器（如有配置），开发过程中通过命令行执行 gcc 进行编译链接，生成可执行文件并运行测试。

1.4　系统测试

在 main 函数中构建功能演示菜单，并对主要功能进行测试。

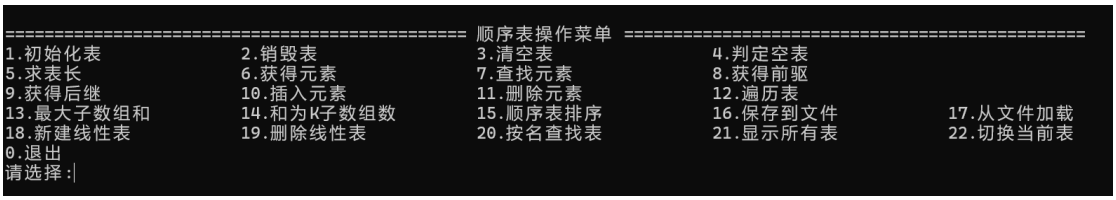


图 1-23　功能演示菜单

1. 初始化表测试

(1) 测试数据

表 1-1　功能 1 的测试数据

测试用例	程序输入	理论结果
首次初始化默认表	程序启动后选择功能 1	初始化成功
重复初始化	已初始化的表再次选择功能 1	已经初始化，无需重复操作

(2) 运行结果截图

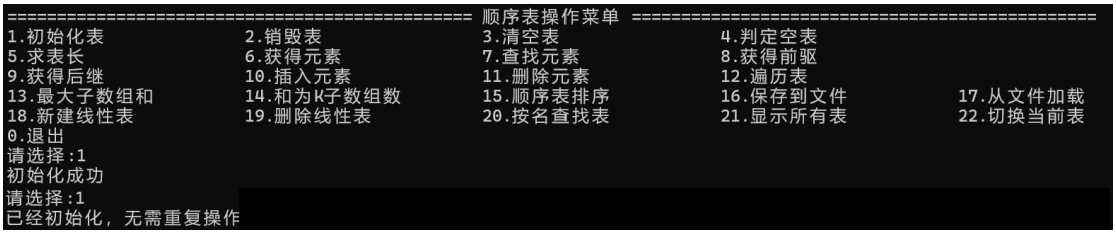


图 1-24　初始化表测试结果

2. 销毁表测试

(1) 测试数据

表 1-2　功能 2 的测试数据

测试用例	程序输入	理论结果
销毁普通表	新建表 test 并初始化，选择功能 2	销毁成功
销毁默认表	选中默认表选择功能 2	默认表不能销毁!
重复销毁	已销毁的表再次选择功能 2	已经销毁，无需重复操作

(2) 运行结果截图

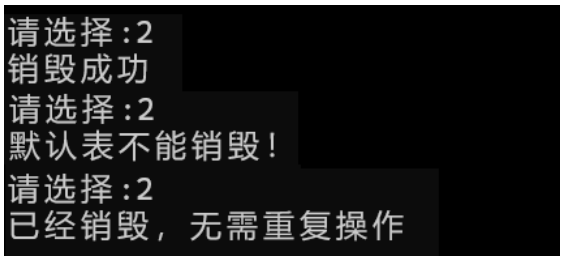


图 1-25　销毁表测试结果

3. 清空表测试

(1) 测试数据

表 1-3　功能 3 的测试数据

测试用例	程序输入	理论结果
清空非空表	默认表插入 1 2 3 后选择功能 3	清空成功
清空空表	已清空的表再次选择功能 3	清空成功
清空未初始化表	新建表 test 不初始化，选择功能 3	表未初始化，无法清空

(2) 运行结果截图

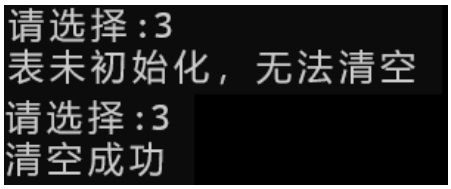


图 1-26　清空表测试结果

4. 判定空表测试

(1) 测试数据

表 1-4　功能 4 的测试数据

测试用例	程序输入	理论结果
空表判断	清空默认表后选择功能 4	空表
非空表判断	默认表插入 1 后选择功能 4	非空表

(2) 运行结果截图

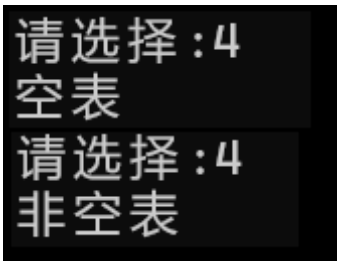


图 1-27　判定空表测试结果

5. 求表长测试

(1) 测试数据

表 1-5　功能 5 的测试数据

测试用例	程序输入	理论结果
空表长度	清空默认表后选择功能 5	表长：0
非空表长度	默认表插入 1 2 3 后选择功能 5	表长：3
未初始化表长度	新建表 test 不初始化，选择功能 5	表未初始化

(2) 运行结果截图



图 1-28　求表长测试结果

6. 获得元素测试

(1) 测试数据

表 1-6　功能 6 的测试数据

测试用例	程序输入	理论结果
正常获取元素	默认表插入 1 2 3，选择功能 6 输入位置：2	元素：2
位置小于 1	默认表插入 1 2 3，选择功能 6 输入位置：0	失败
位置大于表长	默认表插入 1 2 3，选择功能 6 输入位置：4	失败

(2) 运行结果截图



图 1-29　获得元素测试结果

7. 查找元素测试

(1) 测试数据

表 1-7　功能 7 的测试数据

测试用例	程序输入	理论结果
查找存在的元素	默认表插入 1 2 3，选择功能 7 输入值：2	位置：2
查找不存在的元素	默认表插入 1 2 3，选择功能 7 输入值：4	未找到
查找第一个元素	默认表插入 1 2 3，选择功能 7 输入值：1	位置：1
查找最后一个元素	默认表插入 1 2 3，选择功能 7 输入值：3	位置：3

(2) 运行结果截图

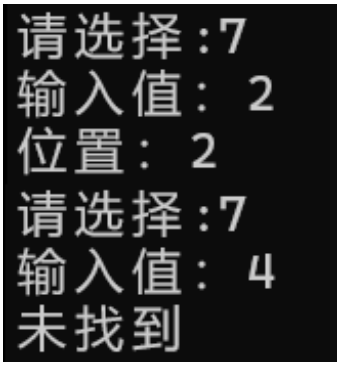


图 1-30　查找元素测试结果

8. 获得前驱测试

(1) 测试数据

表 1-8　功能 8 的测试数据

测试用例	程序输入	理论结果
正常获得前驱	默认表插入 1 2 3，选择功能 8 输入元素：2	前驱：1
第一个元素无前驱	默认表插入 1 2 3，选择功能 8 输入元素：1	无
元素不存在	默认表插入 1 2 3，选择功能 8 输入元素：4	无

(2) 运行结果截图

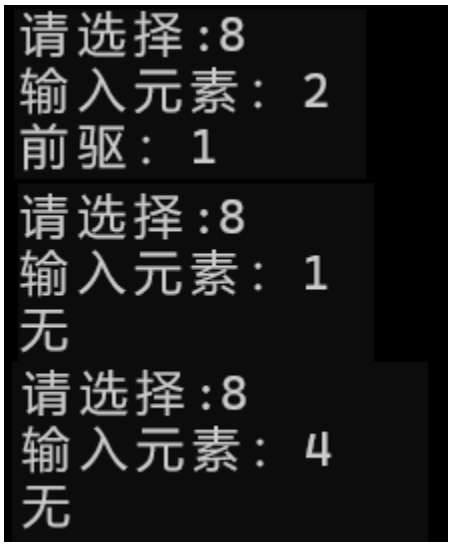


图 1-31　获得前驱测试结果

9. 获得后继测试

(1) 测试数据

表 1-9　功能 9 的测试数据

测试用例	程序输入	理论结果
正常获得后继	默认表插入 1 2 3，选择功能 9 输入元素：2	后继：3
最后一个元素无后继	默认表插入 1 2 3，选择功能 9 输入元素：3	无
元素不存在	默认表插入 1 2 3，选择功能 9 输入元素：4	无

(2) 运行结果截图



图 1-32　获得后继测试结果

10. 插入元素测试

(1) 测试数据

表 1-10　功能 10 的测试数据

测试用例	程序输入	理论结果
头部插入	空表选择功能 10 输入：1 5	成功
中间插入	表 [1,3] 选择功能 10 输入：2 2	成功
尾部插入	表 [1,2] 选择功能 10 输入：3 3	成功
位置小于 1	表 [1,2] 选择功能 10 输入：0 3	失败
位置大于表长 +1	表 [1,2] 选择功能 10 输入：4 3	失败

(2) 运行结果截图

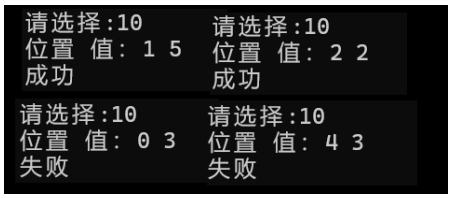


图 1-33　插入元素测试结果

11. 删除元素测试

(1) 测试数据

表 1-11　功能 11 的测试数据

测试用例	程序输入	理论结果
删除头部元素	表 [1,2,3] 选择功能 11 输入位置：1	删除：1
删除中间元素	表 [1,2,3] 选择功能 11 输入位置：2	删除：2
删除尾部元素	表 [1,2,3] 选择功能 11 输入位置：3	删除：3
位置小于 1	表 [1,2,3] 选择功能 11 输入位置：0	失败
位置大于表长	表 [1,2,3] 选择功能 11 输入位置：4	失败
空表删除	空表选择功能 11 输入位置：1	失败

(2) 运行结果截图

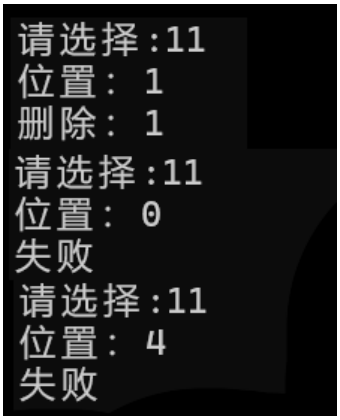


图 1-34　删除元素测试结果

12. 遍历表测试

(1) 测试数据

表 1-12　功能 12 的测试数据

测试用例	程序输入	理论结果
非空表遍历	表 [1,2,3] 选择功能 12	1 2 3
空表遍历	空表选择功能 12	空线性表

(2) 运行结果截图

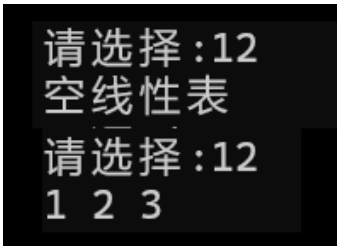


图 1-35　遍历表测试结果

13. 最大子数组和测试 (1) 测试数据

表 1-13　功能 13 的测试数据

测试用例	程序输入	理论结果
全正数数组	表 [1,2,3] 选择功能 13	最大子数组和：6
含负数数组	表 [-2,1,-3,4,-1,2,1,-5,4] 选择功能 13	最大子数组和：6
空表计算	空表选择功能 13	表未初始化或为空
未初始化表计算	新建表 test 不初始化，选择功能 13	表未初始化或为空

(2) 运行结果截图

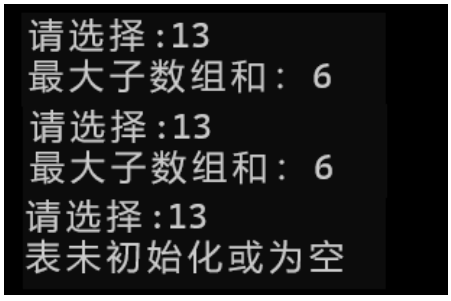


图 1-36　最大子数组和测试结果

14. 和为 K 子数组数测试

(1) 测试数据

表 1-14　功能 14 的测试数据

测试用例	程序输入	理论结果
正常情况	表 [1,1,1] 选择功能 14 输入 k：2	个数：2
含负数情况	表 [1,-1,0] 选择功能 14 输入 k：0	个数：3
无符合条件	表 [1,2,3] 选择功能 14 输入 k：6	个数：1
空表计算	空表选择功能 14 输入 k：1	表未初始化或为空
未初始化表计算	新建表 test 不初始化，选择功能 14 输入 k：1	表未初始化或为空

(2) 运行结果截图



图 1-37　和为 K 子数组数测试结果

15. 顺序表排序测试

(1) 测试数据

表 1-15　功能 15 的测试数据

测试用例	程序输入	理论结果
乱序数组排序	表 [3,1,2] 选择功能 15 再选择功能 12 遍历	1 2 3
已排序数组排序	表 [1,2,3] 选择功能 15 再选择功能 12 遍历	1 2 3
逆序数组排序	表 [3,2,1] 选择功能 15 再选择功能 12 遍历	1 2 3
单元素数组排序	表 [5] 选择功能 15 再选择功能 12 遍历	5

(2) 运行结果截图



图 1-38　顺序表排序测试结果

16. 保存到文件测试

(1) 测试数据

表 1-16　功能 16 的测试数据

测试用例	程序输入	理论结果
正常保存	表 [1,2,3,4,5] 选择功能 16 输入文件名：线性表写入测试.txt	保存成功
保存空表	空表选择功能 16 输入文件名：empty.txt	保存成功
未初始化表保存	新建表 test 不初始化，选择功能 16 输入文件名：test.txt	失败

(2) 运行结果截图

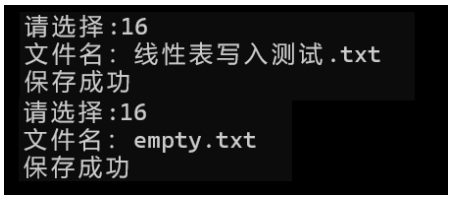


图 1-39　保存到文件测试结果

17. 从文件加载测试

(1) 测试数据

表 1-17　功能 17 的测试数据

测试用例	程序输入	理论结果
正常加载	选择功能 17 输入文件名：线性表写入测试.txt 再选择功能 12 遍历	1 2 3 4 5
文件不存在	选择功能 17 输入文件名：notexist.txt	失败
加载空表文件	选择功能 17 输入文件名：empty.txt 再选择功能 12 遍历	空线性表

(2) 运行结果截图



图 1-40　从文件加载测试结果

18. 新建线性表测试

(1) 测试数据

表 1-18　功能 18 的测试数据

测试用例	程序输入	理论结果
正常新建表	选择功能 18 输入表名：test 输入数据：1 2 3 0	test 1 2 3
新建空表	选择功能 18 输入表名：empty 输入数据：0	empty （无数据）
表数量已满	已创建 10 张表后再次选择功能 18	（自动失败，不报错）

(2) 运行结果截图

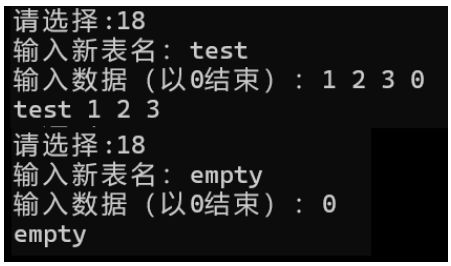


图 1-41　新建线性表测试结果

19. 删除线性表测试

(1) 测试数据

表 1-19　功能 19 的测试数据

测试用例	程序输入	理论结果
删除普通表	新建表 test 后选择功能 19 输入表名：test	删除成功!
删除默认表	选择功能 19 输入表名：默认表	无法删除默认表!
删除不存在的表	选择功能 19 输入表名：notexist	删除失败

(2) 运行结果截图

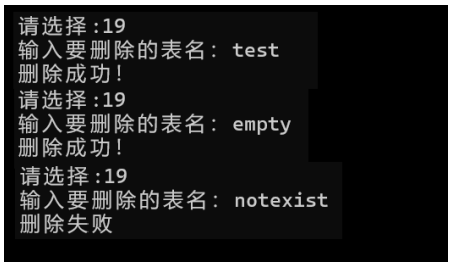


图 1-42　删除线性表测试结果

20. 按名查找表测试

(1) 测试数据

表 1-20　功能 20 的测试数据

测试用例	程序输入	理论结果
查找存在的表	新建表 test 后选择功能 20 输入表名：test	该表是第 2 个
查找默认表	选择功能 20 输入表名：默认表	该表是第 1 个
查找不存在的表	选择功能 20 输入表名：notexist	未找到

(2) 运行结果截图

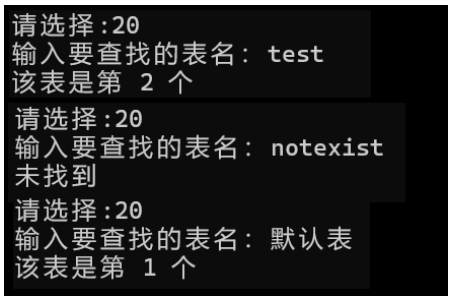


图 1-43　按名查找表测试结果

21. 显示所有表测试

(1) 测试数据

表 1-21　功能 21 的测试数据

测试用例	程序输入	理论结果
显示多个表	新建表 test 并插入 1 2 3 后选择功能 21	1. 表名：默认表数据：未初始化 2. 表名：test 数据：1 2 3 ← 当前操作
显示空表	新建表 empty 后选择功能 21	包含空线性表的条目

(2) 运行结果截图

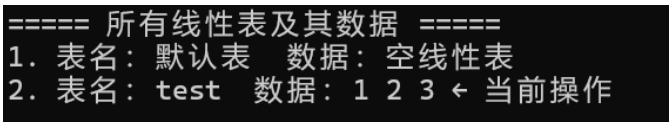


图 1-44　显示所有表测试结果

22. 切换当前表测试

(1) 测试数据

表 1-22　功能 22 的测试数据

测试用例	程序输入	理论结果
正常切换表	新建表 test 后选择功能 22 输入表名：test	已切换到表：test
切换到默认表	切换到 test 后选择功能 22 输入表名：默认表	已切换到表：默认表
切换不存在的表	选择功能 22 输入表名：notexist	表不存在

(2) 运行结果截图

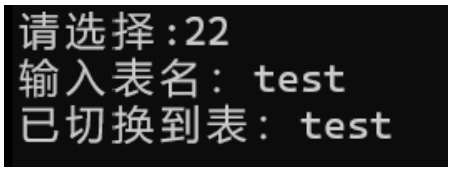


图 1-45　切换当前表测试结果

1.5　实验小结

本次实验主要完成了基于顺序存储结构的线性表各项基本运算与附加功能的实现。过程中除巩固了线性表的基础知识外，也弥补了一些认知盲区。以下是在实验过程中遇到的主要问题及解决思路。

1. 引用传参与指针传参的区分

在实现头歌实验时，起初对于平台给出的 &ElemType e 误以为是指针传参。经查阅资料与调试后认识到，引用从本质上讲是另一个变量的同义词，即一个变量的别名。引用参数传递的过程中，被调函数的形式参数作为局部变量在栈中开辟内存空间，但此时存放的是主调函数传入的实参变量的地址。被调函数对形参（本体）的任何操作都会被处理成间接寻址，即通过栈中存放的地址访问主调函数中的实参变量（根据别名找到主调函数中的本体）。因此，被调函数对形参的任何操作都会直接影响主调函数中的实参变量。这一认识纠正了我原先将引用与指针混用的错误习惯。

2. getchar() 与 cls 的使用

实验任务书中给出了菜单框架的示例，我在此基础上进行了改进。采用 system(”cls”) 实现清屏操作，使菜单界面保持简洁；同时使用 getchar() 清除输入缓

冲区，避免因残留换行符导致菜单跳转异常。在实际测试中发现，当用户输入数字后按回车，缓冲区中的换行符会干扰后续的菜单选择逻辑。通过在用户输入后增加一次 getchar() 调用以吞掉该换行符，有效解决了菜单自动跳过的问题，最终构建了交互较为舒适的菜单系统。

3. 避免重复调用函数

在实现主函数中多个分支判断时，由于部分函数的返回值包含多种情况，我最初在主函数中直接通过多个 if 语句判断函数返回值并执行相应的 printf 操作。然而测试发现结果与预期不符，经检查发现这种写法会导致函数被多次调用，造成逻辑错误。正确的做法是将函数返回值存储到一个变量中，再根据该变量的值进行判断。这一修正避免了函数的冗余调用，保证了程序的正确性。

4. 算法学习与优化

为了实现附加功能中的最大连续子数组和，我学习了贪心思想与 Kadane 算法。而在实现“和为 K 的子数组”统计时，最初采用双重循环枚举所有连续子数组，时间复杂度为 $O(n^2)$。通过查阅力扣上的题解，了解到可以采用哈希表进行优化，利用 map 记录中间的和与出现次数，可将时间复杂度降低至 O(n)。不过本次实验最终实现时仍采用了较为直观的双重循环版本，后续可在性能优化方面作进一步改进。

5. 文件存取与内存管理

在实现文件保存与加载功能时，遇到了加载文件可能导致原有数据丢失或内存泄漏的问题。为此，我在 LoadList 函数中增加了检查：若当前顺序表的 elem 指针非空，则先调用 DestroyList 销毁原有数据，再重新分配内存并加载文件内容。此外，在保存文件时先写入 length 字段，加载时先读取长度再读取数据数组，从而确保了文件读写的数据完整性与正确性。

6. 多表管理中的索引维护

在实现删除指定名称的线性表时，需同时维护全局的当前表下标 current。删除操作若发生在当前表之前，则需要将 current 减 1；若删除的是当前表本身，则将 current 重置为 0。最初遗漏了这一逻辑，导致删除后切换表出现越界或索引错误。通过加入索引调整的逻辑，该问题得到解决。

总体而言，本次实验从理论到实现完成了对顺序存储结构的完整实践。在基础操作、多表管理、持久化存储以及算法优化等方面都经历了调试与修正的过程，加深了对 C 语言指针、内存管理与模块化设计的理解，为后续链式存储结

构的学习积累了宝贵经验。

2 基于二叉链表的二叉树实现

2.1 问题描述

2.1.1 实验目的

通过实验达到：（1）加深对二叉树的概念、基本运算的理解；（2）熟练掌握二叉树的逻辑结构与物理结构的关系；（3）以二叉链表作为物理结构，熟练掌握二叉树基本运算的实现。

2.1.2 二叉树运算定义

依据最小完备性和常用性相结合的原则，以函数形式定义了二叉树的创建二叉树、销毁二叉树、清空二叉树、判定空二叉树和求二叉树深度等 14 种基本运算。具体运算功能定义和说明如下：

（1）创建二叉树：函数名称是 CreateBiTree(T,definition)；初始条件是 definition 给出二叉树 T 的定义，如带空子树的二叉树前序遍历序列、或前序 + 中序、或后序 + 中序；操作结果是按 definition 构造二叉树 T；

注：①要求 T 中各结点关键字具有唯一性。后面各操作的实现，也都要满足一棵二叉树中关键字的唯一性，不再赘述；②CreateBiTree 中根据 definition 生成 T，不应在 CreateBiTree 中输入二叉树的定义。

（2）销毁二叉树：函数名称是 DestroyBiTree(T)；初始条件是二叉树 T 已存在；操作结果是销毁二叉树 T；

注：①销毁了则原先所有分配的空间都销毁。

（3）清空二叉树：函数名称是 ClearBiTree (T)；初始条件是二叉树 T 存在；操作结果是将二叉树 T 清空；

注：①清空只是将二叉树清空了，是空树，但是在内存中还存在空间，只是值不确定。

（4）判定空二叉树：函数名称是 BiTreeEmpty(T)；初始条件是二叉树 T 存在；操作结果是若 T 为空二叉树则返回 TRUE，否则返回 FALSE；

（5）求二叉树深度：函数名称是 BiTreeDepth(T)；初始条件是二叉树 T 存在；操作结果是返回 T 的深度；

（6）查找结点：函数名称是 LocateNode(T,e)；初始条件是二叉树 T 已存在，e 是和 T 中结点关键字类型相同的给定值；操作结果是返回查找到的结点指针，如无关键字为 e 的结点，返回 NULL；

（7）结点赋值：函数名称是 Assign(T,e,value)；初始条件是二叉树 T 已存在，e 是和 T 中结点关键字类型相同的给定值；操作结果是关键字为 e 的结点赋值为 value；

（8）获得兄弟结点：函数名称是 GetSibling(T,e)；初始条件是二叉树 T 存在，e 是和 T 中结点关键字类型相同的给定值；操作结果是返回关键字为 e 的结点的（左或右）兄弟结点指针。若关键字为 e 的结点无兄弟，则返回 NULL；

（9）插入结点：函数名称是 InsertNode(T,e,LR,c)；初始条件是二叉树 T 存在，e 是和 T 中结点关键字类型相同的给定值，LR 为 0 或 1，c 是待插入结点；操作结果是根据 LR 为 0 或者 1，插入结点 c 到 T 中，作为关键字为 e 的结点的左或右孩子结点，结点 e 的原有左子树或右子树则为结点 c 的右子树；

特殊情况，c 插入作为根结点？可以考虑 LR 为-1 时，作为根结点插入，原根结点作为 c 的右子树。

（10）删除结点：函数名称是 DeleteNode(T,e)；初始条件是二叉树 T 存在，e 是和 T 中结点关键字类型相同的给定值。操作结果是删除 T 中关键字为 e 的结点；同时，如果关键字为 e 的结点度为 0，删除即可；如关键字为 e 的结点度为 1，用关键字为 e 的结点孩子代替被删除的 e 位置；如关键字为 e 的结点度为 2，用 e 的左孩子代替被删除的 e 位置，e 的右子树作为 e 的左子树中最右结点的右子树；

（11）前序遍历：函数名称是 PreOrderTraverse(T,Visit())；初始条件是二叉树 T 存在，Visit 是一个函数指针的形参（可使用该函数对结点操作）；操作结果：先序遍历，对每个结点调用函数 Visit 一次且一次，一旦调用失败，则操作失败。

注：前序、中序和后序三种遍历算法，要求至少一个用非递归算法实现。

（12）中序遍历：函数名称是 InOrderTraverse(T,Visit))；初始条件是二叉树 T 存在，Visit 是一个函数指针的形参（可使用该函数对结点操作）；操作结果是中序遍历 t，对每个结点调用函数 Visit 一次且一次，一旦调用失败，则操作失败；

（13）后序遍历：函数名称是 PostOrderTraverse(T,Visit))；初始条件是二叉树 T 存在，Visit 是一个函数指针的形参（可使用该函数对结点操作）；操作结果是后序遍历 t，对每个结点调用函数 Visit 一次且一次，一旦调用失败，则操作失败。

（14）按层遍历：函数名称是 LevelOrderTraverse(T,Visit))；初始条件是二叉树 T 存在，Visit 是对结点操作的应用函数；操作结果是层序遍历 t，对每个结点调用函数 Visit 一次且一次，一旦调用失败，则操作失败。

附加功能：

（1）最大路径和：函数名称是 MaxPathSum(T)，初始条件是二叉树 T 存在；操作结果是返回根结点到叶子结点的最大路径和；

注：①从根结点开始。

（2）最近公共祖先：函数名称是 LowestCommonAncestor(T,e1,e2)；初始条件是二叉树 T 存在；操作结果是该二叉树中 e1 结点和 e2 结点的最近公共祖先；

注：①最近公共祖先可以是 e1，e2 中的某个结点。

（3）翻转二叉树：函数名称是 InvertTree(T)，初始条件是线性表 L 已存在；操作结果是将 T 翻转，使其所有结点的左右结点互换；

（4）实现线性表的文件形式保存：其中，①需要设计文件数据记录格式，以高效保存二叉树数据逻辑结构 (D,R) 的完整信息；②需要设计二叉树文件保存和加载操作合理模式。附录 B 提供了文件存取的方法；

注：①保存到文件后，可以由一个空二叉树加载文件生成二叉树。

（5）实现多个二叉树管理：可采用线性表的方式管理多个二叉树，线性表中的每个数据元素为一个二叉树的基本属性，至少应包含有二叉树的名称。基于顺序表实现的二叉树管理，其物理结构的参考设计如图 2-1 所示。

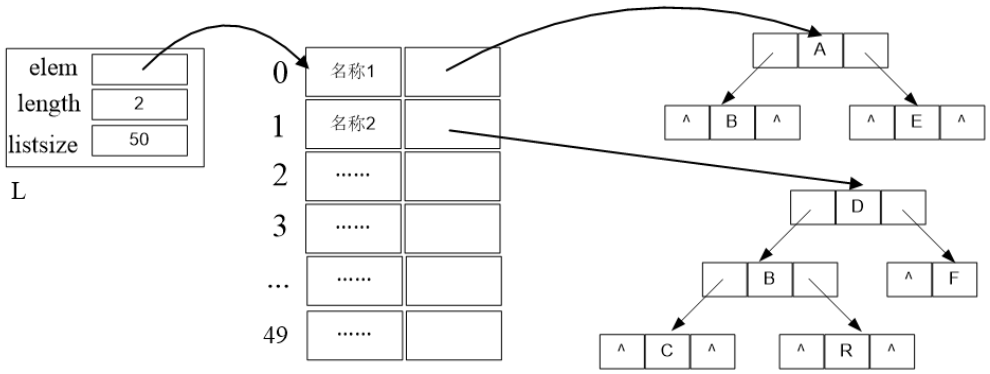


图 2-1　多二叉树管理的物理结构示意图

注：附加功能（5）实现多个二叉树管理应能创建、添加、移除多个二叉树，单二叉树和多二叉树的区别仅仅在于二叉树管理的个数不同，多二叉树管理应可自由切换到管理的每个二叉树，并可单独对某二叉树进行单二叉树的所有操作。

2.1.3 实验任务

采用二叉链表表作为二叉树的物理结构，实现 3.2 小节的运算。其中 ElemType 为数据元素的类型名，具体含义可自行定义，但要求二叉树结点类型为结构型，至少包含二个部分，一个是能唯一标识一个结点的关键字（类似于学号或职工号），另一个是其它部分。其它有关类型和常量的定义和引用详见文献 [1] 的 p10。

构造一个具有菜单的功能演示系统。其中，在主程序中完成函数调用所需实参值的准备和函数执行结果的显示，并给出适当的操作提示显示（供参考）。

2.2 系统设计

2.2.1 系统结构设计

本项目搭建了支持多二叉树管理的交互式控制台系统，通过可视化功能菜单完成各类操作。系统集成的核心功能包含：二叉树的先序创建、销毁释放、清空重置、空树判定、深度计算、按关键字查找结点、结点数据修改、兄弟结点查询、结点插入与删除、先序/中序/后序/层序四种遍历输出；同时拓展实现了根到叶子的最大路径和、最近公共祖先查找、二叉树翻转；支持将二叉树数据持久化保存至本地文件、从文件读取恢复树结构。

系统支持同时创建多棵独立二叉树，可对每棵树自定义命名，实现多树之间的灵活切换，实时查看所有已创建二叉树的状态。

整体架构上，在程序头部完成常量、自定义数据结构与类型的定义，实现各基础功能与拓展功能的函数封装；主函数搭建交互菜单界面，通过用户指令调用对应函数，完成整套系统的逻辑调度与功能执行。

2.2.2 数据结构设计

本实验采用二叉链表存储结构实现二叉树，每个结点包含数据域与指向左右孩子的指针；同时设计多二叉树统一管理结构，实现多棵独立二叉树的创建、切换、删除与维护，整体结构分层清晰，可拓展性强。

1) 常量与基础类型定义

首先定义操作状态宏常量，用于函数返回值区分执行结果；统一将关键字定义为整型，结点数据域由关键字和附加信息字符串组成；设置最大管理树数量，为多树管理器提供容量限制。

```
// 状态常量定义
#define TRUE         1        // 逻辑真
#define FALSE        0        // 逻辑假
#define OK           1        // 操作成功
#define ERROR        0        // 操作出错
#define MAX_TREE     10       // 最多管理10棵二叉树

typedef int status;           // 函数返回状态类型
typedef int KeyType;          // 关键字类型（int）

// 二叉树结点数据域结构
typedef struct {
    KeyType key;              // 关键字
    char others[20];          // 其他信息（字符串）
} TElemType;
```

2) 二叉链表结点结构定义

采用二叉链表作为二叉树的存储结构，每个结点包含一个 TElemType 类型的数据域，以及指向左孩子和右孩子的指针。通过递归或栈结构可以方便地实现各种遍历与查找操作。

```
// 二叉链表结点结构
typedef struct BiTNode {
    TElemType data;                 // 数据域
    struct BiTNode *lchild, *rchild;  // 左右孩子指针
} BiTNode, *BiTree;
```

3) 多二叉树管理结构定义

为实现多棵二叉树的并行管理，设计全局管理结构体 TreeManager，内部包含二叉树根指针数组、树名数组、当前树总数及当前选中树下标。通过该结构体

统一调度所有二叉树，配合全局下标标记当前操作对象，实现多树灵活切换、查找与删除。

```
// 多二叉树管理结构体
typedef struct {
    BiTree trees[MAX_TREE];        // 存储多棵树的根指针
    char names[MAX_TREE][20];      // 各树的名称
    int count;                     // 当前已创建树数量
    int current;                   // 当前选中树的下标
} TreeManager;

TreeManager TM;                    // 全局多树管理器
```

4) 结构逻辑与复杂度说明

整体采用二叉链表与多树管理器相结合的双层架构：底层二叉链表实现单棵二叉树的创建、遍历、查找、插入、删除、翻转、路径和计算等核心功能；上层多树管理器完成多棵树的命名、切换、批量销毁等调度功能。

二叉链表结构不具备随机访问特性，按值查找和插入删除操作通常需要遍历，时间复杂度为 $O(n)$；四种遍历可借助递归或显式栈、队列在线性时间内完成；多树管理通过数组索引快速定位，切换效率为 $O(1)$。该结构既满足基础二叉树操作需求，又拓展了持久化存储、多实例管理等高级功能，适配本系统全部功能设计。

2.3 系统实现

2.3.1 开发环境与工具

本实验采用 Windows 11 家庭版（版本 25H2）作为操作系统，使用 Visual Studio Code 作为代码编辑与集成开发环境。编译器选用 MSYS2 项目提供的 GCC 15.2.0（MinGW-w64 工具链），该编译器支持完整的 C99 标准，并在 Windows 平台下具有良好的兼容性与优化能力。编译时通过 `-std=c99` 显式指定语言标准，确保代码的可移植性与规范一致性。程序调试主要依赖 VS Code 内置终端和 GDB 调试器（如有配置），开发过程中通过命令行执行 `gcc` 进行编译链接，生成可执行文件并运行测试。

2.3.2 函数算法设计

各函数严格按照最小完备性与常用性原则设计，共包含 14 种基本运算和 5 种扩展功能，所有操作均保证二叉树结点关键字的唯一性。

一、基本运算（14 种）

1. status CreateBiTree(BiTree *T, const char *definition)

输入：二叉树根结点指针的指针 T，二叉树定义序列 definition（带空子树标记的前序遍历序列，0 表示空结点，结点格式为"key others"）

输出：状态码（OK, ERROR）

算法思想描述：从给定的 definition 字符串中读取带空子树标记的前序遍历序列，递归创建二叉树。函数内部不进行控制台输入，所有数据完全来自 definition 参数，创建过程中自动检查关键字唯一性，确保树中无重复关键字。

算法处理步骤：

(1) 初始化输入指针指向 definition 起始位置，重置关键字检查标记。

(2) 递归解析 definition 序列：

a. 跳过空格，读取当前关键字 key 和附加信息 others。

b. 若 key 为 0，创建空结点并返回。

c. 检查关键字 key 是否已存在，若存在返回 ERROR。

d. 为当前结点分配大小为 BiTNode 结构体的内存空间。

e. 将 key 和 others 分别赋值给结点的数据域 key 和 others。

f. 将结点的左右孩子指针初始化为 NULL。

g. 递归创建左子树。

h. 递归创建右子树。

(3) 解析完成后返回 OK。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅创建和检查一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

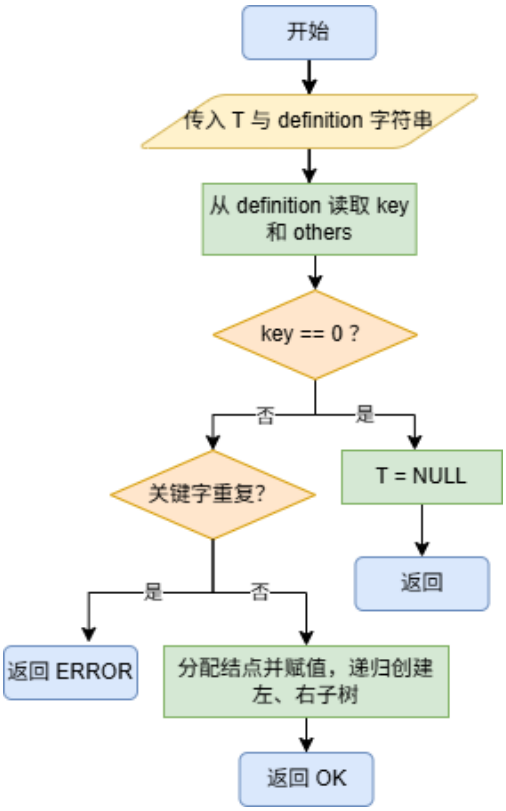


图 2-2　CreateBiTree 的程序流程图

2. status DestroyBiTree(BiTree *T)

输入：二叉树根结点指针的指针 T

输出：状态码（OK, ERROR）

算法思想描述：采用后序遍历顺序递归销毁二叉树所有结点，先销毁左右子树再销毁当前结点，彻底释放所有分配的内存空间，最后将根结点指针置为 NULL，确保无内存泄漏。销毁后二叉树不再存在于内存中。

算法处理步骤：

(1) 若 *T 为 NULL，返回 ERROR（树不存在或已销毁）。

(2) 递归调用 DestroyBiTree 销毁左子树。

(3) 递归调用 DestroyBiTree 销毁右子树。

(4) 释放 *T 指向的结点内存。

(5) 将 *T 置为 NULL。

(6) 返回 OK。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅释放一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

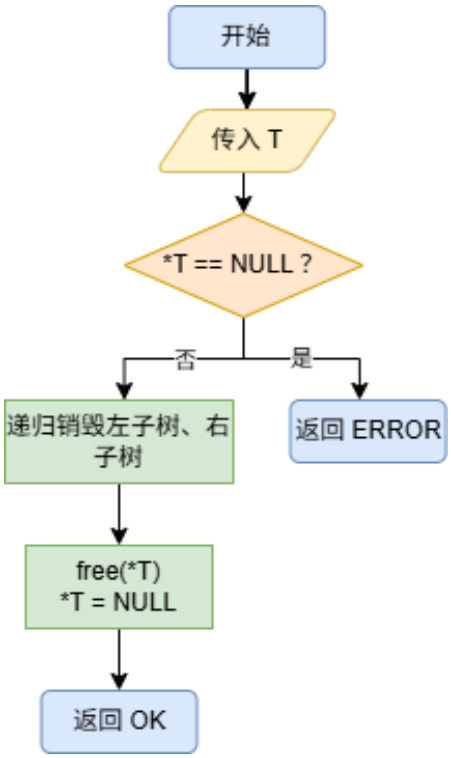


图 2-3　DestroyBiTree 的程序流程图

3. status ClearBiTree(BiTree *T)

输入：二叉树根结点指针的指针 T

输出：状态码（OK, ERROR）

算法思想描述：清空二叉树，将其变为空树。与销毁操作不同，清空操作不释放二叉树已分配的内存空间，仅将所有结点的数据内容重置为不确定状态，并将根结点指针置为 NULL，使二叉树在逻辑上为空，但物理内存仍保留，可后续重新使用该空间创建新的二叉树。

算法处理步骤：

(1) 若 *T 为 NULL，返回 ERROR（树不存在或已清空）。

(2) 递归调用 ClearBiTree 清空左子树。

(3) 递归调用 ClearBiTree 清空右子树。

(4) 将当前结点的数据域 key 置为 0，others 数组置为空字符串。

(5) 将根结点指针 *T 置为 NULL。

(6) 返回 OK。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅清空一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

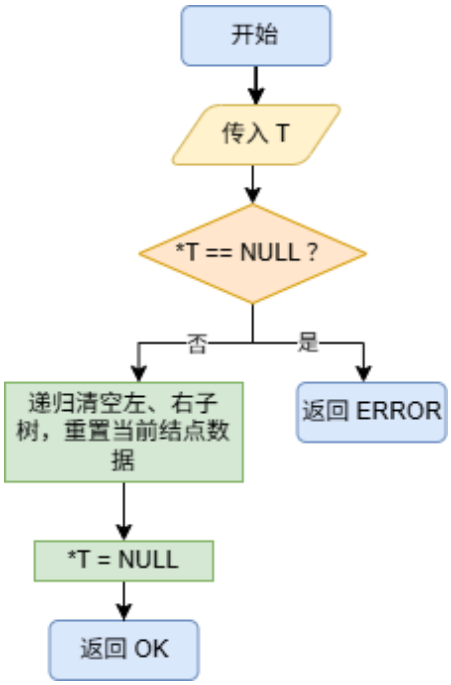


图 2-4　ClearBiTree 的程序流程图

4. status BiTreeEmpty(BiTree T)

输入：二叉树根结点指针 T

输出：状态码（TRUE, FALSE）

算法思想描述：判断二叉树是否为空树，通过检查根结点指针是否为 NULL 来实现，是二叉树最基本的状态查询操作。

算法处理步骤：

(1) 若根结点指针 T 为 NULL，返回 TRUE 表示是空树。

(2) 否则返回 FALSE 表示是非空树。

时间复杂度：O(1)，仅需一次指针比较操作。

空间复杂度：O(1)，无需额外空间。

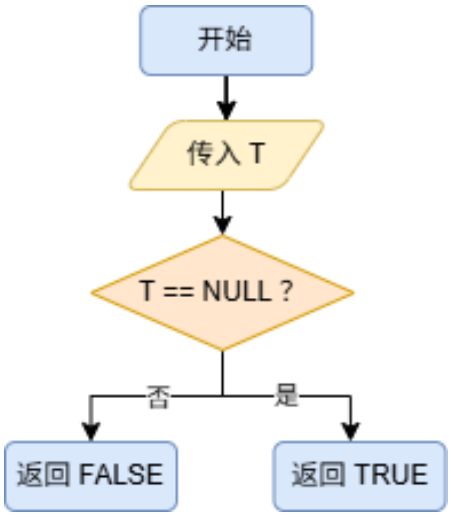


图 2-5　BiTreeEmpty 的程序流程图

5. int BiTreeDepth(BiTree T)

输入：二叉树根结点指针 T

输出：二叉树的深度，空树返回 0

算法思想描述：递归计算二叉树的深度，树的深度定义为从根结点到最远叶子结点的路径上的结点数，等于左子树和右子树深度的最大值加 1（当前结点）。

算法处理步骤：

(1) 若 T 为空，返回 0 表示空树深度为 0。

(2) 递归计算左子树的深度 l。

(3) 递归计算右子树的深度 r。

(4) 返回 l 和 r 中的较大值加 1，即为当前树的深度。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅访问一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

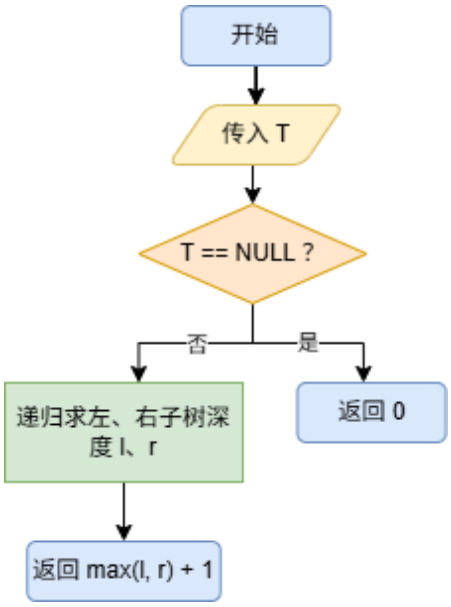


图 2-6　BiTreeDepth 的程序流程图

6. BiTNode* LocateNode(BiTree T, KeyType e)

输入：二叉树根结点指针 T，关键字 e

输出：找到的结点指针，未找到返回 NULL

算法思想描述：递归遍历二叉树查找关键字为 e 的结点，按照”当前结点-左子树-右子树”的先序顺序进行查找，找到后立即返回，无需遍历剩余结点。

算法处理步骤：

(1) 若 T 为空，返回 NULL 表示未找到。

(2) 若当前结点 T 的数据域 key 等于 e，返回 T 表示找到目标结点。

(3) 递归调用 LocateNode 在左子树中查找，结果存入 p。

(4) 若 p 不为 NULL，说明左子树中找到，返回 p。

(5) 递归调用 LocateNode 在右子树中查找并返回结果。

时间复杂度：O(n)，n 为二叉树结点数，最坏情况下需要遍历所有结点。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

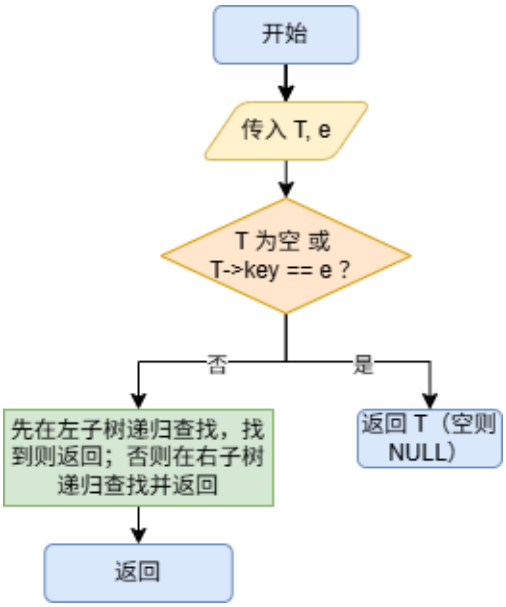


图 2-7　LocateNode 的程序流程图

7. status Assign(BiTree T, KeyType e, TElemType value)

输入：二叉树根结点指针 T，原关键字 e，新结点数据 value

输出：状态码（OK, ERROR）

算法思想描述：修改关键字为 e 的结点数据。若新关键字与原关键字不同且已存在于树中，则修改失败，严格保证树中关键字的唯一性。

算法处理步骤：

(1) 调用 LocateNode 查找关键字为 e 的结点 p。

(2) 若 p 为 NULL，返回 ERROR 表示未找到目标结点。

(3) 若 value.key 不等于 e 且 LocateNode(T, value.key) 不为 NULL，返回 ERROR 表示新关键字已存在。

(4) 将 value 赋值给 p 的数据域，完成修改。

(5) 返回 OK 表示修改成功。

时间复杂度：O(n)，n 为二叉树结点数，主要耗时在两次查找操作。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

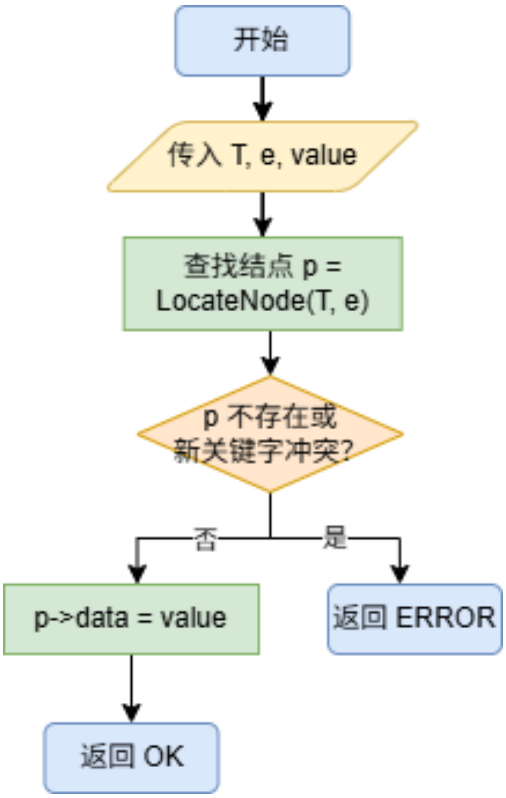


图 2-8　Assign 的程序流程图

8. BiTNode* GetSibling(BiTree T, KeyType e)

输入：二叉树根结点指针 T，关键字 e

输出：兄弟结点指针，无兄弟或未找到返回 NULL

算法思想描述：先通过非递归栈方式查找目标结点及其父结点，再根据目标结点是左孩子还是右孩子，返回对应的另一个孩子作为兄弟结点。根结点无兄弟结点。

算法处理步骤：

(1) 若 T 为空或 T 的数据域 key 等于 e，返回 NULL（树为空或目标是根结点）。

(2) 调用 FindParent 查找关键字为 e 的结点 cur 及其父结点 parent。

(3) 若 cur 或 parent 为 NULL，返回 NULL 表示未找到或无父结点。

(4) 若 cur 是 parent 的左孩子，返回 parent 的右孩子；否则返回 parent 的左孩子。

时间复杂度：O(n)，n 为二叉树结点数，主要耗时在 FindParent 操作。

空间复杂度：O(n)，最坏情况下栈需存储所有结点。

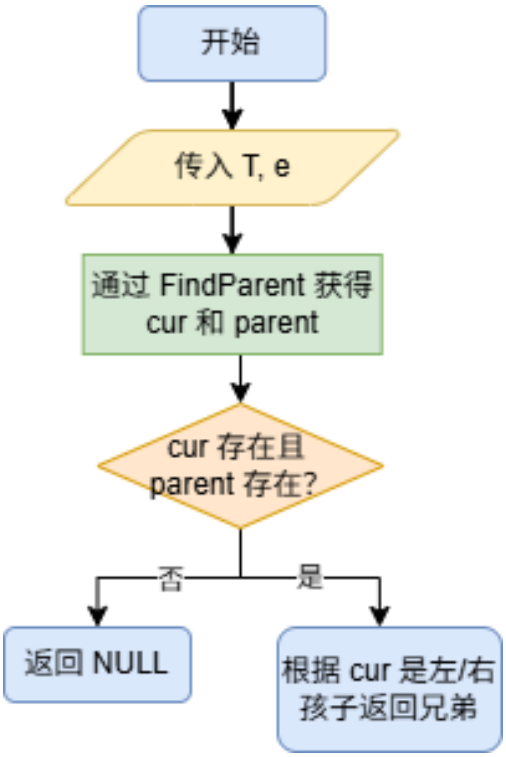


图 2-9　GetSibling 的程序流程图

9. status InsertNode(BiTree *T, KeyType e, int LR, TElemType c)

输入：二叉树根结点指针的指针 T，父结点关键字 e，插入位置 LR（-1 根/0 左/1 右），待插入结点 c

输出：状态码（OK, ERROR）

算法思想描述：在指定位置插入新结点 c，支持三种插入模式：

- LR=-1：作为新根结点插入，原根结点作为 c 的右子树

- LR=0：作为关键字为 e 的结点的左孩子插入，e 原有左子树作为 c 的右子树

- LR=1：作为关键字为 e 的结点的右孩子插入，e 原有右子树作为 c 的左子树

插入前检查关键字唯一性，确保无重复。

算法处理步骤：

(1) 若 LocateNode(*T, c.key) 不为 NULL，返回 ERROR 表示关键字重复。

(2) 为新结点分配大小为 BiTNode 结构体的内存空间。

(3) 将 c 赋值给新结点的数据域，左右孩子指针初始化为 NULL。

(4) 若 LR == -1（作为新根）：

a. 新结点的右孩子 = 原根 *T

b. *T = 新结点

c. 返回 OK

(5) 调用 LocateNode 查找父结点 fa。

(6) 若 fa 为 NULL，释放新结点内存，返回 ERROR 表示未找到父结点。

(7) 若 LR == 0（左孩子）:

a. 新结点的右孩子 = fa 原有左孩子

b. fa 的左孩子 = 新结点

(8) 否则（LR == 1，右孩子）:

a. 新结点的左孩子 = fa 原有右孩子

b. fa 的右孩子 = 新结点

(9) 返回 OK 表示插入成功。

时间复杂度：O(n)，n 为二叉树结点数，主要耗时在查找父结点操作。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

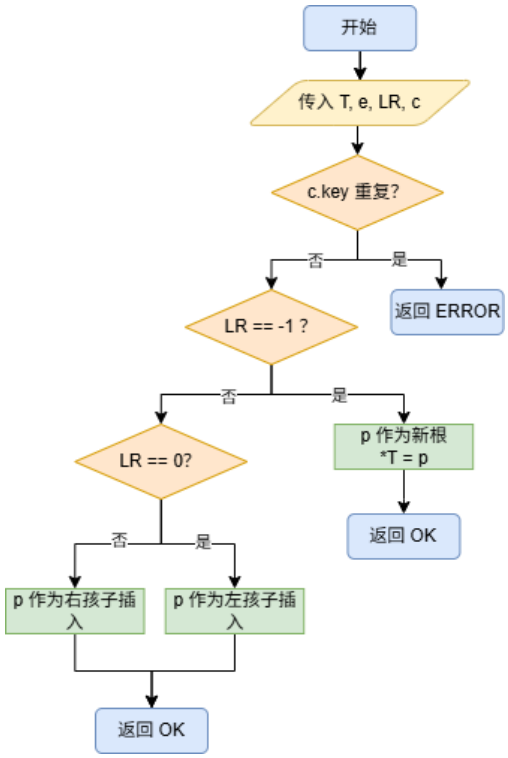


图 2-10　InsertNode 的程序流程图

10. status DeleteNode(BiTree *T, KeyType e)

输入：二叉树根结点指针的指针 T，关键字 e

输出：状态码（OK, ERROR）

算法思想描述：递归删除关键字为 e 的结点，根据结点度数分三种情况处理：

- 度为 0（叶子结点）：直接删除
- 度为 1：用唯一的孩子结点替代被删除结点
- 度为 2：用左孩子替代被删除结点，将右子树挂到左子树的最右结点下

算法处理步骤：

(1) 若 *T 为空，返回 ERROR 表示树为空或未找到结点。

(2) 若 (*T)->data.key == e：

a. 定义 del 指针指向 *T，保存待删除结点地址。

b. 若 *T 无左右孩子（度为 0）：*T = NULL

c. 否则若 *T 无左孩子（度为 1）：*T = (*T)->rchild

d. 否则若 *T 无右孩子（度为 1）：*T = (*T)->lchild

e. 否则（度为 2）：

i. 调用 GetRightMost 找到左子树的最右结点 rightMost

ii. rightMost->rchild = (*T)->rchild

iii. *T = (*T)->lchild

f. 释放 del 指向的内存

g. 返回 OK

(3) 递归调用 DeleteNode 在左子树中删除，若成功返回 OK。

(4) 递归调用 DeleteNode 在右子树中删除并返回结果。

时间复杂度：O(n)，n 为二叉树结点数，最坏情况下需要遍历所有结点。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

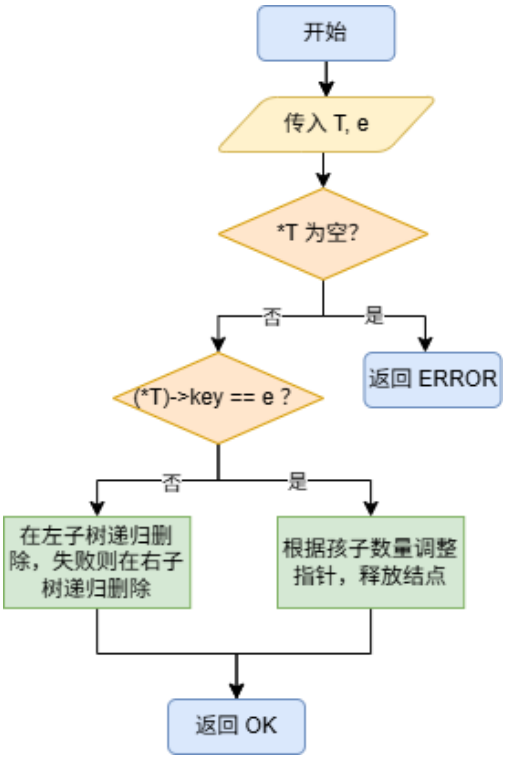


图 2-11　DeleteNode 的程序流程图

11. status PreOrderTraverse(BiTree T, status (*visit)(BiTree))

输入：二叉树根结点指针 T，访问函数指针 visit（对结点进行操作的函数，返回状态码）

输出：状态码（OK, ERROR）

算法思想描述：采用非递归栈方式实现先序遍历（根-左-右），利用栈”后进先出”的特性保证左孩子优先处理。对每个结点调用 visit 函数一次且仅一次，若 visit 调用失败则立即终止遍历并返回 ERROR。

算法处理步骤：

(1) 若 T 为空，返回 OK 表示空树无需遍历。

(2) 初始化栈 stack，栈顶 top 为-1，将根结点 T 压入栈。

(3) 当栈不为空时：

a. 弹出栈顶元素 p。

b. 若调用 visit(p) 返回 ERROR，立即返回 ERROR。

c. 若 p 有右孩子，将右孩子压入栈。

d. 若 p 有左孩子，将左孩子压入栈。

(4) 返回 OK 表示遍历完成。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅访问一次。

空间复杂度：O(n)，最坏情况下（单链树）栈需存储所有结点。

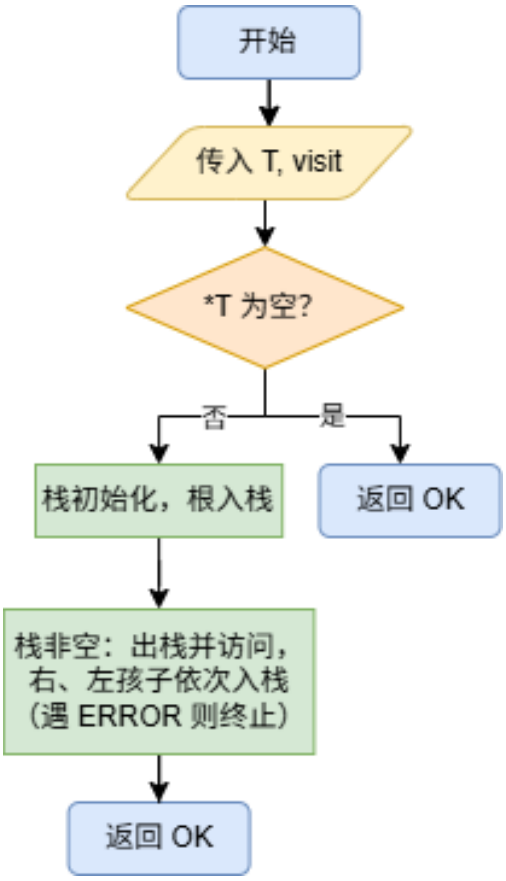


图 2-12　PreOrderTraverse 的程序流程图

12. status InOrderTraverse(BiTree T, status (*visit)(BiTree))

输入：二叉树根结点指针 T，访问函数指针 visit（对结点进行操作的函数，返回状态码）输出：状态码（OK, ERROR）

算法思想描述：采用递归方式实现中序遍历（左-根-右），先遍历左子树，再访问根结点，最后遍历右子树。对每个结点调用 visit 函数一次且仅一次，若 visit 调用失败则立即终止遍历。

算法处理步骤：

(1) 若 T 为空，返回 OK 表示空树无需遍历。

(2) 若调用 InOrderTraverse(T->lchild, visit) 返回 ERROR，返回 ERROR。

(3) 若调用 visit(T) 返回 ERROR，返回 ERROR。

(4) 若调用 InOrderTraverse(T->rchild, visit) 返回 ERROR，返回 ERROR。

(5) 返回 OK 表示遍历完成。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅访问一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

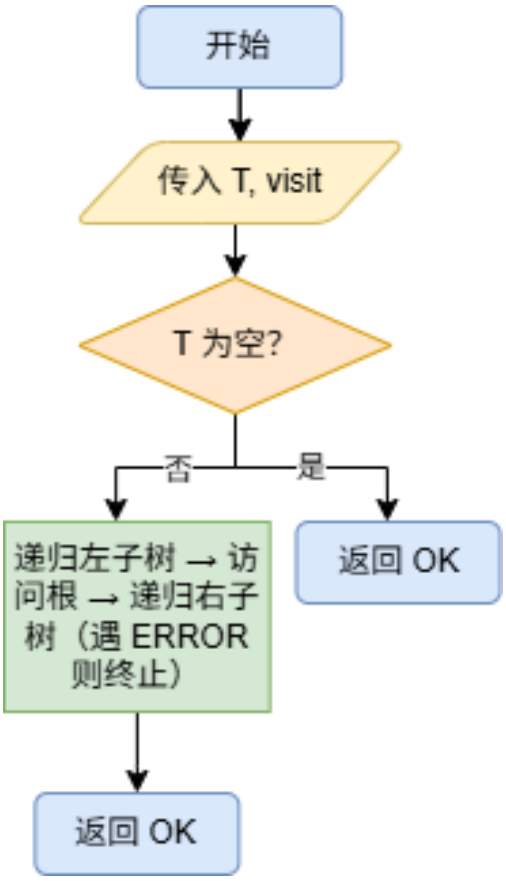


图 2-13　InOrderTraverse 的程序流程图

13. status PostOrderTraverse(BiTree T, status (*visit)(BiTree))

输入：二叉树根结点指针 T，访问函数指针 visit（对结点进行操作的函数，返回状态码）

输出：状态码（OK, ERROR）

算法思想描述：采用递归方式实现后序遍历（左-右-根），先遍历左子树，再遍历右子树，最后访问根结点。对每个结点调用 visit 函数一次且仅一次，若 visit 调用失败则立即终止遍历。

算法处理步骤：

(1) 若 T 为空，返回 OK 表示空树无需遍历。

(2) 若调用 PostOrderTraverse(T->lchild, visit) 返回 ERROR，返回 ERROR。

(3) 若调用 PostOrderTraverse(T->rchild, visit) 返回 ERROR，返回 ERROR。

(4) 若调用 visit(T) 返回 ERROR，返回 ERROR。

(5) 返回 OK 表示遍历完成。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅访问一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

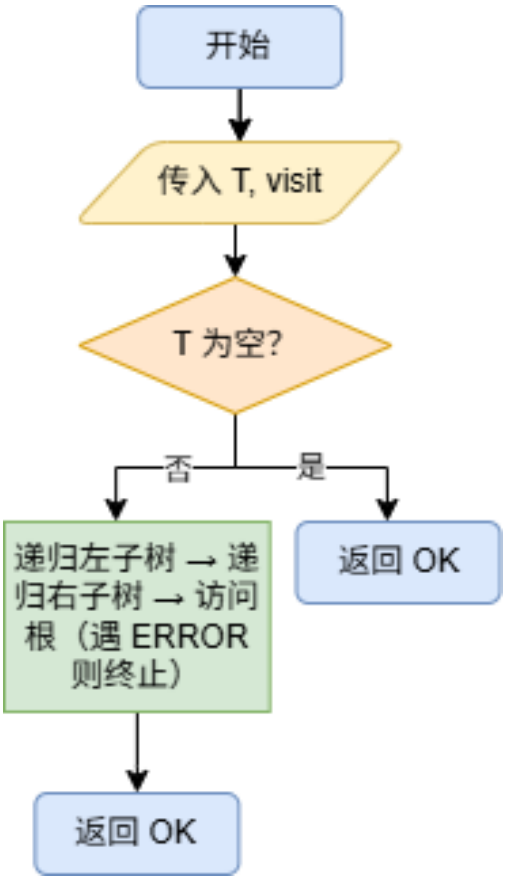


图 2-14　PostOrderTraverse 的程序流程图

14. status LevelOrderTraverse(BiTree T, status (*visit)(BiTree))

输入：二叉树根结点指针 T，访问函数指针 visit（对结点进行操作的函数，返回状态码）

输出：状态码（OK, ERROR）

算法思想描述：采用队列实现层序遍历（按层次从上到下、从左到右），利用队列” 先进先出” 的特性保证按层次顺序访问。对每个结点调用 visit 函数一次且仅一次，若 visit 调用失败则立即终止遍历。

算法处理步骤：

(1) 若 T 为空，返回 OK 表示空树无需遍历。

(2) 初始化队列 que，队头 front=0，队尾 rear=0，将根结点 T 入队。

(3) 当 front < rear 时：

a. 出队元素 p。

b. 若调用 visit(p) 返回 ERROR，立即返回 ERROR。

c. 若 p 有左孩子，将左孩子入队。

d. 若 p 有右孩子，将右孩子入队。

(4) 返回 OK 表示遍历完成。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅访问一次。

空间复杂度：O(n)，最坏情况下（完全二叉树最后一层）队列需存储约 n/2 个结点。

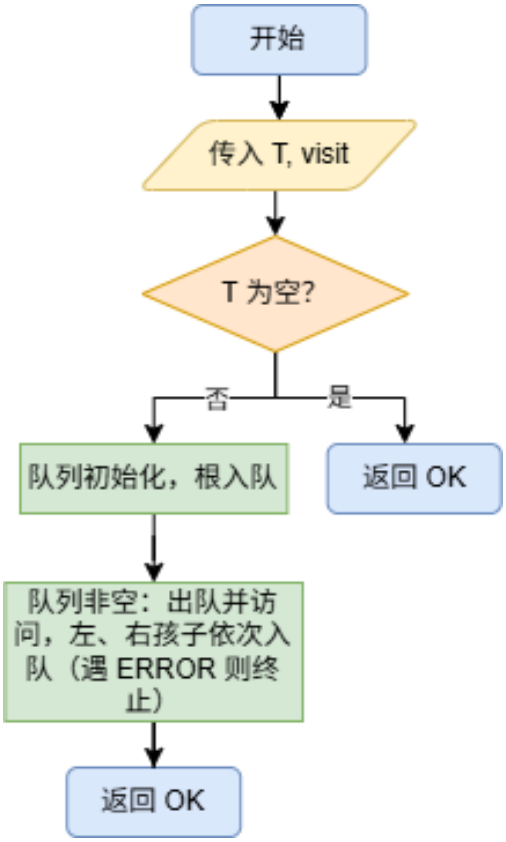


图 2-15　LevelOrderTraverse 的程序流程图

二、扩展功能（5 种）

15. int MaxPathSum(BiTree T)

输入：二叉树根结点指针 T

输出：根到叶子的最大路径和，空树返回 0

算法思想描述：递归计算从根结点到任意叶子结点的路径中，关键字之和最大的那条路径的和。路径必须从根结点开始，到叶子结点结束。

算法处理步骤：

(1) 若 T 为空，返回 0 表示空树路径和为 0。

(2) 递归计算左子树的最大路径和 left。

(3) 递归计算右子树的最大路径和 right。

(4) 返回 left 和 right 中的较大值加上 T->data.key。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅访问一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

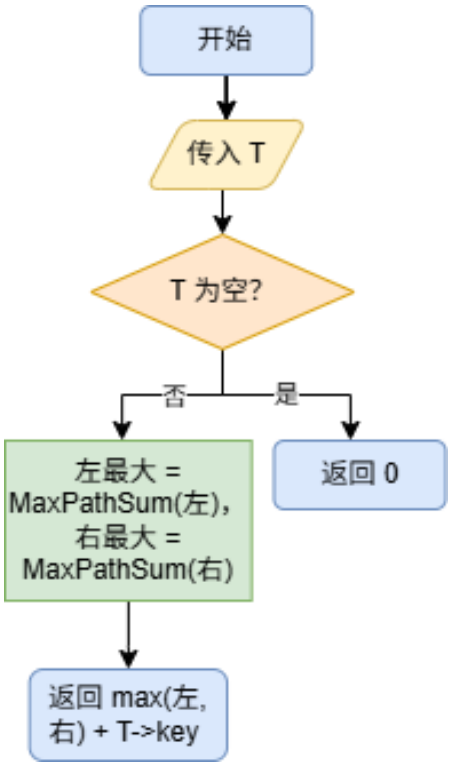


图 2-16　MaxPathSum 的程序流程图

16. BiTNode* LowestCommonAncestor(BiTree T, KeyType e1, KeyType e2)

输入：二叉树根结点指针 T，两个关键字 e1 和 e2

输出：最近公共祖先结点指针，无公共祖先返回 NULL

算法思想描述：递归查找两个结点的最近公共祖先（LCA）：

- 若当前结点是其中一个结点，则当前结点就是 LCA
- 若左右子树分别找到一个结点，则当前结点是 LCA
- 否则返回左右子树中找到的非空结果

最近公共祖先可以是 e1 或 e2 本身。

算法处理步骤：

(1) 若 T 为空，返回 NULL 表示无公共祖先。

(2) 若 T->data.key == e1 || T->data.key == e2，返回 T。

(3) 递归调用 LowestCommonAncestor 在左子树中查找，结果存入 left。

(4) 递归调用 LowestCommonAncestor 在右子树中查找，结果存入 right。

(5) 若 left != NULL && right != NULL，返回 T。

(6) 返回 left != NULL ? left : right。

时间复杂度：O(n)，n 为二叉树结点数，最坏情况下需要遍历所有结点。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

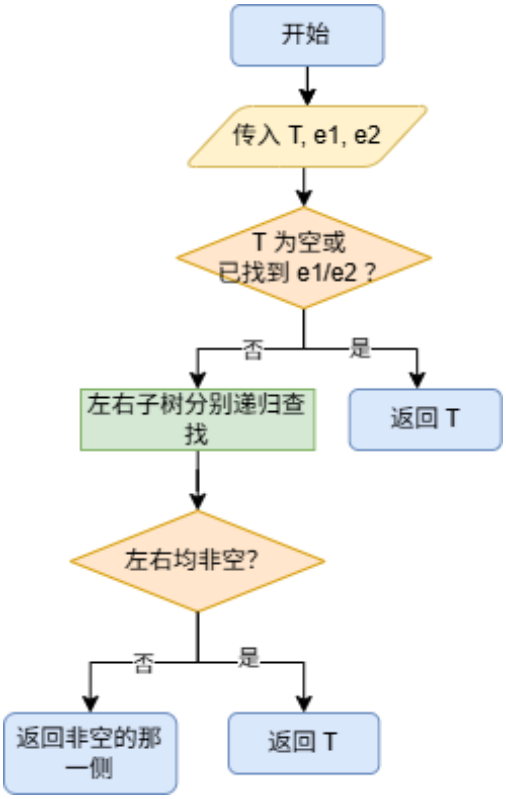


图 2-17　LowestCommonAncestor 的程序流程图

17. status InvertTree(BiTree T)

输入：二叉树根结点指针 T

输出：状态码（OK, ERROR）

算法思想描述：递归翻转二叉树，交换每个结点的左右孩子指针，实现整棵树的镜像翻转。翻转后原左子树变为右子树，原右子树变为左子树。

算法处理步骤：

(1) 若 T 为空，返回 OK 表示空树无需翻转。

(2) 交换 T 的左孩子和右孩子指针。

(3) 递归调用 InvertTree 翻转左子树。

(4) 递归调用 InvertTree 翻转右子树。

(5) 返回 OK 表示翻转完成。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅交换一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

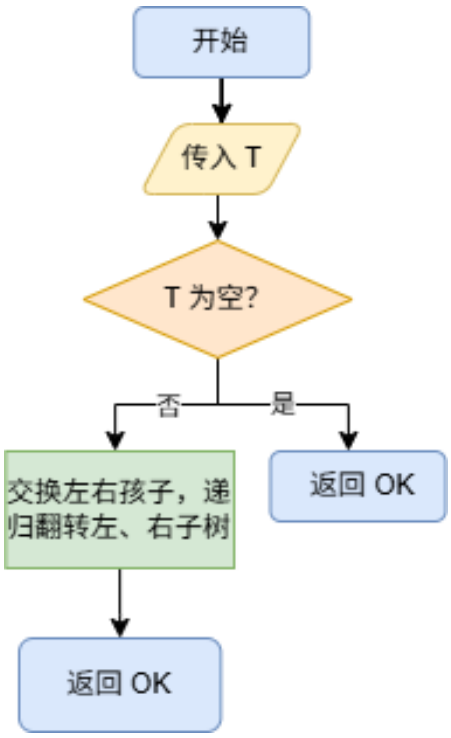


图 2-18　InvertTree 的程序流程图

18. status SaveBiTree(BiTree T, const char fname[])

status LoadBiTree(BiTree *T, const char fname[])

(1) SaveBiTree

输入：二叉树根结点指针 T，保存文件名 fname

输出：状态码（OK, ERROR）

算法思想描述：将二叉树完整保存到文件中，采用先序遍历顺序写入，空结点用”0 null” 表示，文件末尾写入结束标记”-1 end”。该格式完整保留了树的逻辑结构 (D,R)，可通过 LoadBiTree 完全恢复原树的所有数据和结构。

算法处理步骤：

(1) 以写入模式打开文件 fname，获取文件指针 fp。

(2) 若 fp 为 NULL，返回 ERROR 表示文件打开失败。

(3) 递归按先序顺序写入结点数据：

a. 若结点为空，写入”0 null ”。

b. 否则写入结点的 key 和 others，格式为”

(4) 写入结束标记”-1 end”。

(5) 关闭文件。

(6) 返回 OK 表示保存成功。

(2) LoadBiTree

输入：二叉树根结点指针的指针 T，读取文件名 fname

输出：状态码（OK, ERROR）

算法思想描述：从文件中读取保存的二叉树数据，按照先序序列重建二叉树，完全恢复原树的逻辑结构和数据。重建过程中自动检查关键字唯一性，确保树的正确性。

算法处理步骤：

(1) 以读取模式打开文件 fname，获取文件指针 fp。

(2) 若 fp 为 NULL，返回 ERROR 表示文件打开失败。

(3) 将全局变量 inputEnd 设为 0，重置读取结束标记。

(4) 递归按先序顺序读取结点数据重建树：

a. 读取关键字 k 和字符串 s。

b. 若 k == 0，创建空结点。

c. 若 k == -1，结束重建。

d. 否则创建结点并赋值，递归创建左右子树。

(5) 关闭文件。

(6) 返回 OK 表示加载成功。

时间复杂度：O(n)，n 为二叉树结点数，每个结点仅读写一次。

空间复杂度：O(h)，h 为二叉树高度，递归调用栈深度等于树高。

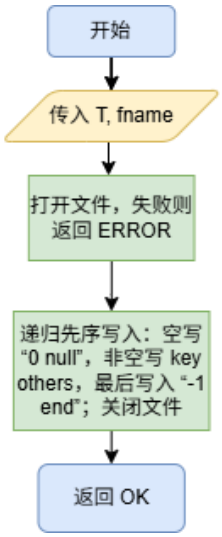


图 2-19　SaveBiTree 的程序流程图

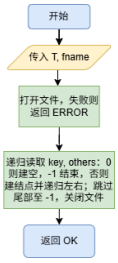


图 2-20　LoadBiTree 的程序流程图

19. 多二叉树管理函数

采用顺序表管理多个二叉树，顺序表的每个数据元素包含二叉树的名称和根结点指针，支持添加、删除、切换和显示操作，可自由切换到任意二叉树并执行所有单二叉树操作。

(1) status AddTree(const char name[])

输入：新二叉树名称 name

输出：状态码（OK, ERROR）

算法思想：在多二叉树管理顺序表中添加一个新的空二叉树，指定名称。若名称已存在或顺序表已满（最多 10 棵）则返回 ERROR。

算法处理步骤：

(1) 检查顺序表是否已满（treeCnt >= MAX_TREE），若已满返回 ERROR。

(2) 检查名称 name 是否已存在，若存在返回 ERROR。

(3) 将 name 复制到 treeNames[treeCnt] 数组中。

(4) 将 trees[treeCnt] 置为 NULL（空树）。

(5) treeCnt 加 1。

(6) 返回 OK 表示添加成功。

时间复杂度：O(m)，m 为当前二叉树数量，主要耗时在名称检查操作。

空间复杂度：O(1)，无需额外空间。

(2) status RemoveTree(const char name[])

输入：待删除二叉树名称 name

输出：状态码（OK, ERROR）

算法思想：删除指定名称的二叉树，先调用 DestroyBiTree 销毁其所有结点，再从顺序表中移除该条目。若删除的是当前操作树，则自动切换到第一个树（若存在）。

算法处理步骤：

(1) 遍历顺序表查找名称为 name 的二叉树索引 i。

(2) 若未找到返回 ERROR。

(3) 调用 DestroyBiTree 销毁 trees[i]。

(4) 将顺序表中 i 之后的所有元素向前移动一位。

(5) treeCnt 减 1。

(6) 若当前选中的树 curSel == i，将 curSel 置为 0（若 treeCnt > 0），否则置为-1。

(7) 返回 OK 表示删除成功。

时间复杂度：O(m + n)，m 为当前二叉树数量，n 为被删除树的结点数。

空间复杂度：O(h)，h 为被删除树的高度，递归调用栈深度等于树高。

(3) status SwitchTree(const char name[])

输入：目标二叉树名称 name

输出：状态码（OK, ERROR）

算法思想：将当前操作树切换为指定名称的树。后续所有单二叉树操作都将作用于该树。

算法处理步骤：

(1) 遍历顺序表查找名称为 name 的二叉树索引 i。

(2) 若未找到返回 ERROR。

(3) 将 curSel 置为 i。

(4) 返回 OK 表示切换成功。

时间复杂度：O(m)，m 为当前二叉树数量，主要耗时在名称查找操作。

空间复杂度：O(1)，无需额外空间。

(4) void ShowAllTrees()

输入：无

输出：无

算法思想：显示所有已创建的二叉树列表，标记当前操作的树，方便用户选择和切换。

算法处理步骤：

(1) 若 treeCnt == 0，打印” 暂无二叉树” 并返回。(2) 遍历顺序表，打印每个二叉树的序号、名称。

(3) 若当前树 curSel == i，在该行末尾打印”← 当前操作”。

时间复杂度：O(m)，m 为当前二叉树数量。

空间复杂度：O(1)，无需额外空间。

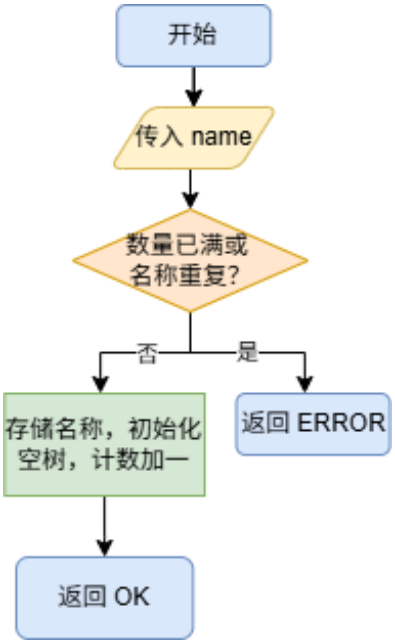


图 2-21　AddTree 的程序流程图

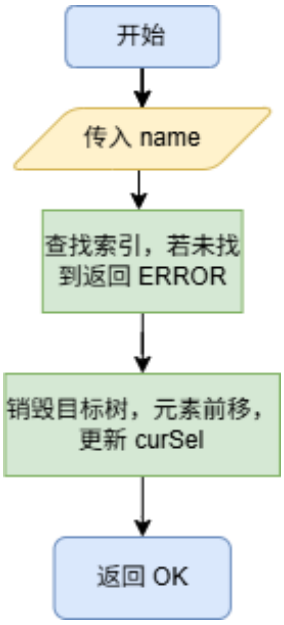


图 2-22　RemoveTree 的程序流程图

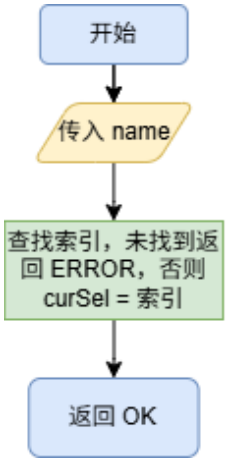


图 2-23　SwitchTree 的程序流程图

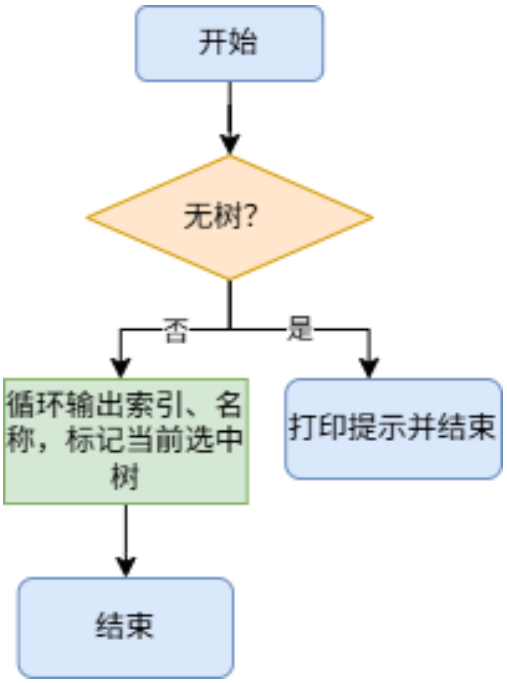


图 2-24　ShowAllTrees 的程序流程图

2.4　系统测试

在 main 函数中构建功能演示菜单，并对主要功能进行测试。

```
==============================二叉链表二叉树操作系统==============================
0.退出程序       1.创建树         2.切换二叉树     3.销毁当前树
4.清空当前树     5.判断空树       6.求树深度       7.关键字查找结点
8.修改结点数据   9.查询兄弟结点   10.插入结点      11.删除结点
12.先序遍历      13.中序遍历      14.后序遍历      15.层序遍历      16.根叶最大路径和
17.最近公共祖先  18.翻转二叉树    19.保存二叉树到文件               20.从文件读取二叉树
================================================================================
请选择功能：
```

图 2-25　二叉树功能演示菜单

1. 创建树测试

(1) 测试数据

表 2-1　功能 1 的测试数据

测试用例	程序输入	理论结果
正常创建树	选择 1，输入先序序列：1 a 2 b 0 null 0 null 3 c 0 null 0 null -1 end，树名：treeA	创建成功，当前树为 treeA
空树创建	选择 1，输入先序序列：0 null -1 end，树名：emptyTree	创建成功，树为空
树数量上限	已创建 10 棵树，再次选择 1	提示“二叉树数量已满”

(2) 运行结果截图

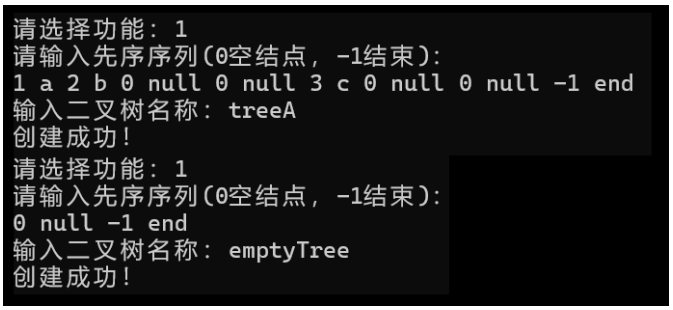


图 2-26　创建树测试结果

2. 切换二叉树测试

(1) 测试数据

表 2-2　功能 2 的测试数据

测试用例	程序输入	理论结果
正常切换	已有两棵树，选择 2，输入序号 1	切换到第二棵树，后续操作针对该树
无效序号	选择 2，输入序号 99	提示“无效序号”，当前树不变
无树切换	未创建任何树时选择 2	提示“暂无二叉树可切换”

(2) 运行结果截图



图 2-27　切换二叉树测试结果

3. 销毁当前树测试

(1) 测试数据

表 2-3　功能 3 的测试数据

测试用例	程序输入	理论结果
销毁非空树	当前树非空，选择 3	树被彻底销毁，当前树变为空指针
未选择树	未创建任何树时选择 3	提示“请先创建二叉树”

(2) 运行结果截图

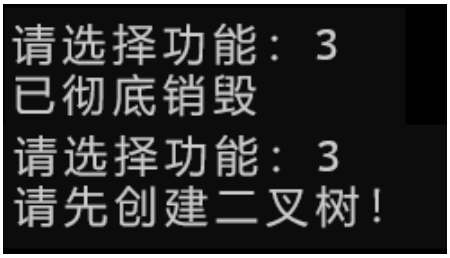


图 2-28　销毁当前树测试结果

4. 清空当前树测试

(1) 测试数据

表 2-4　功能 4 的测试数据

测试用例	程序输入	理论结果
清空非空树	当前树非空，选择 4	树变为空树
清空空树	当前树已为空，选择 4	仍为空树，操作成功

(2) 运行结果截图

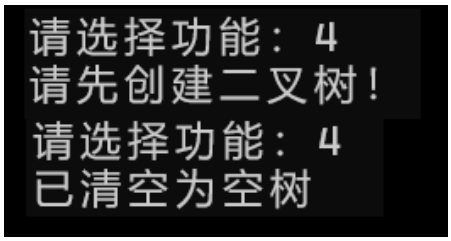


图 2-29　清空当前树测试结果

5. 判断空树测试

(1) 测试数据

表 2-5　功能 5 的测试数据

测试用例	程序输入	理论结果
空树判断	当前树为空，选择 5	输出“是空树”
非空树判断	当前树非空，选择 5	输出“非空树”

(2) 运行结果截图

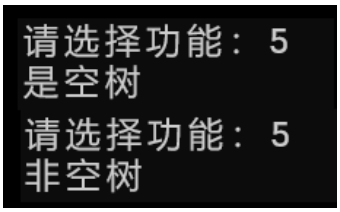


图 2-30　判断空树测试结果

6. 求树深度测试

(1) 测试数据

表 2-6　功能 6 的测试数据

测试用例	程序输入	理论结果
单结点树	树只有一个根结点，选择 6	深度：1
三层满二叉树	树为满二叉树高度 3，选择 6	深度：3
空树	当前树为空，选择 6	深度：0

(2) 运行结果截图



图 2-31　求树深度测试结果

7. 关键字查找结点测试

(1) 测试数据

表 2-7　功能 7 的测试数据

测试用例	程序输入	理论结果
查找存在结点	树中有关键字 5 的结点，输入 5	输出结点数据
查找不存在结点	输入 99	输出“不存在该结点”
空树查找	当前树为空，输入任意关键字	输出“不存在该结点”

(2) 运行结果截图

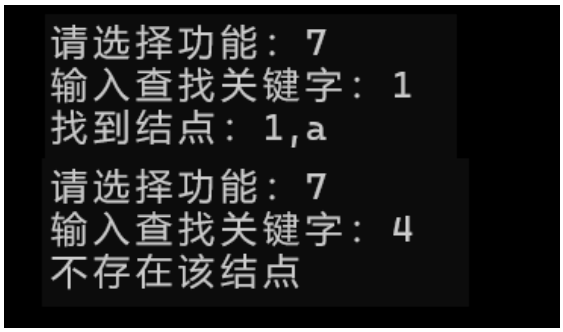


图 2-32　关键字查找结点测试结果

8. 修改结点数据测试

(1) 测试数据

表 2-8　功能 8 的测试数据

测试用例	程序输入	理论结果
修改存在结点	原关键字 1，新关键字 5，新内容“new”	修改成功，结点变为 10,new
修改结点关键字与已有冲突	原关键字 1，新关键字 2 与另一结点相同	修改失败
修改不存在结点	原关键字 99，新关键字 10	修改失败

(2) 运行结果截图

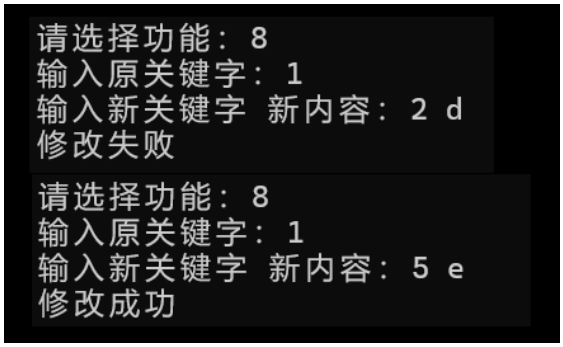


图 2-33　修改结点数据测试结果

9. 查询兄弟结点测试

(1) 测试数据

表 2-9　功能 9 的测试数据

测试用例	程序输入	理论结果
存在兄弟	树中结点 2 有兄弟结点 3，输入 2	输出兄弟结点数据
根结点	输入根结点关键字	输出“无兄弟结点”
无兄弟结点	输入只有一个孩子的结点关键字	输出“无兄弟结点”
结点不存在	输入 99	输出“无兄弟结点”

(2) 运行结果截图

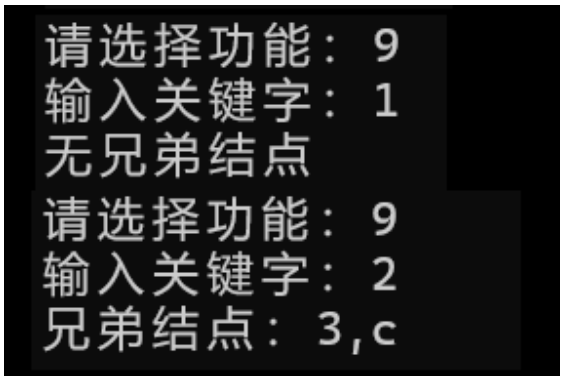


图 2-34　查询兄弟结点测试结果

10. 插入结点测试

(1) 测试数据

表 2-10　功能 10 的测试数据

测试用例	程序输入	理论结果
插入为根	空树，输入父结点关键字 0，位置-1，新结点 (5,hello)	树根变为 5,hello
插入为左孩子	树根 5 下有左孩子位置空，输入父结点 1，位置 0，新结点 (4,left)	成为 1 的左孩子
插入为右孩子	输入父结点 5，位置 1，新结点 (3,right)	成为 1 的右孩子
父结点不存在	输入父结点 1	插入失败
关键字重复	插入的关键字已存在	插入失败

(2) 运行结果截图

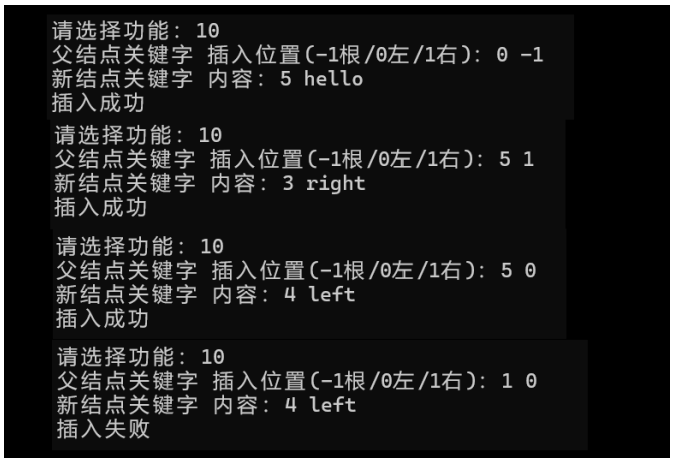


图 2-35　插入结点测试结果

11. 删除结点测试

(1) 测试数据

表 2-11　功能 11 的测试数据

测试用例	程序输入	理论结果
删除叶子结点	删除叶子结点 5	结点消失，树结构正确
删除只有左孩子的结点	删除结点 2（有左子树）	其左子树提升到当前位置
删除只有右孩子的结点	删除结点 3（有右子树）	其右子树提升
删除有两个孩子的结点	删除根结点（左右孩子均存在）	左子树的最右结点接上右子树，左子树成为新根
删除不存在结点	输入 99	删除失败

(2) 运行结果截图

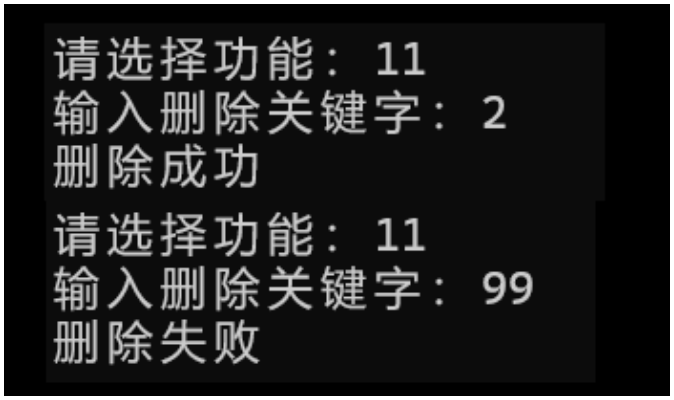


图 2-36　删除结点测试结果

12. 先序遍历测试

(1) 测试数据

表 2-12　功能 12 的测试数据

测试用例	程序输入	理论结果
非空树先序	树为：1(2,3)，选择 12	输出：1,node1 2,node2 3,node3
空树先序	空树选择 12	无输出

(2) 运行结果截图

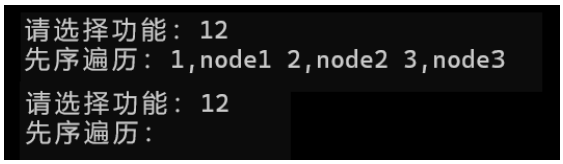


图 2-37　先序遍历测试结果

13. 中序遍历测试

(1) 测试数据

表 2-13　功能 13 的测试数据

测试用例	程序输入	理论结果
非空树中序	树为：1(2,3)，选择 13	输出：2,node2 1,node1 3,node3
空树中序	空树选择 13	无输出

(2) 运行结果截图

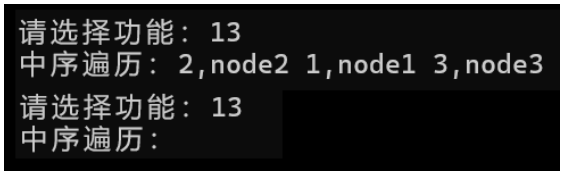


图 2-38　中序遍历测试结果

14. 后序遍历测试

(1) 测试数据

表 2-14　功能 14 的测试数据

测试用例	程序输入	理论结果
非空树后序	树为：1(2,3)，选择 14	输出：2,node2 3,node3 1,node1
空树后序	空树选择 14	无输出

(2) 运行结果截图

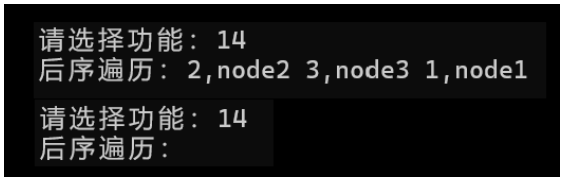


图 2-39　后序遍历测试结果

15. 层序遍历测试

(1) 测试数据

表 2-15　功能 15 的测试数据

测试用例	程序输入	理论结果
非空树层序	树为：1(2,3)，选择 15	输出：1,node1 2,node2 3,node3
空树层序	空树选择 15	无输出

(2) 运行结果截图

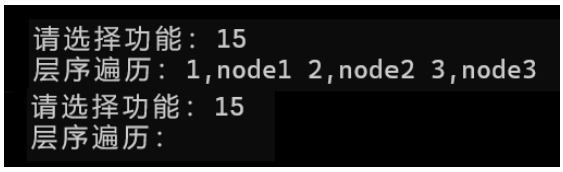


图 2-40　层序遍历测试结果

16. 根叶最大路径和测试

(1) 测试数据

表 2-16　功能 16 的测试数据

测试用例	程序输入	理论结果
单结点树	根结点值为 10，选择 16	最大路径和：10
复杂树	树：5(4,8) 其中 4 有左子-3，8 有右子 2，选择 16	路径 5+8+2=15
空树	空树选择 16	0

(2) 运行结果截图

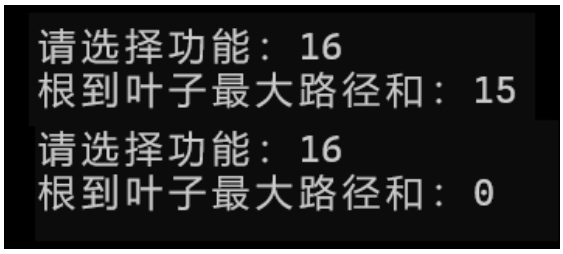


图 2-41　根叶最大路径和测试结果

17. 最近公共祖先测试

(1) 测试数据

表 2-17　功能 17 的测试数据

测试用例	程序输入	理论结果
两个结点都在树中	树：1(2,3(4,5))，输入 4 和 5	公共祖先：3
一个结点是另一个的祖先	输入 2 和 4	公共祖先：2
结点不存在	输入 99 和 100	输出“无公共祖先”
空树	空树输入任意	输出“无公共祖先”

(2) 运行结果截图

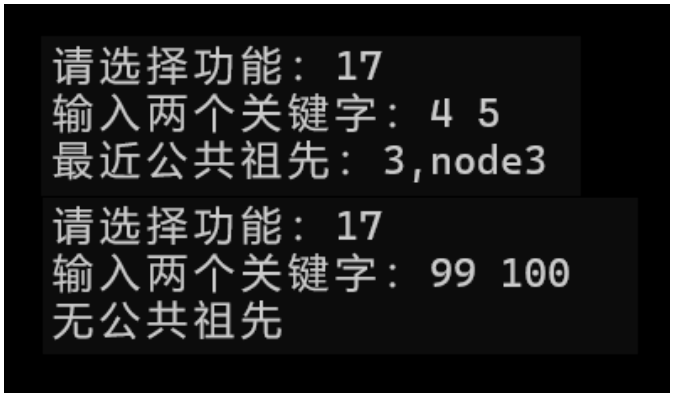


图 2-42　最近公共祖先测试结果

18. 翻转二叉树测试

(1) 测试数据

表 2-18　功能 18 的测试数据

测试用例	程序输入	理论结果
非空树翻转	树为：1(2,3)，选择 18 后再中序遍历	原中序 2 1 3 变为 3 1 2
空树翻转	空树选择 18	无变化，操作成功

(2) 运行结果截图

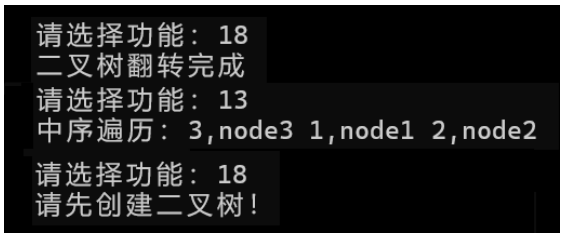


图 2-43　翻转二叉树测试结果

19. 保存二叉树到文件测试

(1) 测试数据

表 2-19　功能 19 的测试数据

测试用例	程序输入	理论结果
正常保存	当前树非空，输入文件名“tree.dat”	保存成功，文件内容为先序序列
保存空树	空树，输入文件名“empty.dat”	保存成功，文件内容为“0 null -1 end”
保存失败（如目录只读）	输入非法路径	保存失败

(2) 运行结果截图



图 2-44　保存二叉树到文件测试结果

20. 从文件读取二叉树测试

(1) 测试数据

表 2-20　功能 20 的测试数据

测试用例	程序输入	理论结果
正常读取	选择 20，输入之前保存的“tree.dat”，输入树名“restored”	读取成功，恢复原树
文件不存在	输入“noexist.dat”	读取失败
读取后树数量已满	已有 10 棵树时再读取	提示“二叉树数量已满”

(2) 运行结果截图

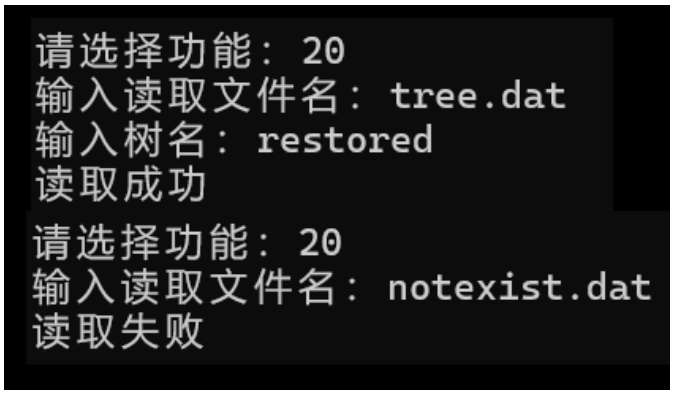


图 2-45　从文件读取二叉树测试结果

2.5　实验小结

本次实验以二叉链表为存储结构，实现了二叉树的基本运算与多项扩展功能。从基础创建遍历到算法优化，再到多树管理与文件持久化，整个过程涉及大量递归与指针操作，调试过程中遇到了若干典型问题，现总结如下。

1. 递归创建二叉树时的输入处理

在实现 CreateBiTree 函数时，要求根据给定的先序序列（带空子树标记）构造二叉树。起初我采用标准输入流 scanf 逐项读取，但发现当用户输入中包含空格（如附加信息字符串”hello world”）时，scanf(”%s”) 会在空格处截断，导致数据错位。解决方法是改用 fgets 按行读取整串定义，再通过字符串解析函数（如 sscanf 或自定义解析）逐个提取关键字与附加信息。同时，为满足“不在函数内部输入二叉树定义”的要求，我将解析逻辑与外部输入分离，主函数中准备好定义字符串后直接传递给 CreateBiTree，避免了函数内部依赖控制台输入的问题。

2. 插入结点的特殊场景处理

InsertNode 函数支持三种模式：作为新根插入（LR=-1）、作为左孩子插入（LR=0）、作为右孩子插入（LR=1）。在实现时，我遗漏了“被替换的子树应接入新结点的特定位置”这一逻辑。例如，当插入左孩子时，父结点原有的左子树应成为新结点的右子树（而非丢弃）。最初我简单地将父结点左孩子指向新结点，未处理原左子树，导致子树丢失。通过查阅实验要求中的说明，修正了这一逻辑：插入左孩子时，新结点的右孩子 = 父结点的原左孩子；插入右孩子时，新结点的左孩子 = 父结点的原右孩子；插入为根时，新结点的右孩子 = 原根结点。该修正确保了树结构的完整性。

3. 删除度为 2 的结点时左子树最右结点的维护

DeleteNode 函数对度为 2 的结点采用“左孩子替代，右子树挂到左子树最右结点下”的策略。在实现过程中，我误将最右结点指针的更新方向弄反，导致右子树被错误地挂到了左子树最左结点下，破坏了二叉搜索树的有序性（虽然本实验未要求二叉搜索树，但插入规则仍依赖此结构）。排查后发现 GetRightMost 函数递归查找右孩子时未正确处理空指针情况，同时替换后忘记将原结点的左右孩子指针置空，造成内存泄漏。修正方法：先递归找到左子树的最右结点（即不断向右遍历直到 rchild == NULL），然后将其 rchild 指向待删除结点的右子树，最后将待删除结点的左孩子提升为根，释放原结点内存。

4. 非递归先序遍历中栈操作的顺序

实验要求至少一种遍历采用非递归实现，我选择了先序遍历。初始实现时，我按照“先压左后压右”的顺序入栈，结果输出顺序错误（变成了中序）。分析发现，先序遍历的访问顺序是“根 → 左 → 右”，而栈是后进先出的，因此应当先压右孩子、再压左孩子，这样弹出时左孩子先被处理。修正入栈顺序后，遍历结果与递归版本完全一致。此外，在处理空树时，我忘记初始化栈顶指针为-1，导致第一次判断栈空时使用了未初始化的值，造成程序崩溃，修复后正常运行。

5. 最大路径和计算中的递归边界条件

MaxPathSum 函数要求从根结点到叶子结点的最大路径和。最初我采用的递归逻辑是 return T->data.key + max(MaxPathSum(T->lchild), MaxPathSum(T->rchild))，但测试时发现当树只有一个叶子结点时结果正确，而当结点有单子树时却出现了错误：若某结点左孩子为空、右孩子非空，按上述逻辑会计算 max(0, right_sum)，但路径必须延续到叶子，空子树不应该被视为有效路径。修正方案：当左子树为空时，只取右子树结果；当右子树为空时，只取左子树结果；当左右子树都为空时，返回当前结点关键字。这样避免了空子树的干扰，保证了路径必须结束于叶子结点。

6. 多树管理中的索引切换与删除同步

多二叉树管理模块中，RemoveTree 函数删除指定名称的树后，需更新当前操作树的下标 curSel。最初我未做处理，导致删除后 curSel 指向已删除的位置，后续操作访问越界。修改为：删除当前树时，将 curSel 置为 0（若还有其它树）；删除非当前树时，若删除的索引小于 curSel，则将 curSel 减 1。同时，在 SwitchTree 切换时，我未做名称重复检查，允许用户将多个树命名为相同名字，导致查找时仅返回第一个匹配项。后续在 AddTree 中增加了名称唯一性校验，并在 SwitchTree

中如果找到同名树则报错，保证每个名称唯一对应一棵树。

总体而言，本次实验通过实现二叉链表结构及其操作，加深了对递归思想、指针操作和树形数据结构的理解。上述各问题涵盖了输入解析、指针变更、栈队列应用、算法边界处理及多实例管理，每个问题的解决都经过了反复调试，积累了宝贵的经验，为后续更复杂的树形结构（如线索二叉树、哈夫曼树）打下了基础。

3 课程的收获和建议

3.1 基于顺序存储结构的线性表实现

收获：

通过第一个实验，我对“动态数组”的实现有了更深入的理解。以往学习 C 语言时对 `malloc` 和 `realloc` 仅停留在概念层面，本次亲自动手编写了顺序表的扩容逻辑——当 `length >= listsize` 时重新分配内存，由此切身体会到顺序表插入操作的时间复杂度为 $O(n)$ 的本质原因（需移动元素）。此外，`InitList` 函数中当 `L->elem != NULL` 时返回 `INFEASIBLE` 的设计，也让我认识到良好的接口设计应避免因重复初始化导致的内存泄漏。

遇到的问题：

- 在实现主函数菜单时，曾在 `switch` 分支中直接调用 `ListDelete` 两次（一次用于判断返回值，一次用于执行），导致逻辑被重复执行。修正方法是将函数返回值先存入变量，再根据变量值进行判断。
- 文件保存功能实现初期，仅调用 `fwrite` 写入数据，未将表长写入文件，导致加载时无法正确读取数据个数。修正方案为先写入长度，再写入数据数组。
- 多表管理中删除表后未及时更新当前操作表索引 `current`，导致后续操作访问已释放的位置。在 `RemoveList` 中增加索引调整逻辑后问题解决。

建议：

优化实验配套测试用例，补充边界场景数据，例如全负数数组、单元素数组、全正数数组，方便学生快速排查代码漏洞，提升自测效率。

3.2 基于链式存储结构的线性表实现

收获：

链表的指针操作是我在本实验中投入时间最多的环节。编写 `ListInsert` 时，通过绘制示意图辅助理解指针指向关系，显著减少了指针顺序错误的发生。另一重要收获是“头结点”的作用：引入头结点后，对第一个数据结点的插入与删除操作无需单独处理，统一采用从 `p = L` 开始向前寻找 $i-1$ 个结点的方式实现。

此外，reverseList 使用双指针法在一次遍历中完成链表翻转，代码简洁性超出预期。

遇到的问题：

- 删除倒数第 n 个结点时，快慢指针的步进距离计算有误，导致快指针到达尾部时慢指针的位置不符合预期。虽然可通过计算顺序位置（len - n）更可靠地实现，但我坚持调试了快慢指针版本，最终通过多次修正正确实现。
- 多链表管理初始化时，将 manager.lists[i] 置为 NULL 但 manager.current 初始化为 -1，调用 AddList 后未立即调整当前索引，导致后续操作异常。修正为在 AddList 中将 current 设为 count-1。
- 从文件加载链表时采用尾插法，需维护 tail 指针以避免每次插入都遍历到末尾，初始实现忽略了这一点导致效率低下，修正后性能得到改善。

建议：

实验测试中，手动输入大量链表数据较为耗时。建议提供一个生成随机链表的辅助函数，便于测试排序、删除等功能的正确性与性能。

3.3 基于二叉链表的二叉树实现

收获：

通过本实验，我对递归在二叉树中的应用有了更加熟练的掌握。从 CreateBiTree 到 DestroyTree 再到各类遍历，递归代码的简洁性与直观性得到了充分体现。同时，我首次实现了非递归先序遍历（使用显式栈），加深了对“手动模拟栈”这一技术的理解。另一个重要收获是 GetRightMost 函数的实现：在删除度为 2 的结点时，需找到左子树的最右结点并将右子树接入该位置，该逻辑通过画图推导后才得以正确实现。

遇到的问题：

- 在 InsertNode 中，插入左孩子时父结点原有的左子树应成为新结点的右子树，初始实现遗漏了这一关系，导致子树丢失。参照实验要求中的说明修正后恢复正常。
- FindParent 函数需使用非递归方式查找父结点，初期采用递归实现但返回值处理较为复杂，改为显式栈后逻辑清晰度显著提高。

- 文件保存与读取采用先序序列形式，以 0 表示空结点、-1 表示结束。早期使用字符读取方式易出错，后改为 fscanf 按格式读取整数与字符串，并配合 inputEnd 标记，稳定性得到保障。

建议：

删除结点的逻辑（度为 2 时以左孩子替代被删结点，右子树挂至左子树最右结点下）在实验要求中描述较为详细，但对初学者仍易出错。建议在实验指导中附上示意图或提供针对性的测试用例，便于验证实现正确性。

3.4 基于邻接表的图实现

收获：

图实验是四个实验中最具“工程化”色彩的一个。邻接表结构中每个顶点附带一个链表，需同时管理顶点数组与边链表，数据结构呈现两层嵌套。编写 CreateGraph 时，需传入顶点数组与边数组并据此构建邻接表，其逻辑复杂度显著高于线性表与二叉树。同时，我首次使用广度优先搜索（BFS）计算最短路径长度，队列的 front 与 rear 下标管理方式比链式队列更为简洁。

遇到的问题：

- 在 DeleteVex 中删除顶点时，需同步删除所有指向该顶点的边。我实现时使用了两个循环：首先遍历其他顶点的边链表，删除指向 pos 的边，同时调整 adjvex 索引（因删除顶点后后续顶点下标前移）。该逻辑调试耗时较长，特别是索引调整部分容易遗漏。
- 保存图至文件时，为避免无向边重复保存，使用了 visited 二维数组标记已保存的边（仅当 i < j 时保存）。对于 MAX_VERTEX_NUM=20 而言该方案可接受，但若图规模较大则存在空间浪费。
- 多图管理中切换图时，初始实现未检查 current_graph 是否越界，后于 SwitchGraph 中增加边界判断予以修正。

建议：

图实验涉及的算法较多（DFS、BFS、最短路径、连通分量），建议在实验任务中给出清晰的渐进式测试数据，例如“使用以下边集测试 BFS 的输出”，便于学生验证各功能的正确性。此外，VerticesSetLessThanK 功能可提示使用 BFS 层次遍历实现，而非采用暴力枚举方式。

总体建议：

四个实验从线性结构到树再到图，难度逐级递进，课程设计合理。但每个实验中的“多结构管理”功能（管理多张表、多棵树、多张图）在代码实现上存在较多重复，可考虑将其合并为一个通用的对象管理器模板，或在实验指导中给出参考设计，以减少重复劳动。此外，文件保存与加载部分，建议统一采用“先写长度后写数据”的格式，并提供示例文件内容便于调试。

参考文献

[1] 袁玲等. 数据结构——从概念到算法（C 语言版 | 微课版）[M]. 北京 : 人民邮电出版社, 2023.

[2] 严蔚敏, 吴伟民. 数据结构（C 语言版）[M]. [S.l.] : 清华大学出版社, 2018.

[3] 严蔚敏等. 数据结构题集（C 语言版）[M]. [S.l.] : 清华大学出版社, 2017.

[4] NYHOFF L. ADTs, Data Structures, and Problem Solving with C++[M]. Second. [S.l.] : Calvin College, 2005.

[5] 殷立峰. Qt C++ 跨平台图形界面程序设计基础 [M]. [S.l.] : 清华大学出版社, 2014 : 192 – 197.

[6] WEISS M A. 数据结构与算法分析——C 语言描述 [M]. 冯舜玺, trans. 原书第 2 版. 北京 : 机械工业出版社, 2008.

附录 A 基于顺序存储结构线性表实现的源程序

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 宏定义：状态码，用于函数返回值表示操作结果
#define TRUE         1        // 真
#define FALSE        0        // 假
#define OK           1        // 操作成功
#define ERROR        0        // 操作出错
#define INFEASIBLE   -1       // 操作不可行（如未初始化）
#define OVERFLOW     -2       // 内存溢出

typedef int status;           // 状态类型，用于返回函数执行状态
typedef int ElemType;         // 数据元素类型（这里设为int）

// 顺序表初始分配容量与增量
#define LIST_INIT_SIZE 100   // 初始容量
#define LISTINCREMENT  10    // 扩容增量

// 顺序表结构定义
typedef struct{
    ElemType *elem;          // 指向数据存储空间的基地址
    int length;              // 当前表中元素个数
    int listsize;            // 当前分配的存储容量
}SqList;

// 多线性表管理结构（可同时管理多张顺序表）
typedef struct{
    struct {
        char name[30];       // 表名
        SqList L;            // 对应的顺序表
    } elem[10];              // 最多管理10张表
    int length;              // 当前管理表的数量
```

```
}LISTS;

LISTS Lists;                    // 全局多表管理器
int current = 0;                // 当前正在操作的表的下标

//========================= 函数声明 =========================
// 基础顺序表操作
status InitList(SqList *L);                          // 初始化顺序表
status DestroyList(SqList *L);                       // 销毁顺序表
status ClearList(SqList *L);                         // 清空表
status ListEmpty(SqList L);                          // 判断是否为空表
status ListLength(SqList L);                         // 获取表长度
status GetElem(SqList L,int i,ElemType *e);          // 获取第i个元素
int LocateElem(SqList L,ElemType e);                 // 查找元素e第一
    ↪ 次出现的位置
status PriorElem(SqList L,ElemType e,ElemType *pre_e);  // 求前驱
status NextElem(SqList L,ElemType e,ElemType *next_e);  // 求后继
status ListInsert(SqList *L,int i,ElemType e);       // 在第i个位置插
    ↪ 入e
status ListDelete(SqList *L,int i,ElemType *e);      // 删除第i个元素
status ListTraverse(SqList L);                       // 遍历输出表

// 扩展功能
int MaxSubArray(SqList L);                           // 最大子数组和
    ↪ (Kadane算法)
int SubArrayNum(SqList L,int k);                     // 和为k的子数组
    ↪ 数量
void sortList(SqList *L);                            // 冒泡排序
status SaveList(SqList L,char FileName[]);           // 保存到文件
status LoadList(SqList *L,char FileName[]);          // 从文件加载

// 多表管理
status AddList(LISTS *Lists,char ListName[]);        // 新建一张表
status RemoveList(LISTS *Lists,char ListName[]);     // 删除一张表
int LocateList(LISTS Lists,char ListName[]);         // 按表名查找
```

```
65 void ShowAllLists(LISTS Lists);                    // 显示所有表
66 status SwitchList(LISTS *Lists, char *name);      // 切换当前操作
   ↪ 表
67
68 //========================== 函数实现 ==========================
69
70 // 初始化顺序表：分配内存，设置初始状态
71 status InitList(SqList *L)
72 {
73     // 如果表已经初始化过，不允许重复初始化
74     if(L->elem != NULL){
75         return INFEASIBLE;
76     }
77     // 动态分配初始内存
78     L->elem = (ElemType*)malloc(sizeof(ElemType)*LIST_INIT_SIZE);
79     if(!L->elem) return OVERFLOW;     // 分配失败
80     L->length = 0;                    // 初始元素个数为0
81     L->listsize = LIST_INIT_SIZE;     // 初始容量
82     return OK;
83 }
84
85 // 销毁顺序表：释放内存，重置表状态
86 status DestroyList(SqList *L)
87 {
88     if(L->elem == NULL){
89         return INFEASIBLE;
90     }
91     free(L->elem);            // 释放动态分配的内存
92     L->elem = NULL;           // 指针置空
93     L->length = 0;            // 长度清零
94     L->listsize = 0;          // 容量清零
95     return OK;
96 }
97
98 // 清空表：只清空数据，不释放内存
```

```
99  status ClearList(SqList *L)
100 {
101     if (L->elem == NULL) return INFEASIBLE;
102     L->length = 0;
103     return OK;
104 }
105
106 // 判断表是否为空
107 status ListEmpty(SqList L)
108 {
109     if(L.elem == NULL) return INFEASIBLE;
110     return (L.length == 0) ? TRUE : FALSE;
111 }
112
113 // 返回表的长度
114 status ListLength(SqList L)
115 {
116     if(L.elem == NULL) return INFEASIBLE;
117     return L.length;
118 }
119
120 // 获取第i个元素，存入e
121 status GetElem(SqList L,int i,ElemType *e)
122 {
123     if(L.elem == NULL) return INFEASIBLE;
124     if(i < 1 || i > L.length) return ERROR;    // 位置不合法
125     *e = L.elem[i-1];                          // 取元素
126     return OK;
127 }
128
129 // 查找元素e，返回第一次出现的位序，找不到返回0
130 int LocateElem(SqList L,ElemType e)
131 {
132     int i;
133     if(L.elem == NULL) return INFEASIBLE;
```

```
134     // 遍历查找
135     for(i=0; i<L.length && L.elem[i]!=e; i++);
136     return (i == L.length) ? 0 : i+1;
137 }
138 
139 // 求元素e的前驱，存入pre_e
140 status PriorElem(SqList L,ElemType e,ElemType *pre_e)
141 {
142     if(L.elem == NULL) return INFEASIBLE;
143     int i;
144     for(i=0; i<L.length && L.elem[i]!=e; i++);
145     // 第一个元素无前驱，或没找到e
146     if(i == 0 || i >= L.length) return ERROR;
147     *pre_e = L.elem[i-1];
148     return OK;
149 }
150 
151 // 求元素e的后继
152 status NextElem(SqList L,ElemType e,ElemType *next_e)
153 {
154     if(L.elem == NULL) return INFEASIBLE;
155     int i;
156     for(i=0; i<L.length && L.elem[i]!=e; i++);
157     // 最后一个元素无后继
158     if(i >= L.length-1) return ERROR;
159     *next_e = L.elem[i+1];
160     return OK;
161 }
162 
163 // 在第i个位置插入元素e
164 status ListInsert(SqList *L,int i,ElemType e)
165 {
166     if (L->elem == NULL) return INFEASIBLE;
167     if (i < 1 || i > L->length+1) return ERROR;
168 
```

```
169     // 空间不足则扩容
170     if (L->length >= L->listsize){
171         ElemType *newbase = (ElemType*)realloc(L->elem, sizeof(
                ↪ ElemType)*(L->listsize+LISTINCREMENT));
172         if(!newbase) return OVERFLOW;
173         L->elem = newbase;
174         L->listsize += LISTINCREMENT;
175     }
176 
177     // 元素后移, 腾出位置i
178     for (int j = L->length-1; j >= i-1; j--)
179         L->elem[j+1] = L->elem[j];
180 
181     L->elem[i-1] = e;        // 插入新元素
182     L->length++;             // 长度+1
183     return OK;
184 }
185 
186 // 删除第i个元素, 将值存入e
187 status ListDelete(SqList *L,int i,ElemType *e)
188 {
189     if (L->elem == NULL) return INFEASIBLE;
190     if (i < 1 || i > L->length) return ERROR;
191     *e = L->elem[i-1];                    // 取出被删元素
192     for (int j = i-1; j < L->length-1; j++)  // 元素前移
193         L->elem[j] = L->elem[j+1];
194     L->length--;                          // 长度-1
195     return OK;
196 }
197 
198 // 遍历输出顺序表所有元素
199 status ListTraverse(SqList L)
200 {
201     if (L.elem == NULL) return INFEASIBLE;
202     if(L.length == 0) printf("空线性表");
```

```
203     else{
204         for(int i=0; i<L.length; i++){
205             printf("%d", L.elem[i]);
206             if(i != L.length-1) printf(" ");
207         }
208     }
209     printf("\n");
210     return OK;
211 }
212 
213 // 最大子数组和（Kadane算法）
214 int MaxSubArray(SqList L)
215 {
216     if(L.elem == NULL || L.length == 0) return INFEASIBLE;
217     int maxSum = L.elem[0], curSum = L.elem[0];
218     for(int i=1; i<L.length; i++){
219         curSum = (curSum > 0) ? curSum + L.elem[i] : L.elem[i];
220         maxSum = (maxSum > curSum) ? maxSum : curSum;
221     }
222     return maxSum;
223 }
224 
225 // 求和为k的子数组个数
226 int SubArrayNum(SqList L,int k)
227 {
228     if(L.elem == NULL || L.length == 0) return INFEASIBLE;
229     int cnt = 0;
230     for(int i=0; i<L.length; i++){
231         int sum = 0;
232         for(int j=i; j<L.length; j++){
233             sum += L.elem[j];
234             if(sum == k) cnt++;
235         }
236     }
237     return cnt;
```

```
238 }
239
240 // 冒泡排序（升序）
241 void sortList(SqList *L)
242 {
243     if(L->elem == NULL || L->length <= 1) return;
244     for(int i=0; i<L->length-1; i++){
245         for(int j=0; j<L->length-i-1; j++){
246             if(L->elem[j] > L->elem[j+1]){
247                 ElemType t = L->elem[j];
248                 L->elem[j] = L->elem[j+1];
249                 L->elem[j+1] = t;
250             }
251         }
252     }
253 }
254
255 // 将顺序表保存到文件（二进制）
256 status SaveList(SqList L,char FileName[])
257 {
258     if(L.elem == NULL) return INFEASIBLE;
259     FILE *fp = fopen(FileName,"wb");
260     if(!fp) return INFEASIBLE;
261     fwrite(&L.length, sizeof(int), 1, fp);          // 保存长度
262     fwrite(L.elem, sizeof(ElemType), L.length, fp); // 保存数据
263     fclose(fp);
264     return OK;
265 }
266
267 // 从文件加载顺序表
268 status LoadList(SqList *L,char FileName[])
269 {
270     // 如果已有数据，先释放
271     if(L->elem != NULL)
272     {
```

```
273         free(L->elem);
274         L->elem = NULL;
275     }
276 
277     FILE* fp = fopen(FileName,"rb");
278     if(!fp) return INFEASIBLE;
279 
280     int len;
281     fread(&len, sizeof(int), 1, fp);                    // 读取长度
282     L->elem = (ElemType*)malloc(sizeof(ElemType)*len);
283     L->length = len;
284     L->listsize = len;
285     fread(L->elem, sizeof(ElemType), len, fp);      // 读取数据
286     fclose(fp);
287     return OK;
288 }
289 
290 //========================= 多表管理函数
    ↪ ================================
291 
292 // 新建一张线性表，加入表管理器
293 status AddList(LISTS *Lists,char ListName[])
294 {
295     if (Lists->length >= 10) return ERROR;
296     strcpy(Lists->elem[Lists->length].name, ListName);
297     Lists->elem[Lists->length].L.elem = NULL;
298     Lists->elem[Lists->length].L.length = 0;
299     Lists->elem[Lists->length].L.listsize = 0;
300     Lists->length++;
301     return OK;
302 }
303 
304 // 删除指定名称的线性表
305 status RemoveList(LISTS *Lists,char ListName[])
306 {
```

```
307     int i,j;
308     for(i = 0; i < Lists->length; i++)
309     {
310         if(strcmp(ListName, Lists->elem[i].name) == 0)
311         {
312             // 默认表不允许删除
313             if(strcmp(ListName, "默认表") == 0)
314             {
315                 printf("无法删除默认表! \n");
316                 return ERROR;
317             }
318 
319             DestroyList(&Lists->elem[i].L);
320             // 后面的表前移
321             for(j = i; j < Lists->length - 1; j++)
322             {
323                 Lists->elem[j] = Lists->elem[j+1];
324             }
325             Lists->length--;
326             // 调整当前表指针
327             if (i == current) current = 0;
328             else if (i < current) current--;
329             return OK;
330         }
331     }
332     return ERROR;
333 }
334 
335 // 根据表名查找位置
336 int LocateList(LISTS Lists,char ListName[])
337 {
338     int i;
339     for(i = 0; i < Lists.length; i++)
340     {
341         if(strcmp(ListName, Lists.elem[i].name) == 0)
```

```
342         {
343             return i;
344         }
345     }
346     return -1;
347 }
348 
349 // 切换当前操作表
350 status SwitchList(LISTS *Lists, char *name)
351 {
352     int idx = LocateList(*Lists, name);
353     if (idx < 0) return ERROR;
354     current = idx;
355     return OK;
356 }
357 
358 // 显示所有表及其内容
359 void ShowAllLists(LISTS Lists)
360 {
361     printf("\n===== 所有线性表及其数据 =====\n");
362     if(Lists.length == 0)
363     {
364         printf("暂无任何线性表\n");
365         return;
366     }
367     for(int i = 0; i < Lists.length; i++)
368     {
369         printf("%d. 表名: %s  数据: ", i+1, Lists.elem[i].name);
370         if(Lists.elem[i].L.elem == NULL)
371             printf("未初始化");
372         else{
373             if(Lists.elem[i].L.length == 0) printf("空线性表");
374             else{
375                 for(int j=0; j<Lists.elem[i].L.length; j++)
376                     printf("%d ", Lists.elem[i].L.elem[j]);
```

```
377                 }
378             }
379             if (i == current) printf("← 当前操作");
380             printf("\n");
381         }
382         printf("\n");
383     }
384
385     //========================= 菜单与主函数
        ↪ ================================
386
387     // 菜单界面
388     void Menu()
389     {
390         printf("\n================ 顺序表操作菜单 ================\n");
391         printf("1.初始化表\t\t2.销毁表\t\t3.清空表\t\t4.判定空表\n");
392         printf("5.求表长\t\t6.获得元素\t\t7.查找元素\t\t8.获得前驱\n"
            ↪ );
393         printf("9.获得后继\t\t10.插入元素\t\t11.删除元素\t\t12.遍历表
            ↪ \n");
394         printf("13.最大子数组和\t\t14.和为K子数组数\t15.顺序表排序\t\
            ↪ t16.保存到文件\t\t17.从文件加载\n");
395         printf("18.新建线性表\t\t19.删除线性表\t\t20.按名查找表\t\t21
            ↪ .显示所有表\t\t22.切换当前表\n");
396         printf("0.退出\n");
397         printf("请选择:");
398     }
399
400     // 主函数: 程序入口
401     int main()
402     {
403         Lists.length = 0;
404         AddList(&Lists, "默认表");      // 默认创建一张表
405         current = 0;                    // 默认操作第一张表
406
```

```
407     int op,i,k,pos;
408     ElemType e;
409     char fname[30],lname[30];
410
411     while(1){
412         Menu();
413         scanf("%d",&op);
414         SqList *now = &Lists.elem[current].L;  // 当前操作表
415
416         switch(op){
417             case 1:
418             {
419                 status res = InitList(now);
420                 if(res == OK) printf("初始化成功\n");
421                 else if(res == INFEASIBLE) printf("已经初始化，无
                        ↪ 需重复操作\n");
422                 else printf("初始化失败\n");
423                 break;
424             }
425             case 2:
426             {
427                 // 默认表不能销毁
428                 if(strcmp(Lists.elem[current].name, "默认表") ==
                        ↪ 0)
429                 {
430                     printf("默认表不能销毁! \n");
431                     break;
432                 }
433                 status res = DestroyList(now);
434                 if(res == OK) printf("销毁成功\n");
435                 else if(res == INFEASIBLE) printf("已经销毁，无需
                        ↪ 重复操作\n");
436                 else printf("销毁失败\n");
437                 break;
438             }
```

```
439             case 3:
440             {
441                 status res = ClearList(now);
442                 if(res == OK) printf("清空成功\n");
443                 else printf("表未初始化，无法清空\n");
444                 break;
445             }
446             case 4:
447             {
448                 status res = ListEmpty(*now);
449                 if(res == TRUE) printf("空表\n");
450                 else if(res == FALSE) printf("非空表\n");
451                 else printf("表未初始化\n");
452                 break;
453             }
454             case 5:
455             {
456                 status res = ListLength(*now);
457                 if(res != INFEASIBLE) printf("表长：%d\n",res);
458                 else printf("表未初始化\n");
459                 break;
460             }
461             case 6:
462                 printf("输入位置："); scanf("%d",&i);
463                 if(GetElem(*now,i,&e)==OK) printf("元素：%d\n",e)
                    ↪ ;
464                 else printf("失败\n"); break;
465             case 7:
466                 printf("输入值："); scanf("%d",&e);
467                 pos=LocateElem(*now,e);
468                 pos?printf("位置：%d\n",pos):printf("未找到\n");
                    ↪ break;
469             case 8:
470                 printf("输入元素："); scanf("%d",&e);
```

```
471                 if(PriorElem(*now,e,&i)==OK) printf("前驱: %d\n",
                        ↪ i);
472                 else printf("无\n"); break;
473             case 9:
474                 printf("输入元素: "); scanf("%d",&e);
475                 if(NextElem(*now,e,&i)==OK) printf("后继: %d\n",i
                        ↪ );
476                 else printf("无\n"); break;
477             case 10:
478                 printf("位置 值: "); scanf("%d%d",&i,&e);
479                 if(ListInsert(now,i,e)==OK) printf("成功\n");
480                 else printf("失败\n"); break;
481             case 11:
482                 printf("位置: "); scanf("%d",&i);
483                 if(ListDelete(now,i,&e)==OK) printf("删除: %d\n",
                        ↪ e);
484                 else printf("失败\n"); break;
485             case 12:
486                 if(ListTraverse(*now) == INFEASIBLE) printf("表未
                        ↪ 初始化\n");
487                 break;
488             case 13:
489             {
490                 int sum = MaxSubArray(*now);
491                 if(sum != INFEASIBLE) printf("最大子数组和: %d\n"
                        ↪ ,sum);
492                 else printf("表未初始化或为空\n");
493                 break;
494             }
495             case 14:
496                 printf("输入k: "); scanf("%d",&k);
497                 {
498                     int num = SubArrayNum(*now,k);
499                     if(num != INFEASIBLE) printf("个数: %d\n",num
                            ↪ );
```

```
500                 else printf("表未初始化或为空\n");
501             }
502             break;
503         case 15:
504             sortList(now);
505             printf("排序完成\n");
506             break;
507         case 16:
508             printf("文件名: "); scanf("%s",fname);
509             if(SaveList(*now,fname)==OK) printf("保存成功\n")
                    ↪ ;
510             else printf("失败\n"); break;
511         case 17:
512             printf("文件名: "); scanf("%s",fname);
513             if(LoadList(now,fname)==OK) printf("加载成功\n");
514             else printf("失败\n"); break;
515         case 18:
516         {
517             int data;
518             printf("输入新表名: ");
519             scanf("%s", lname);
520             AddList(&Lists, lname);
521
522             SqList *cur = &Lists.elem[Lists.length-1].L;
523             InitList(cur);
524
525             printf("输入数据 (以0结束) : ");
526             while(1)
527             {
528                 scanf("%d", &data);
529                 if(data == 0) break;
530                 ListInsert(cur, cur->length+1, data);
531             }
532
533             printf("%s ", lname);
```

```
534                 for(int j=0; j<cur->length; j++)
535                     printf("%d ", cur->elem[j]);
536                 printf("\n");
537                 break;
538             }
539             case 19:
540                 printf("输入要删除的表名: ");
541                 scanf("%s", lname);
542                 if(RemoveList(&Lists, lname))
543                     printf("删除成功! \n");
544                 else
545                     printf("删除失败\n");
546                 break;
547             case 20:
548                 printf("输入要查找的表名: ");
549                 scanf("%s", lname);
550                 pos = LocateList(Lists, lname);
551                 if(pos >= 0)
552                     printf("该表是第 %d 个\n", pos+1);
553                 else
554                     printf("未找到\n");
555                 break;
556             case 21:
557                 ShowAllLists(Lists);
558                 break;
559
560             case 22:
561                 printf("输入表名: ");
562                 scanf("%s", lname);
563                 if (SwitchList(&Lists, lname))
564                     printf("已切换到表: %s\n", lname);
565                 else
566                     printf("表不存在\n");
567                 break;
568
```

```
569                 case 0:
570                     // 退出前销毁所有表
571                     for (int i=0; i<Lists.length; i++)
572                         DestroyList(&Lists.elem[i].L);
573                     printf("退出程序\n");
574                     return 0;
575                 default:
576                     printf("输入错误\n");
577             }
578         }
579         return 0;
580 }
```

附录 B 基于链式存储结构线性表实现的源程序

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 宏定义状态码，用于函数返回值表示操作结果
#define TRUE 1    // 真
#define FALSE 0   // 假
#define OK 1      // 操作成功
#define ERROR 0   // 操作失败
#define OVERFLOW -1 // 内存溢出

typedef int ElemType;  // 数据元素类型（这里定义为 int）
typedef int Status;    // 状态类型，用于返回函数执行状态

// 单链表结点结构定义
typedef struct LNode {
    ElemType data;        // 数据域：存储结点数据
    struct LNode *next;   // 指针域：指向下一个结点
} LNode, *LinkList;

// 多链表管理器：最多管理 10 个链表
#define MAX_LIST 10
typedef struct {
    LinkList lists[MAX_LIST];  // 存储多个链表的头指针
    char names[MAX_LIST][20];  // 每个链表对应的名称
    int count;                 // 当前已创建的链表数量
    int current;               // 当前正在操作的链表下标
} ListManager;

ListManager manager;  // 全局链表管理器

// ==================== 函数声明 ====================
// 基础单链表操作
```

```
34 Status InitList(LinkList *L);              // 初始化链表（创建头结
   ↪ 点）
35 Status DestroyList(LinkList *L);           // 销毁整个链表（含头结
   ↪ 点）
36 Status ClearList(LinkList L);              // 清空链表（保留头结点）
37 Status ListEmpty(LinkList L);              // 判断链表是否为空
38 int ListLength(LinkList L);                // 获取链表长度
39 Status GetElem(LinkList L, int i, ElemType *e); // 获取第 i 个元
   ↪ 素
40 int LocateElem(LinkList L, ElemType e, Status (*compare)(ElemType
   ↪ , ElemType));  // 查找元素
41 Status PriorElem(LinkList L, ElemType cur_e, ElemType *pre_e);
   ↪    // 求前驱
42 Status NextElem(LinkList L, ElemType cur_e, ElemType *next_e);
   ↪    // 求后继
43 Status ListInsert(LinkList L, int i, ElemType e);   // 在第 i 个
   ↪ 位置插入元素
44 Status ListDelete(LinkList L, int i, ElemType *e);  // 删除第 i
   ↪ 个元素
45 Status ListTraverse(LinkList L, Status (*visit)(ElemType));  //
   ↪ 遍历链表
46
47 // 扩展功能
48 Status reverseList(LinkList L);              // 翻转链表
49 Status RemoveNthFromEnd(LinkList L, int n); // 删除倒数第 n 个结
   ↪ 点
50 Status sortList(LinkList L);                // 链表排序（冒泡）
51 Status SaveListToFile(LinkList L, char *filename);  // 保存链表到
   ↪ 文件
52 Status LoadListFromFile(LinkList L, char *filename); // 从文件加
   ↪ 载链表
53
54 // 多链表管理
55 void InitManager();                          // 初始化链表管理器
56 int AddList(char *name);                     // 新建一个链表
```

```
57 Status DelList(int index);                          // 删除指定序号的链表
58 void SwitchList(int index);                         // 切换当前操作的链表
59 void ShowAllLists();                                 // 显示所有链表
60
61 // 辅助函数
62 Status visit(ElemType e);                           // 遍历输出函数
63 Status compare(ElemType a, ElemType b);             // 比较函数（相等判
   ↪ 断）
64 void Menu();                                        // 菜单函数
65
66 // ==================== 主函数 ====================
67 int main() {
68     InitManager();          // 初始化链表管理器
69     AddList("默认表");      // 默认创建一个“默认表”
70     int choice, i, pos, n;
71     ElemType e, pre_e, next_e;
72     char filename[50], name[20];
73
74     // 循环菜单
75     while (1) {
76         Menu();
77         scanf("%d", &choice);
78         // 获取当前正在操作的链表
79         LinkList L = manager.lists[manager.current];
80
81         switch (choice) {
82             case 1:  // 1. 初始化表
83                 if (L != NULL) {
84                     printf("线性表已初始化，无需重复操作！\n");
85                 } else {
86                     if (InitList(&L)) {
87                         manager.lists[manager.current] = L;
88                         printf("线性表初始化成功！\n");
89                     }
90                 }
```

```
91                  break;
92
93              case 2:  // 2. 销毁表
94                  if (L == NULL) {
95                      printf("线性表已销毁，无需重复操作！\n");
96                  } else {
97                      if (DestroyList(&L)) {
98                          manager.lists[manager.current] = NULL;
99                          printf("线性表销毁成功！\n");
100                     }
101                 }
102                 break;
103
104             case 3:  // 3. 清空表
105                 if (L == NULL) {
106                     printf("错误：线性表未初始化！\n");
107                 } else {
108                     if (ClearList(L)) printf("线性表清空成功！\n"
                        ↪ );
109                 }
110                 break;
111
112             case 4:  // 4. 判断空表
113                 if (L == NULL) {
114                     printf("错误：线性表未初始化！\n");
115                 } else {
116                     if (ListEmpty(L)) printf("线性表为空！\n");
117                     else printf("线性表非空！\n");
118                 }
119                 break;
120
121             case 5:  // 5. 求表长
122                 if (L == NULL) {
123                     printf("错误：线性表未初始化！\n");
124                 } else {
```

```
125                 printf("线性表长度: %d\n", ListLength(L));
126             }
127             break;
128
129         case 6:  // 6. 获取元素
130             if (L == NULL) {
131                 printf("错误: 线性表未初始化! \n");
132                 break;
133             }
134             printf("请输入要获取的元素位置: ");
135             scanf("%d", &i);
136             if (GetElem(L, i, &e)) printf("第%d个元素: %d\n",
                ↪  i, e);
137             else printf("位置非法! \n");
138             break;
139
140         case 7:  // 7. 查找元素
141             if (L == NULL) {
142                 printf("错误: 线性表未初始化! \n");
143                 break;
144             }
145             printf("请输入要查找的元素: ");
146             scanf("%d", &e);
147             pos = LocateElem(L, e, compare);
148             if (pos) printf("元素%d的位置: %d\n", e, pos);
149             else printf("未找到该元素! \n");
150             break;
151
152         case 8:  // 8. 求前驱
153             if (L == NULL) {
154                 printf("错误: 线性表未初始化! \n");
155                 break;
156             }
157             printf("请输入元素: ");
158             scanf("%d", &e);
```

```
159                 if (PriorElem(L, e, &pre_e)) printf("前驱元素: %d
                        ↪ \n", pre_e);
160                 else printf("无前驱元素! \n");
161                 break;
162
163             case 9:  // 9. 求后继
164                 if (L == NULL) {
165                     printf("错误: 线性表未初始化! \n");
166                     break;
167                 }
168                 printf("请输入元素: ");
169                 scanf("%d", &e);
170                 if (NextElem(L, e, &next_e)) printf("后继元素: %d
                        ↪ \n", next_e);
171                 else printf("无后继元素! \n");
172                 break;
173
174             case 10:  // 10. 插入元素
175                 if (L == NULL) {
176                     printf("错误: 线性表未初始化! \n");
177                     break;
178                 }
179                 printf("请输入插入位置和元素 (空格分隔): ");
180                 scanf("%d %d", &i, &e);
181                 if (ListInsert(L, i, e)) printf("插入成功! \n");
182                 else printf("插入失败! \n");
183                 break;
184
185             case 11:  // 11. 删除元素
186                 if (L == NULL) {
187                     printf("错误: 线性表未初始化! \n");
188                     break;
189                 }
190                 printf("请输入删除位置: ");
191                 scanf("%d", &i);
```

```
192                 if (ListDelete(L, i, &e)) printf("删除成功, 元素
                        ↪ 值: %d\n", e);
193                 else printf("删除失败! \n");
194                 break;
195
196             case 12:  // 12. 遍历表
197                 if (L == NULL) {
198                     printf("错误: 线性表未初始化! \n");
199                     break;
200                 }
201                 printf("链表元素: ");
202                 ListTraverse(L, visit);
203                 printf("\n");
204                 break;
205
206             case 13:  // 13. 翻转链表
207                 if (L == NULL) {
208                     printf("错误: 线性表未初始化! \n");
209                     break;
210                 }
211                 reverseList(L);
212                 printf("链表翻转完成! \n");
213                 break;
214
215             case 14:  // 14. 删除倒数第n个结点
216                 if (L == NULL) {
217                     printf("错误: 线性表未初始化! \n");
218                     break;
219                 }
220                 printf("请输入n值: ");
221                 scanf("%d", &n);
222                 if (RemoveNthFromEnd(L, n)) printf("删除成功! \n"
                        ↪ );
223                 else printf("删除失败! \n");
224                 break;
```

```
225
226             case 15:  // 15. 链表排序
227                 if (L == NULL) {
228                     printf("错误: 线性表未初始化! \n");
229                     break;
230                 }
231                 sortList(L);
232                 printf("链表排序完成 (升序) ! \n");
233                 break;
234
235             case 16:  // 16. 保存到文件
236                 if (L == NULL) {
237                     printf("错误: 线性表未初始化! \n");
238                     break;
239                 }
240                 printf("请输入文件名: ");
241                 scanf("%s", filename);
242                 if (SaveListToFile(L, filename)) printf("保存成
                        ↪ 功! \n");
243                 else printf("保存失败! \n");
244                 break;
245
246             case 17:  // 17. 从文件加载
247                 if (L == NULL) {
248                     printf("错误: 线性表未初始化! \n");
249                     break;
250                 }
251                 printf("请输入文件名: ");
252                 scanf("%s", filename);
253                 ClearList(L);
254                 if (LoadListFromFile(L, filename)) printf("加载成
                        ↪ 功! \n");
255                 else printf("加载失败! \n");
256                 break;
257
```

```
258                 case 18:  // 18. 添加线性表
259                     printf("请输入线性表名称: ");
260                     scanf("%s", name);
261                     if (AddList(name)) printf("创建成功! \n");
262                     else printf("创建失败! \n");
263                     break;
264 
265                 case 19:  // 19. 删除线性表
266                     ShowAllLists();
267                     printf("请输入要删除的表序号: ");
268                     scanf("%d", &i);
269                     if (DelList(i)) printf("删除成功! \n");
270                     else printf("删除失败! \n");
271                     break;
272 
273                 case 20:  // 20. 切换线性表
274                     ShowAllLists();
275                     printf("请输入要切换的表序号: ");
276                     scanf("%d", &i);
277                     SwitchList(i);
278                     break;
279 
280                 case 0:  // 0. 退出程序
281                     printf("程序退出! \n");
282                     return 0;
283 
284                 default:
285                     printf("输入错误, 请重新选择! \n");
286             }
287         }
288 }
289 
290 // 初始化单链表 (创建头结点, 不存放数据)
291 Status InitList(LinkList *L) {
292     *L = (LinkList)malloc(sizeof(LNode));  // 分配头结点空间
```

```
293     if (!(*L)) exit(OVERFLOW);                  // 分配失败则退出
294     (*L)->next = NULL;                          // 头结点指针域置空
295     return OK;
296 }
297
298 // 销毁整个链表（释放所有结点，包括头结点）
299 Status DestroyList(LinkList *L) {
300     LinkList p, q;
301     p = *L;                 // p 指向头结点
302     while (p) {             // 遍历释放所有结点
303         q = p->next;
304         free(p);
305         p = q;
306     }
307     *L = NULL;              // 头指针置空
308     return OK;
309 }
310
311 // 清空链表（只删除数据结点，保留头结点）
312 Status ClearList(LinkList L) {
313     LinkList p = L->next, q;
314     while (p) {
315         q = p->next;
316         free(p);
317         p = q;
318     }
319     L->next = NULL;  // 头结点指针置空
320     return OK;
321 }
322
323 // 判断链表是否为空（空返回 TRUE，非空返回 FALSE）
324 Status ListEmpty(LinkList L) {
325     if (L && L->next == NULL) return TRUE;
326     return FALSE;
327 }
```

```
328
329 // 获取链表长度（数据结点个数）
330 int ListLength(LinkList L) {
331     int len = 0;
332     LinkList p = L->next;
333     while (p) {
334         len++;
335         p = p->next;
336     }
337     return len;
338 }
339
340 // 获取第 i 个元素，存入 e
341 Status GetElem(LinkList L, int i, ElemType *e) {
342     int j = 1;
343     LinkList p = L->next;  // p 指向第一个数据结点
344     // 找到第 i 个结点
345     while (p && j < i) {
346         p = p->next;
347         j++;
348     }
349     if (!p || j > i) return ERROR;  // 位置非法
350     *e = p->data;
351     return OK;
352 }
353
354 // 查找元素 e，返回位序，找不到返回 0
355 int LocateElem(LinkList L, ElemType e, Status (*compare)(ElemType
    ↪ , ElemType)) {
356     int pos = 1;
357     LinkList p = L->next;
358     while (p) {
359         if (compare(p->data, e)) return pos;
360         p = p->next;
361         pos++;
```

```
362     }
363     return 0;
364 }
365
366 // 求 cur_e 的前驱，存入 pre_e
367 Status PriorElem(LinkList L, ElemType cur_e, ElemType *pre_e) {
368     LinkList p = L->next, q = L;
369     // p 指向目标结点，q 指向 p 的前驱
370     while (p && p->data != cur_e) {
371         q = p;
372         p = p->next;
373     }
374     if (!p || q == L) return ERROR;  // 无前驱
375     *pre_e = q->data;
376     return OK;
377 }
378
379 // 求 cur_e 的后继，存入 next_e
380 Status NextElem(LinkList L, ElemType cur_e, ElemType *next_e) {
381     LinkList p = L->next;
382     while (p && p->data != cur_e) p = p->next;
383     if (!p || !p->next) return ERROR;  // 无后继
384     *next_e = p->next->data;
385     return OK;
386 }
387
388 // 在第 i 个位置插入元素 e
389 Status ListInsert(LinkList L, int i, ElemType e) {
390     LinkList p = L, s;
391     int j = 0;
392     // 找到第 i-1 个结点
393     while (p && j < i - 1) {
394         p = p->next;
395         j++;
396     }
```

```
397     if (!p || j > i - 1) return ERROR;  // 位置非法
398     // 创建新结点
399     s = (LinkList)malloc(sizeof(LNode));
400     s->data = e;
401     s->next = p->next;
402     p->next = s;
403     return OK;
404 }
405
406 // 删除第 i 个结点，将值存入 e
407 Status ListDelete(LinkList L, int i, ElemType *e) {
408     LinkList p = L, q;
409     int j = 0;
410     // 找到第 i-1 个结点
411     while (p->next && j < i - 1) {
412         p = p->next;
413         j++;
414     }
415     if (!p->next || j > i - 1) return ERROR;  // 位置非法
416     q = p->next;          // q 为待删除结点
417     *e = q->data;         // 保存被删结点数据
418     p->next = q->next;    // 移除结点
419     free(q);              // 释放空间
420     return OK;
421 }
422
423 // 遍历链表，调用 visit 函数输出
424 Status ListTraverse(LinkList L, Status (*visit)(ElemType)) {
425     LinkList p = L->next;
426     while (p) {
427         visit(p->data);
428         p = p->next;
429     }
430     return OK;
431 }
```

```
432
433 // 翻转链表（双指针法）
434 Status reverseList(LinkList L) {
435     LinkList pre = NULL, cur = L->next, next;
436     while (cur) {
437         next = cur->next;
438         cur->next = pre;
439         pre = cur;
440         cur = next;
441     }
442     L->next = pre;  // 头结点指向翻转后的第一个结点
443     return OK;
444 }
445
446 // 删除倒数第 n 个结点（快慢指针法）
447 Status RemoveNthFromEnd(LinkList L, int n) {
448     LinkList fast = L, slow = L;
449     int len = ListLength(L);
450     if (n < 1 || n > len) return ERROR;
451     // 快指针先走 n 步
452     for (int i = 0; i < n; i++) fast = fast->next;
453     // 快慢指针一起走
454     while (fast->next) {
455         fast = fast->next;
456         slow = slow->next;
457     }
458     // 删除慢指针的下一个结点
459     LinkList del = slow->next;
460     slow->next = del->next;
461     free(del);
462     return OK;
463 }
464
465 // 链表排序（冒泡排序，交换数据域）
466 Status sortList(LinkList L) {
```

```
467     int len = ListLength(L);
468     if (len <= 1) return OK;
469     LinkList p, q;
470     ElemType temp;
471     for (int i = 0; i < len - 1; i++) {
472         p = L->next;
473         for (int j = 0; j < len - i - 1; j++) {
474             q = p->next;
475             if (p->data > q->data) {
476                 // 交换数据
477                 temp = p->data;
478                 p->data = q->data;
479                 q->data = temp;
480             }
481             p = p->next;
482         }
483     }
484     return OK;
485 }
486 
487 // 将链表保存到文本文件
488 Status SaveListToFile(LinkList L, char *filename) {
489     FILE *fp = fopen(filename, "w");
490     if (!fp) return ERROR;
491     LinkList p = L->next;
492     while (p) {
493         fprintf(fp, "%d ", p->data);
494         p = p->next;
495     }
496     fclose(fp);
497     return OK;
498 }
499 
500 // 从文本文件加载数据到链表
501 Status LoadListFromFile(LinkList L, char *filename) {
```

```
502     FILE *fp = fopen(filename, "r");
503     if (!fp) return ERROR;
504     ElemType e;
505     LinkList tail = L;
506     // 读取数据并尾插法建表
507     while (fscanf(fp, "%d", &e) != EOF) {
508         LinkList s = (LinkList)malloc(sizeof(LNode));
509         s->data = e;
510         s->next = NULL;
511         tail->next = s;
512         tail = s;
513     }
514     fclose(fp);
515     return OK;
516 }
517 
518 // 初始化链表管理器
519 void InitManager() {
520     manager.count = 0;
521     manager.current = -1;
522     for (int i = 0; i < MAX_LIST; i++) manager.lists[i] = NULL;
523 }
524 
525 // 新建一个链表，加入管理器
526 int AddList(char *name) {
527     if (manager.count >= MAX_LIST) return 0;
528     manager.lists[manager.count] = NULL;
529     strcpy(manager.names[manager.count], name);
530     manager.current = manager.count;
531     manager.count++;
532     return 1;
533 }
534 
535 // 删除指定序号的链表
536 Status DelList(int index) {
```

```
537     if (index < 0 || index >= manager.count) return ERROR;
538     DestroyList(&manager.lists[index]);
539     // 后面的链表前移
540     for (int i = index; i < manager.count - 1; i++) {
541         manager.lists[i] = manager.lists[i + 1];
542         strcpy(manager.names[i], manager.names[i + 1]);
543     }
544     manager.count--;
545     manager.current = manager.count > 0 ? 0 : -1;
546     return OK;
547 }
548 
549 // 切换当前操作的链表
550 void SwitchList(int index) {
551     if (index >= 0 && index < manager.count) {
552         manager.current = index;
553         printf("已切换到表: %s\n", manager.names[index]);
554     } else printf("序号非法! \n");
555 }
556 
557 // 显示所有链表列表
558 void ShowAllLists() {
559     printf("当前所有线性表: \n");
560     for (int i = 0; i < manager.count; i++) {
561         if (i == manager.current) printf("[%d] %s (当前) \n", i,
            ↪ manager.names[i]);
562         else printf("[%d] %s\n", i, manager.names[i]);
563     }
564 }
565 
566 // 遍历输出函数
567 Status visit(ElemType e) {
568     printf("%d ", e);
569     return OK;
570 }
```

```
571
572 // 比较函数：判断两个元素是否相等
573 Status compare(ElemType a, ElemType b) {
574     return a == b;
575 }
576
577 // 菜单界面
578 void Menu() {
579     printf("\n=============== 单链表功能菜单 ===============\n");
580     printf("1. 初始化表   2. 销毁表   3. 清空表   4. 判断空表\n");
581     printf("5. 求表长     6. 获取元素   7. 查找元素   8. 求前驱\n");
582     printf("9. 求后继    10. 插入元素 11. 删除元素 12. 遍历表\n");
583     printf("13. 翻转链表  14. 删除倒数第n个结点 15. 链表排序\n");
584     printf("16. 保存到文件  17. 从文件加载   18. 添加线性表\n");
585     printf("19. 删除线性表  20. 切换线性表   0. 退出程序\n");
586     printf("=============================================\n");
587     printf("当前操作：%s，请输入功能序号：", manager.names[
        ↪ manager.current]);
588 }
```

附录 C 基于二叉链表二叉树实现的源程序

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 宏定义状态常量
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define MAX_TREE 10  // 最多管理10棵二叉树

typedef int status;    // 状态类型
typedef int KeyType;   // 关键字类型（int）

// 二叉树结点数据域结构
typedef struct
{
    KeyType key;           // 关键字
    char others[20];       // 其他信息（字符串）
} TElemType;

// 二叉链表结点结构
typedef struct BiTNode
{
    TElemType data;        // 数据域
    struct BiTNode *lchild, *rchild;  // 左右孩子指针
} BiTNode, *BiTree;

int inputEnd = 0;  // 输入结束标记

// 多棵树管理（全局）
BiTree trees[MAX_TREE];        // 存储多棵树的根
char treeNames[MAX_TREE][20]; // 树名
```

```
34 int treeCnt = 0;                  // 当前树数量
35 int curSel = -1;                  // 当前选中的树下标
36
37 // 访问结点函数：输出关键字和附加信息
38 void Visit(BiTree p)
39 {
40     printf("%d,%s ", p->data.key, p->data.others);
41 }
42
43 // 获取一棵树的最右结点（用于删除时调整结构）
44 BiTNode* GetRightMost(BiTree T)
45 {
46     while (T && T->rchild)
47         T = T->rchild;
48     return T;
49 }
50
51 //==================== 二叉树创建（先序） ====================
52 void CreatePre(BiTree *T)
53 {
54     if (inputEnd) {
55         *T = NULL;
56         return;
57     }
58
59     int key;
60     char str[20];
61     if (scanf("%d %s", &key, str) != 2) {
62         inputEnd = 1;
63         *T = NULL;
64         return;
65     }
66
67     if (key == -1) {  // -1 表示输入结束
68         inputEnd = 1;
```

```
69          *T = NULL;
70          return;
71      }
72      if (key == 0) {    // 0 表示空结点
73          *T = NULL;
74          return;
75      }
76
77      // 创建当前结点
78      *T = (BiTree)malloc(sizeof(BiTNode));
79      (*T)->data.key = key;
80      strcpy((*T)->data.others, str);
81      (*T)->lchild = NULL;
82      (*T)->rchild = NULL;
83
84      CreatePre(&(*T)->lchild);  // 递归创建左子树
85      CreatePre(&(*T)->rchild);  // 递归创建右子树
86  }
87
88  // 创建二叉树 (对外接口)
89  status CreateBiTree(BiTree *T)
90  {
91      inputEnd = 0;
92      CreatePre(T);
93
94      int key;
95      char str[20];
96      while (!inputEnd) {
97          if (scanf("%d %s", &key, str) != 2) break;
98          if (key == -1) {
99              inputEnd = 1;
100             break;
101         }
102     }
103     return OK;
```

```
104 }
105
106 //==================== 销毁 & 清空 ====================
107 void DestroyTree(BiTree *T)
108 {
109     if (!*T) return;
110     DestroyTree(&(*T)->lchild);
111     DestroyTree(&(*T)->rchild);
112     free(*T);
113     *T = NULL;
114 }
115
116 // 销毁二叉树
117 status DestroyBiTree(BiTree *T)
118 {
119     DestroyTree(T);
120     return OK;
121 }
122
123 // 清空二叉树（等同于销毁）
124 status ClearBiTree(BiTree *T)
125 {
126     DestroyTree(T);
127     return OK;
128 }
129
130 //==================== 树状态 ====================
131 // 判断是否为空树
132 status BiTreeEmpty(BiTree T)
133 {
134     return T == NULL ? TRUE : FALSE;
135 }
136
137 // 求树深度
138 int BiTreeDepth(BiTree T)
```

```
{
    if (!T) return 0;
    int l = BiTreeDepth(T->lchild);
    int r = BiTreeDepth(T->rchild);
    return (l > r ? l : r) + 1;
}

//==================== 查找 & 修改 ====================
// 根据关键字查找结点
BiTNode* LocateNode(BiTree T, KeyType e)
{
    if (!T) return NULL;
    if (T->data.key == e) return T;
    BiTNode *p = LocateNode(T->lchild, e);
    if (p) return p;
    return LocateNode(T->rchild, e);
}

// 修改结点数据
status Assign(BiTree T, KeyType e, TElemType val)
{
    BiTNode *p = LocateNode(T, e);
    if (!p) return ERROR;
    if (val.key != e && LocateNode(T, val.key))
        return ERROR;
    p->data = val;
    return OK;
}

// 查找结点的父结点（非递归栈实现）
static BiTNode* FindParent(BiTree root, KeyType e, BiTree *parent
    ↪ )
{
    *parent = NULL;
    if (!root || root->data.key == e) return NULL;
```

```
173
174     BiTree stack[1000];
175     int top = -1;
176     stack[++top] = root;
177
178     while (top >= 0)
179     {
180         BiTree cur = stack[top--];
181
182         if (cur->lchild)
183         {
184             if (cur->lchild->data.key == e)
185             {
186                 *parent = cur;
187                 return cur->lchild;
188             }
189             stack[++top] = cur->lchild;
190         }
191
192         if (cur->rchild)
193         {
194             if (cur->rchild->data.key == e)
195             {
196                 *parent = cur;
197                 return cur->rchild;
198             }
199             stack[++top] = cur->rchild;
200         }
201     }
202
203     return NULL;
204 }
205
206 // 获取兄弟结点
207 BiTNode* GetSibling(BiTree T, KeyType e)
```

```
208 {
209     if (!T || T->data.key == e) return NULL;
210 
211     BiTree parent = NULL;
212     BiTNode *cur = FindParent(T, e, &parent);
213 
214     if (!cur || !parent) return NULL;
215     return (parent->lchild == cur) ? parent->rchild : parent->
        ↪ lchild;
216 }
217 
218 //==================== 插入结点 ====================
219 status InsertNode(BiTree *T, KeyType e, int LR, TElemType c)
220 {
221     if (LocateNode(*T, c.key)) return ERROR;
222     BiTNode *newNode = (BiTree)malloc(sizeof(BiTNode));
223     newNode->data = c;
224     newNode->lchild = newNode->rchild = NULL;
225 
226     if (LR == -1)  // 作为新根
227     {
228         newNode->lchild = *T;
229         *T = newNode;
230         return OK;
231     }
232 
233     BiTNode *fa = LocateNode(*T, e);
234     if (!fa)
235     {
236         free(newNode);
237         return ERROR;
238     }
239 
240     if (LR == 0)  // 左孩子
241     {
```

```
242         newNode->lchild = fa->lchild;
243         fa->lchild = newNode;
244     }
245     else  // 右孩子
246     {
247         newNode->rchild = fa->rchild;
248         fa->rchild = newNode;
249     }
250     return OK;
251 }
252
253 //==================== 删除结点 ====================
254 status DeleteNode(BiTree *T, KeyType e)
255 {
256     if (!*T) return ERROR;
257     if ((*T)->data.key == e)
258     {
259         BiTNode *del = *T;
260         if (!(*T)->lchild && !(*T)->rchild)  // 叶子结点
261             *T = NULL;
262         else if (!(*T)->lchild)  // 只有右孩子
263             *T = (*T)->rchild;
264         else if (!(*T)->rchild)  // 只有左孩子
265             *T = (*T)->lchild;
266         else  // 左右孩子都有
267         {
268             BiTree rightMost = GetRightMost((*T)->lchild);
269             rightMost->rchild = (*T)->rchild;
270             *T = (*T)->lchild;
271         }
272         free(del);
273         return OK;
274     }
275     if (DeleteNode(&(*T)->lchild, e)) return OK;
276     return DeleteNode(&(*T)->rchild, e);
```

```
277 }
278
279 //==================== 四种遍历 ====================
280 // 先序遍历（非递归栈）
281 status PreOrderTraverse(BiTree T, void (*visit)(BiTree))
282 {
283     if (!T) return OK;
284     BiTree stack[1000];
285     int top = -1;
286     stack[++top] = T;
287     while (top >= 0)
288     {
289         BiTree p = stack[top--];
290         visit(p);
291         if (p->rchild) stack[++top] = p->rchild;
292         if (p->lchild) stack[++top] = p->lchild;
293     }
294     return OK;
295 }
296
297 // 中序遍历（递归）
298 status InOrderTraverse(BiTree T, void (*visit)(BiTree))
299 {
300     if (!T) return OK;
301     InOrderTraverse(T->lchild, visit);
302     visit(T);
303     InOrderTraverse(T->rchild, visit);
304     return OK;
305 }
306
307 // 后序遍历（递归）
308 status PostOrderTraverse(BiTree T, void (*visit)(BiTree))
309 {
310     if (!T) return OK;
311     PostOrderTraverse(T->lchild, visit);
```

```
312     PostOrderTraverse(T->rchild, visit);
313     visit(T);
314     return OK;
315 }
316 
317 // 层序遍历 (队列)
318 status LevelOrderTraverse(BiTree T, void (*visit)(BiTree))
319 {
320     if (!T) return OK;
321     BiTree que[1000];
322     int front = 0, rear = 0;
323     que[rear++] = T;
324     while (front < rear)
325     {
326         BiTree p = que[front++];
327         visit(p);
328         if (p->lchild) que[rear++] = p->lchild;
329         if (p->rchild) que[rear++] = p->rchild;
330     }
331     return OK;
332 }
333 
334 //==================== 扩展功能 ====================
335 // 求根到叶子的最大路径和
336 int GetMaxPath(BiTree T)
337 {
338     if (!T) return 0;
339     int left = GetMaxPath(T->lchild);
340     int right = GetMaxPath(T->rchild);
341     return (left > right ? left : right) + T->data.key;
342 }
343 
344 int MaxPathSum(BiTree T)
345 {
346     return GetMaxPath(T);
```

```
347 }
348
349 // 最近公共祖先 LCA
350 BiTNode* LowestCommonAncestor(BiTree T, KeyType e1, KeyType e2)
351 {
352     if (!T) return NULL;
353     if (T->data.key == e1 || T->data.key == e2)
354         return T;
355     BiTNode *left = LowestCommonAncestor(T->lchild, e1, e2);
356     BiTNode *right = LowestCommonAncestor(T->rchild, e1, e2);
357     if (left && right) return T;
358     return left ? left : right;
359 }
360
361 // 翻转二叉树
362 status InvertTree(BiTree T)
363 {
364     if (!T) return OK;
365     BiTree temp = T->lchild;
366     T->lchild = T->rchild;
367     T->rchild = temp;
368     InvertTree(T->lchild);
369     InvertTree(T->rchild);
370     return OK;
371 }
372
373 //==================== 文件保存与读取 ====================
374 // 先序写入文件
375 void SaveWrite(BiTree T, FILE *fp)
376 {
377     if (!T)
378     {
379         fprintf(fp, "0 null ");
380         return;
381     }
```

```
382     fprintf(fp, "%d %s ", T->data.key, T->data.others);
383     SaveWrite(T->lchild, fp);
384     SaveWrite(T->rchild, fp);
385 }
386 
387 // 保存树到文件
388 status SaveBiTree(BiTree T, const char fname[])
389 {
390     FILE *fp = fopen(fname, "w");
391     if (!fp) return ERROR;
392     SaveWrite(T, fp);
393     fprintf(fp, "-1 end");
394     fclose(fp);
395     return OK;
396 }
397 
398 // 从文件读取建树
399 void FileReadCreate(BiTree *T, FILE *fp)
400 {
401     if (inputEnd) {
402         *T = NULL;
403         return;
404     }
405 
406     int k;
407     char s[20];
408     if (fscanf(fp, "%d %s", &k, s) != 2) {
409         inputEnd = 1;
410         *T = NULL;
411         return;
412     }
413 
414     if (k == -1) {
415         inputEnd = 1;
416         *T = NULL;
```

```
417             return;
418         }
419         if (k == 0) {
420             *T = NULL;
421             return;
422         }
423 
424         *T = (BiTree)malloc(sizeof(BiTNode));
425         (*T)->data.key = k;
426         strcpy((*T)->data.others, s);
427         (*T)->lchild = NULL;
428         (*T)->rchild = NULL;
429 
430         FileReadCreate(&(*T)->lchild, fp);
431         FileReadCreate(&(*T)->rchild, fp);
432     }
433 
434     // 加载树
435     status LoadBiTree(BiTree *T, const char fname[])
436     {
437         FILE *fp = fopen(fname, "r");
438         if (!fp) return ERROR;
439 
440         inputEnd = 0;
441         FileReadCreate(T, fp);
442 
443         int k;
444         char s[20];
445         while (!inputEnd) {
446             if (fscanf(fp, "%d %s", &k, s) != 2) break;
447             if (k == -1) {
448                 inputEnd = 1;
449                 break;
450             }
451         }
```

```
452
453     fclose(fp);
454     return OK;
455 }
456
457 //==================== 菜单 ====================
458 void ShowMenu()
459 {
460     printf("\n============二叉链表二叉树操作系统============\n");
461     printf("0.退出程序\t1.创建树\t2.切换二叉树\t3.销毁当前树\n");
462     printf("4.清空当前树\t5.判断空树\t6.求树深度\t7.关键字查找结
        ↪ 点\n");
463     printf("8.修改结点数据\t9.查询兄弟结点\t10.插入结点\t11.删除
        ↪ 结点\n");
464     printf("12.先序遍历\t13.中序遍历\t14.后序遍历\t15.层序遍历\
        ↪ t16.根叶最大路径和\n");
465     printf("17.最近公共祖先\t18.翻转二叉树\t19.保存二叉树到文件\t
        ↪ \t20.从文件读取二叉树\n");
466     printf("=======================================\n");
467     printf("请选择功能: ");
468 }
469
470 //==================== 主函数 ====================
471 int main()
472 {
473     int opt;
474     KeyType k1, k2;
475     TElemType nodeVal;
476     int lrFlag;
477     char fileStr[30];
478
479     while (1)
480     {
481         ShowMenu();
482         scanf("%d", &opt);
```

```
483         if (opt == 0) break;
484
485         // 未选择树时不能操作
486         if (curSel == -1 && opt >= 3)
487         {
488             printf("请先创建二叉树! \n");
489             continue;
490         }
491         BiTree nowTree = trees[curSel];
492
493         switch (opt)
494         {
495             case 1:  // 创建树
496                 if (treeCnt >= MAX_TREE)
497                 {
498                     printf("二叉树数量已满! \n");
499                     break;
500                 }
501                 printf("请输入先序序列(0空结点, -1结束): \n");
502                 CreateBiTree(&trees[treeCnt]);
503                 printf("输入二叉树名称: ");
504                 scanf("%s", treeNames[treeCnt]);
505                 curSel = treeCnt;
506                 treeCnt++;
507                 printf("创建成功! \n");
508                 break;
509
510             case 2:  // 切换树
511                 if (treeCnt == 0) {
512                     printf("暂无二叉树可切换! \n");
513                     break;
514                 }
515                 printf("已有二叉树列表: \n");
516                 for (int i = 0; i < treeCnt; i++)
517                     printf("%d -> %s\n", i, treeNames[i]);
```

```
518                 printf("选择序号: ");
519                 scanf("%d", &curSel);
520                 if (curSel < 0 || curSel >= treeCnt) {
521                     printf("无效序号, 切换失败! \n");
522                     curSel = (treeCnt > 0) ? 0 : -1;
523                 }
524                 break;
525
526             case 3: DestroyBiTree(&trees[curSel]); printf("已彻底
                    ↪ 销毁\n"); break;
527             case 4: ClearBiTree(&trees[curSel]); printf("已清空为
                    ↪ 空树\n"); break;
528             case 5: printf(BiTreeEmpty(nowTree) ? "是空树\n" : "
                    ↪ 非空树\n"); break;
529             case 6: printf("二叉树深度: %d\n", BiTreeDepth(
                    ↪ nowTree)); break;
530             case 7:  // 查找结点
531                 printf("输入查找关键字: ");
532                 scanf("%d", &k1);
533                 BiTNode *foundNode = LocateNode(nowTree, k1);
534                 if (foundNode) {
535                     printf("找到结点: %d,%s\n", foundNode->data.
                        ↪ key, foundNode->data.others);
536                 } else {
537                     printf("不存在该结点\n");
538                 }
539                 break;
540
541             case 8:  // 修改结点
542                 printf("输入原关键字: ");
543                 scanf("%d", &k1);
544                 printf("输入新关键字 新内容: ");
545                 scanf("%d %s", &nodeVal.key, nodeVal.others);
546                 printf(Assign(nowTree, k1, nodeVal) ? "修改成功\n
                    ↪ " : "修改失败\n");
```

```
547                 break;
548
549             case 9:  // 查兄弟
550                 printf("输入关键字: ");
551                 scanf("%d", &k1);
552                 BiTNode *sib = GetSibling(nowTree, k1);
553                 if (sib)
554                     printf("兄弟结点: %d,%s\n", sib->data.key,
                        ↪ sib->data.others);
555                 else
556                     printf("无兄弟结点\n");
557                 break;
558
559             case 10:  // 插入结点
560                 printf("父结点关键字 插入位置(-1根/0左/1右): ");
561                 scanf("%d %d", &k1, &lrFlag);
562                 printf("新结点关键字 内容: ");
563                 scanf("%d %s", &nodeVal.key, nodeVal.others);
564                 if (InsertNode(&nowTree, k1, lrFlag, nodeVal))
565                 {
566                     trees[curSel] = nowTree;
567                     printf("插入成功\n");
568                 }
569                 else
570                     printf("插入失败\n");
571                 break;
572
573             case 11:  // 删除结点
574                 printf("输入删除关键字: ");
575                 scanf("%d", &k1);
576                 if (DeleteNode(&nowTree, k1))
577                 {
578                     trees[curSel] = nowTree;
579                     printf("删除成功\n");
580                 }
```

```
581                 else
582                     printf("删除失败\n");
583                 break;
584
585             case 12:printf("先序遍历: ");PreOrderTraverse(nowTree
                    ↪ ,Visit);printf("\n");break;
586             case 13:printf("中序遍历: ");InOrderTraverse(nowTree,
                    ↪ Visit);printf("\n");break;
587             case 14:printf("后序遍历: ");PostOrderTraverse(
                    ↪ nowTree,Visit);printf("\n");break;
588             case 15:printf("层序遍历: ");LevelOrderTraverse(
                    ↪ nowTree,Visit);printf("\n");break;
589             case 16:printf("根到叶子最大路径和: %d\n", MaxPathSum
                    ↪ (nowTree));break;
590
591             case 17:  // 最近公共祖先
592                 printf("输入两个关键字: ");
593                 scanf("%d %d", &k1, &k2);
594                 BiTNode *anc = LowestCommonAncestor(nowTree, k1,
                        ↪ k2);
595                 if (anc)
596                     printf("最近公共祖先: %d,%s\n", anc->data.key
                            ↪ , anc->data.others);
597                 else
598                     printf("无公共祖先\n");
599                 break;
600
601             case 18: InvertTree(nowTree); printf("二叉树翻转完成\
                    ↪ n"); break;
602             case 19:  // 保存到文件
603                 printf("输入保存文件名: ");
604                 scanf("%s", fileStr);
605                 printf(SaveBiTree(nowTree, fileStr) ? "保存成功\n
                        ↪ " : "保存失败\n");
606                 break;
```

```
607
608             case 20:  // 从文件加载
609                 if (treeCnt >= MAX_TREE) {
610                     printf("二叉树数量已满! \n");
611                     break;
612                 }
613                 printf("输入读取文件名: ");
614                 scanf("%s", fileStr);
615                 BiTree readT = NULL;
616                 if (LoadBiTree(&readT, fileStr))
617                 {
618                     printf("输入树名: ");
619                     scanf("%s", treeNames[treeCnt]);
620                     trees[treeCnt++] = readT;
621                     curSel = treeCnt - 1;
622                     printf("读取成功\n");
623                 }
624                 else
625                     printf("读取失败\n");
626                 break;
627
628             default:
629                 printf("无效选项, 请重新输入\n");
630         }
631     }
632     printf("程序正常退出\n");
633     return 0;
634 }
```

附录 D 基于邻接表图实现的源程序

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 最大顶点数
#define MAX_VERTEX_NUM 20
// 最大管理图数量
#define MAX_GRAPH_NUM 10
// 状态码定义
#define OK 1
#define ERROR 0
#define TRUE 1
#define FALSE 0

typedef int status;    // 状态类型
typedef int KeyType;   // 顶点关键字类型

// 顶点数据结构：关键字 + 名称
typedef struct {
    KeyType key;
    char name[20];
} VertexType;

// 边结点结构（邻接表）
typedef struct ArcNode {
    int adjvex;               // 邻接点的索引
    struct ArcNode *nextarc; // 指向下一条边
} ArcNode;

// 顶点结构
typedef struct VNode {
    VertexType data;          // 顶点数据
    ArcNode *firstarc;        // 指向第一条依附该顶点的边
```

```
} VNode, AdjList[MAX_VERTEX_NUM];

// 图结构（邻接表）
typedef struct {
    AdjList vertices;          // 邻接表
    int vexnum, arcnum;        // 顶点数、边数
    char graph_name[20];       // 图名
} ALGraph;

// 图管理器：管理多张图
typedef struct {
    ALGraph graphs[MAX_GRAPH_NUM];
    int graph_count;           // 当前图数量
    int current_graph;         // 当前操作图下标
} GraphManager;

// ====================== 函数原型声明 ======================
void InitGraphManager(GraphManager *gm);
status AddGraph(GraphManager *gm, char graph_name[]);
status DeleteGraph(GraphManager *gm, char graph_name[]);
status SwitchGraph(GraphManager *gm, char graph_name[]);
void ListAllGraphs(GraphManager *gm);

status CreateGraph(ALGraph *G, VertexType V[], int n, KeyType VR
    ↪ [][2], int m);
status DestroyGraph(ALGraph *G);
int LocateVex(ALGraph *G, KeyType u);
status PutVex(ALGraph *G, KeyType u, char value[]);
int FirstAdjVex(ALGraph *G, int u);
int NextAdjVex(ALGraph *G, int v, int w);
status InsertVex(ALGraph *G, VertexType v);
status DeleteVex(ALGraph *G, KeyType v);
status InsertArc(ALGraph *G, KeyType v, KeyType w);
status DeleteArc(ALGraph *G, KeyType v, KeyType w);
```

```
67  static void DFS(ALGraph *G, int v, int visited[], void (*visit)(
        ↪ VertexType));
68  status DFSTraverse(ALGraph *G, void (*visit)(VertexType));
69  status BFSTraverse(ALGraph *G, void (*visit)(VertexType));
70
71  void VerticesSetLessThanK(ALGraph *G, KeyType v, int k);
72  int ShortestPathLength(ALGraph *G, KeyType v, KeyType w);
73  int ConnectedComponentsNums(ALGraph *G);
74  status SaveGraph(ALGraph *G, char FileName[]);
75  status LoadGraph(ALGraph *G, char FileName[]);
76
77  void Visit(VertexType v);
78  void DisplayGraph(ALGraph *G);
79  void Menu(GraphManager *gm);
80
81  // ====================== 图管理器实现 ======================
82
83  // 初始化图管理器
84  void InitGraphManager(GraphManager *gm) {
85      gm->graph_count = 0;
86      gm->current_graph = -1;
87  }
88
89  // 添加新图
90  status AddGraph(GraphManager *gm, char graph_name[]) {
91      if (gm->graph_count >= MAX_GRAPH_NUM) {
92          printf("错误：最多只能管理%d个图！\n", MAX_GRAPH_NUM);
93          return ERROR;
94      }
95
96      // 检查重名
97      for (int i = 0; i < gm->graph_count; i++) {
98          if (strcmp(gm->graphs[i].graph_name, graph_name) == 0) {
99              printf("错误：图名\"%s\"已存在！\n", graph_name);
100             return ERROR;
```

```
101             }
102         }
103
104         strcpy(gm->graphs[gm->graph_count].graph_name, graph_name);
105         gm->graphs[gm->graph_count].vexnum = 0;
106         gm->graphs[gm->graph_count].arcnum = 0;
107
108         gm->current_graph = gm->graph_count;
109         gm->graph_count++;
110
111         printf("成功添加图\"%s\", 已自动切换到该图\n", graph_name);
112         return OK;
113     }
114
115     // 删除指定图
116     status DeleteGraph(GraphManager *gm, char graph_name[]) {
117         int pos = -1;
118
119         // 查找位置
120         for (int i = 0; i < gm->graph_count; i++) {
121             if (strcmp(gm->graphs[i].graph_name, graph_name) == 0) {
122                 pos = i;
123                 break;
124             }
125         }
126
127         if (pos == -1) {
128             printf("错误: 图\"%s\"不存在! \n", graph_name);
129             return ERROR;
130         }
131
132         // 销毁图内存
133         DestroyGraph(&gm->graphs[pos]);
134
135         // 后面的图前移
```

```
136     for (int i = pos; i < gm->graph_count - 1; i++) {
137         gm->graphs[i] = gm->graphs[i + 1];
138     }
139 
140     gm->graph_count--;
141 
142     // 调整当前图
143     if (gm->current_graph == pos) {
144         if (gm->graph_count > 0) {
145             gm->current_graph = 0;
146             printf("已删除当前图，自动切换到图\"%s\"\n", gm->
                ↪ graphs[0].graph_name);
147         } else {
148             gm->current_graph = -1;
149             printf("已删除所有图，当前无激活图\n");
150         }
151     } else if (gm->current_graph > pos) {
152         gm->current_graph--;
153     }
154 
155     printf("成功删除图\"%s\"\n", graph_name);
156     return OK;
157 }
158 
159 // 切换当前操作图
160 status SwitchGraph(GraphManager *gm, char graph_name[]) {
161     for (int i = 0; i < gm->graph_count; i++) {
162         if (strcmp(gm->graphs[i].graph_name, graph_name) == 0) {
163             gm->current_graph = i;
164             printf("已切换到图\"%s\"\n", graph_name);
165             return OK;
166         }
167     }
168 
169     printf("错误：图\"%s\"不存在！\n", graph_name);
```

```
170     return ERROR;
171 }
172
173 // 列出所有图
174 void ListAllGraphs(GraphManager *gm) {
175     if (gm->graph_count == 0) {
176         printf("当前没有管理任何图\n");
177         return;
178     }
179
180     printf("\n===== 所有图列表 =====\n");
181     for (int i = 0; i < gm->graph_count; i++) {
182         if (i == gm->current_graph) {
183             printf("* %d. %s (当前操作图)\n", i + 1, gm->graphs[i
                ↪ ].graph_name);
184         } else {
185             printf("  %d. %s\n", i + 1, gm->graphs[i].graph_name)
                ↪ ;
186         }
187     }
188     printf("======================\n");
189 }
190
191 // ====================== 基础图操作 ======================
192
193 // 创建无向图
194 status CreateGraph(ALGraph *G, VertexType V[], int n, KeyType VR
    ↪ [][2], int m) {
195     int i, j, k;
196     ArcNode *p;
197
198     G->vexnum = n;
199     G->arcnum = 0;
200
201     // 初始化顶点
```

```
202     for (i = 0; i < n; i++) {
203         G->vertices[i].data = V[i];
204         G->vertices[i].firstarc = NULL;
205     }
206 
207     // 建立边
208     for (k = 0; k < m; k++) {
209         int pos1 = -1, pos2 = -1;
210         for (i = 0; i < n; i++) {
211             if (G->vertices[i].data.key == VR[k][0]) pos1 = i;
212             if (G->vertices[i].data.key == VR[k][1]) pos2 = i;
213         }
214         if (pos1 == -1 || pos2 == -1 || pos1 == pos2) continue;
215 
216         // 无向图双向插入
217         p = (ArcNode*)malloc(sizeof(ArcNode));
218         p->adjvex = pos2;
219         p->nextarc = G->vertices[pos1].firstarc;
220         G->vertices[pos1].firstarc = p;
221 
222         p = (ArcNode*)malloc(sizeof(ArcNode));
223         p->adjvex = pos1;
224         p->nextarc = G->vertices[pos2].firstarc;
225         G->vertices[pos2].firstarc = p;
226 
227         G->arcnum++;
228     }
229 
230     return OK;
231 }
232 
233 // 销毁图（释放所有边结点）
234 status DestroyGraph(ALGraph *G) {
235     int i;
236     ArcNode *p, *q;
```

```
237
238     for (i = 0; i < G->vexnum; i++) {
239         p = G->vertices[i].firstarc;
240         while (p) {
241             q = p;
242             p = p->nextarc;
243             free(q);
244         }
245         G->vertices[i].firstarc = NULL;
246     }
247
248     G->vexnum = 0;
249     G->arcnum = 0;
250     return OK;
251 }
252
253 // 根据关键字查找顶点位置
254 int LocateVex(ALGraph *G, KeyType u) {
255     int i;
256     for (i = 0; i < G->vexnum; i++) {
257         if (G->vertices[i].data.key == u) return i;
258     }
259     return -1;
260 }
261
262 // 修改顶点名称
263 status PutVex(ALGraph *G, KeyType u, char value[]) {
264     int pos = LocateVex(G, u);
265     if (pos == -1) return ERROR;
266     strcpy(G->vertices[pos].data.name, value);
267     return OK;
268 }
269
270 // 获取第一个邻接点
271 int FirstAdjVex(ALGraph *G, int u) {
```

```
272     if (u < 0 || u >= G->vexnum) return -1;
273     if (G->vertices[u].firstarc == NULL) return -1;
274     return G->vertices[u].firstarc->adjvex;
275 }
276 
277 // 获取下一个邻接点
278 int NextAdjVex(ALGraph *G, int v, int w) {
279     ArcNode *p;
280     if (v < 0 || v >= G->vexnum || w < 0 || w >= G->vexnum)
        ↪ return -1;
281     p = G->vertices[v].firstarc;
282     while (p) {
283         if (p->adjvex == w) {
284             if (p->nextarc) return p->nextarc->adjvex;
285             else return -1;
286         }
287         p = p->nextarc;
288     }
289     return -1;
290 }
291 
292 // 插入顶点
293 status InsertVex(ALGraph *G, VertexType v) {
294     int i;
295     if (G->vexnum >= MAX_VERTEX_NUM) return ERROR;
296     // 检查关键字重复
297     for (i = 0; i < G->vexnum; i++) {
298         if (G->vertices[i].data.key == v.key) return ERROR;
299     }
300     G->vertices[G->vexnum].data = v;
301     G->vertices[G->vexnum].firstarc = NULL;
302     G->vexnum++;
303     return OK;
304 }
305 
```

```
306 // 删除顶点及所有关联边
307 status DeleteVex(ALGraph *G, KeyType v) {
308     int i, j, pos;
309     ArcNode *p, *q, *pre;
310 
311     pos = LocateVex(G, v);
312     if (pos == -1) return ERROR;
313 
314     // 删除所有指向该顶点的边
315     for (i = 0; i < G->vexnum; i++) {
316         if (i == pos) continue;
317         pre = NULL;
318         p = G->vertices[i].firstarc;
319         while (p) {
320             if (p->adjvex == pos) {
321                 if (pre == NULL)
322                     G->vertices[i].firstarc = p->nextarc;
323                 else
324                     pre->nextarc = p->nextarc;
325                 q = p;
326                 p = p->nextarc;
327                 free(q);
328                 G->arcnum--;
329             } else {
330                 if (p->adjvex > pos) p->adjvex--;
331                 pre = p;
332                 p = p->nextarc;
333             }
334         }
335     }
336 
337     // 删除该顶点的所有边
338     p = G->vertices[pos].firstarc;
339     while (p) {
340         q = p;
```

```
341         p = p->nextarc;
342         free(q);
343     }
344
345     // 顶点前移
346     for (i = pos; i < G->vexnum-1; i++) {
347         G->vertices[i] = G->vertices[i+1];
348     }
349
350     G->vexnum--;
351     return OK;
352 }
353
354 // 插入边（无向图双向插入）
355 status InsertArc(ALGraph *G, KeyType v, KeyType w) {
356     int pos1 = LocateVex(G, v);
357     int pos2 = LocateVex(G, w);
358     ArcNode *p;
359
360     if (pos1 == -1 || pos2 == -1 || pos1 == pos2) return ERROR;
361
362     // 检查边已存在
363     p = G->vertices[pos1].firstarc;
364     while (p) {
365         if (p->adjvex == pos2) return ERROR;
366         p = p->nextarc;
367     }
368
369     p = (ArcNode*)malloc(sizeof(ArcNode));
370     p->adjvex = pos2;
371     p->nextarc = G->vertices[pos1].firstarc;
372     G->vertices[pos1].firstarc = p;
373
374     p = (ArcNode*)malloc(sizeof(ArcNode));
375     p->adjvex = pos1;
```

```
376     p->nextarc = G->vertices[pos2].firstarc;
377     G->vertices[pos2].firstarc = p;
378 
379     G->arcnum++;
380     return OK;
381 }
382 
383 // 删除边
384 status DeleteArc(ALGraph *G, KeyType v, KeyType w) {
385     int pos1 = LocateVex(G, v);
386     int pos2 = LocateVex(G, w);
387     ArcNode *p, *pre, *q;
388     int found = 0;
389 
390     if (pos1 == -1 || pos2 == -1 || pos1 == pos2) return ERROR;
391 
392     // 删除 v -> w
393     pre = NULL;
394     p = G->vertices[pos1].firstarc;
395     while (p) {
396         if (p->adjvex == pos2) {
397             if (pre == NULL)
398                 G->vertices[pos1].firstarc = p->nextarc;
399             else
400                 pre->nextarc = p->nextarc;
401             q = p;
402             p = p->nextarc;
403             free(q);
404             found = 1;
405             break;
406         }
407         pre = p;
408         p = p->nextarc;
409     }
410 
```

```
411     if (!found) return ERROR;
412
413     // 删除 w -> v
414     pre = NULL;
415     p = G->vertices[pos2].firstarc;
416     while (p) {
417         if (p->adjvex == pos1) {
418             if (pre == NULL)
419                 G->vertices[pos2].firstarc = p->nextarc;
420             else
421                 pre->nextarc = p->nextarc;
422             q = p;
423             p = p->nextarc;
424             free(q);
425             break;
426         }
427         pre = p;
428         p = p->nextarc;
429     }
430
431     G->arcnum--;
432     return OK;
433 }
434
435 // ====================== 遍历 ======================
436
437 // 深度优先遍历递归函数
438 static void DFS(ALGraph *G, int v, int visited[], void (*visit)(
    ↪ VertexType)) {
439     ArcNode *p;
440     int w;
441
442     if (visit != NULL) {
443         visit(G->vertices[v].data);
444     }
```

```
445     visited[v] = 1;
446
447     p = G->vertices[v].firstarc;
448     while (p) {
449         w = p->adjvex;
450         if (!visited[w]) {
451             DFS(G, w, visited, visit);
452         }
453         p = p->nextarc;
454     }
455 }
456
457 // 深度优先遍历
458 status DFSTraverse(ALGraph *G, void (*visit)(VertexType)) {
459     int i;
460     int *visited;
461
462     visited = (int *)malloc(G->vexnum * sizeof(int));
463     for (i = 0; i < G->vexnum; i++)
464         visited[i] = 0;
465
466     for (i = 0; i < G->vexnum; i++) {
467         if (!visited[i])
468             DFS(G, i, visited, visit);
469     }
470
471     free(visited);
472     return OK;
473 }
474
475 // 广度优先遍历
476 status BFSTraverse(ALGraph *G, void (*visit)(VertexType)) {
477     int i, u, w;
478     int *visited;
479     int *queue;
```

```
480     int front, rear;
481     ArcNode *p;
482 
483     visited = (int *)malloc(G->vexnum * sizeof(int));
484     for (i = 0; i < G->vexnum; i++)
485         visited[i] = 0;
486 
487     queue = (int *)malloc(G->vexnum * sizeof(int));
488     front = rear = 0;
489 
490     for (i = 0; i < G->vexnum; i++) {
491         if (!visited[i]) {
492             visit(G->vertices[i].data);
493             visited[i] = 1;
494             queue[rear++] = i;
495 
496             while (front != rear) {
497                 u = queue[front++];
498                 p = G->vertices[u].firstarc;
499                 while (p != NULL) {
500                     w = p->adjvex;
501                     if (!visited[w]) {
502                         visit(G->vertices[w].data);
503                         visited[w] = 1;
504                         queue[rear++] = w;
505                     }
506                     p = p->nextarc;
507                 }
508             }
509         }
510     }
511 
512     free(visited);
513     free(queue);
514     return OK;
```

```
515 }
516
517 // ====================== 扩展功能 ======================
518
519 // 输出与顶点 v 距离小于 k 的所有顶点
520 void VerticesSetLessThanK(ALGraph *G, KeyType v, int k) {
521     int i, u, w, dist;
522     int *visited, *distance;
523     int *queue;
524     int front, rear;
525     ArcNode *p;
526     int pos = LocateVex(G, v);
527
528     if (pos == -1 || k <= 0) {
529         printf("输入错误! \n");
530         return;
531     }
532
533     visited = (int *)malloc(G->vexnum * sizeof(int));
534     distance = (int *)malloc(G->vexnum * sizeof(int));
535     queue = (int *)malloc(G->vexnum * sizeof(int));
536     front = rear = 0;
537
538     for (i = 0; i < G->vexnum; i++) {
539         visited[i] = 0;
540         distance[i] = -1;
541     }
542
543     visited[pos] = 1;
544     distance[pos] = 0;
545     queue[rear++] = pos;
546
547     printf("与顶点 %d(%s) 距离小于 %d 的顶点有: \n", v, G->
        ↪ vertices[pos].data.name, k);
548     while (front != rear) {
```

```
549             u = queue[front++];
550             if (distance[u] < k-1) {
551                 p = G->vertices[u].firstarc;
552                 while (p) {
553                     w = p->adjvex;
554                     if (!visited[w]) {
555                         visited[w] = 1;
556                         distance[w] = distance[u] + 1;
557                         printf("%d(%s) 距离: %d\n",
558                             G->vertices[w].data.key,
559                             G->vertices[w].data.name,
560                             distance[w]);
561                         queue[rear++] = w;
562                     }
563                     p = p->nextarc;
564                 }
565             }
566         }
567 
568         free(visited);
569         free(distance);
570         free(queue);
571 }
572 
573 // 求两点间最短路径长度 (BFS)
574 int ShortestPathLength(ALGraph *G, KeyType v, KeyType w) {
575         int i, u, x;
576         int *visited, *distance;
577         int *queue;
578         int front, rear;
579         ArcNode *p;
580         int pos1 = LocateVex(G, v);
581         int pos2 = LocateVex(G, w);
582 
583         if (pos1 == -1 || pos2 == -1) return -1;
```

```
584     if (pos1 == pos2) return 0;
585
586     visited = (int *)malloc(G->vexnum * sizeof(int));
587     distance = (int *)malloc(G->vexnum * sizeof(int));
588     queue = (int *)malloc(G->vexnum * sizeof(int));
589     front = rear = 0;
590
591     for (i = 0; i < G->vexnum; i++) {
592         visited[i] = 0;
593         distance[i] = -1;
594     }
595
596     visited[pos1] = 1;
597     distance[pos1] = 0;
598     queue[rear++] = pos1;
599
600     while (front != rear) {
601         u = queue[front++];
602         p = G->vertices[u].firstarc;
603         while (p) {
604             x = p->adjvex;
605             if (!visited[x]) {
606                 visited[x] = 1;
607                 distance[x] = distance[u] + 1;
608                 if (x == pos2) {
609                     int len = distance[x];
610                     free(visited);
611                     free(distance);
612                     free(queue);
613                     return len;
614                 }
615                 queue[rear++] = x;
616             }
617             p = p->nextarc;
618         }
```

```
619     }
620
621     free(visited);
622     free(distance);
623     free(queue);
624     return -1;
625 }
626
627 // 求连通分量个数
628 int ConnectedComponentsNums(ALGraph *G) {
629     int i, count = 0;
630     int *visited;
631
632     visited = (int *)malloc(G->vexnum * sizeof(int));
633     for (i = 0; i < G->vexnum; i++)
634         visited[i] = 0;
635
636     for (i = 0; i < G->vexnum; i++) {
637         if (!visited[i]) {
638             count++;
639             DFS(G, i, visited, NULL);
640         }
641     }
642
643     free(visited);
644     return count;
645 }
646
647 // ====================== 文件操作 ======================
648
649 // 保存图到文件
650 status SaveGraph(ALGraph *G, char FileName[]) {
651     FILE *fp = fopen(FileName, "w");
652     if (!fp) return ERROR;
653
```

```
654     ArcNode *p;
655     int i, j;
656     int **visited;
657 
658     fprintf(fp, "%d %d\n", G->vexnum, G->arcnum);
659 
660     for (i = 0; i < G->vexnum; i++) {
661         fprintf(fp, "%d %s\n", G->vertices[i].data.key, G->
            ↪ vertices[i].data.name);
662     }
663 
664     // 避免重复保存无向边
665     visited = (int**)malloc(G->vexnum * sizeof(int*));
666     for (i = 0; i < G->vexnum; i++) {
667         visited[i] = (int*)calloc(G->vexnum, sizeof(int));
668     }
669 
670     for (i = 0; i < G->vexnum; i++) {
671         p = G->vertices[i].firstarc;
672         while (p) {
673             j = p->adjvex;
674             if (i < j && !visited[i][j]) {
675                 fprintf(fp, "%d %d\n",
676                     G->vertices[i].data.key,
677                     G->vertices[j].data.key);
678                 visited[i][j] = visited[j][i] = 1;
679             }
680             p = p->nextarc;
681         }
682     }
683 
684     for (i = 0; i < G->vexnum; i++) {
685         free(visited[i]);
686     }
687     free(visited);
```

```
688
689     fclose(fp);
690     return OK;
691 }
692
693 // 从文件加载图
694 status LoadGraph(ALGraph *G, char FileName[]) {
695     FILE *fp = fopen(FileName, "r");
696     if (!fp) return ERROR;
697
698     int i, j;
699     int key1, key2;
700     ArcNode *p;
701     int vexnum, arcnum;
702
703     fscanf(fp, "%d%d", &vexnum, &arcnum);
704     G->vexnum = vexnum;
705     G->arcnum = 0;
706
707     for (i = 0; i < vexnum; i++) {
708         fscanf(fp, "%d%s", &G->vertices[i].data.key, G->vertices[
            ↪ i].data.name);
709         G->vertices[i].firstarc = NULL;
710     }
711
712     for (i = 0; i < arcnum; i++) {
713         fscanf(fp, "%d%d", &key1, &key2);
714
715         int pos1 = -1, pos2 = -1;
716         for (j = 0; j < vexnum; j++) {
717             if (G->vertices[j].data.key == key1) pos1 = j;
718             if (G->vertices[j].data.key == key2) pos2 = j;
719         }
720         if (pos1 == -1 || pos2 == -1) continue;
721
```

```
722         p = (ArcNode*)malloc(sizeof(ArcNode));
723         p->adjvex = pos2;
724         p->nextarc = G->vertices[pos1].firstarc;
725         G->vertices[pos1].firstarc = p;
726 
727         p = (ArcNode*)malloc(sizeof(ArcNode));
728         p->adjvex = pos1;
729         p->nextarc = G->vertices[pos2].firstarc;
730         G->vertices[pos2].firstarc = p;
731 
732         G->arcnum++;
733     }
734 
735     fclose(fp);
736     return OK;
737 }
738 
739 // 访问顶点：输出信息
740 void Visit(VertexType v) {
741     printf("%d %s ", v.key, v.name);
742 }
743 
744 // 显示图邻接表
745 void DisplayGraph(ALGraph *G) {
746     int i;
747     ArcNode *p;
748 
749     printf("\n图\"%s\"的邻接表表示：\n", G->graph_name);
750     for (i = 0; i < G->vexnum; i++) {
751         printf("%d %s", G->vertices[i].data.key, G->vertices[i].
            ↪ data.name);
752         p = G->vertices[i].firstarc;
753         while (p) {
754             printf(" %d", p->adjvex);
755             p = p->nextarc;
```

```
756             }
757             printf("\n");
758         }
759     }
760
761     // 菜单
762     void Menu(GraphManager *gm) {
763         printf("\n=========== 无向图邻接表操作菜单 ===========\n");
764         printf("//单图操作 (当前操作图: ");
765         if (gm->current_graph != -1) {
766             printf("%s", gm->graphs[gm->current_graph].graph_name);
767         } else {
768             printf("无");
769         }
770         printf(") \n");
771         printf("1. 创建图\t\t2. 销毁图\t\t3. 查找顶点\t\t4. 顶点赋值\
            ↪ n");
772         printf("5. 获得第一邻接点\t6. 获得下一邻接点\t7. 插入顶点\t\
            ↪ t8. 删除顶点\n");
773         printf("9. 插入弧\t\t10. 删除弧\t\t11. 深度优先遍历\t12. 广度
            ↪ 优先遍历\n");
774         printf("13. 距离小于k的顶点集合\t14. 顶点间最短路径长度\t15.
            ↪ 图的连通分量个数\t16. 保存图到文件\n");
775         printf("17. 从文件加载图\t18. 显示图\n");
776         printf("//多图管理\n");
777         printf("19. 添加新图\t\t20. 删除图\t\t21. 切换当前图\t\t22.
            ↪ 显示所有图\n");
778         printf("0. 退出\n");
779         printf("=============================================\n");
780         printf("请输入您的选择: ");
781     }
782
783     // 主函数
784     int main() {
785         GraphManager gm;
```

```
786     InitGraphManager(&gm);
787     int choice;
788     KeyType key1, key2;
789     char name[20], filename[50];
790     int k, len, pos;
791     VertexType v;
792 
793     // 默认创建一张图
794     AddGraph(&gm, "默认图");
795 
796     while (1) {
797         Menu(&gm);
798         scanf("%d", &choice);
799         printf("\n");
800 
801         // 未选择图时不能操作
802         if (choice >= 1 && choice <= 18 && gm.current_graph ==
            ↪ -1) {
803             printf("错误: 当前没有激活的图, 请先添加新图! \n");
804             continue;
805         }
806 
807         ALGraph *current_G = &gm.graphs[gm.current_graph];
808 
809         switch (choice) {
810             case 1: {
811                 int n, m, i;
812                 VertexType V[MAX_VERTEX_NUM];
813                 KeyType VR[MAX_VERTEX_NUM*MAX_VERTEX_NUM][2];
814 
815                 printf("请输入顶点个数: ");
816                 scanf("%d", &n);
817                 printf("请输入顶点信息 (关键字 名称) : \n");
818                 for (i = 0; i < n; i++) {
819                     scanf("%d%s", &V[i].key, V[i].name);
```

```
820                 }
821                 printf("请输入边的个数: ");
822                 scanf("%d", &m);
823                 printf("请输入边信息 (顶点1关键字 顶点2关键字) :
                        ↪ \n");
824                 for (i = 0; i < m; i++) {
825                     scanf("%d%d", &VR[i][0], &VR[i][1]);
826                 }
827 
828                 if (CreateGraph(current_G, V, n, VR, m) == OK)
829                     printf("图创建成功! \n");
830                 else
831                     printf("图创建失败! \n");
832                 break;
833             }
834             case 2:
835                 DestroyGraph(current_G);
836                 printf("图销毁成功! \n");
837                 break;
838             case 3:
839                 printf("请输入要查找的顶点关键字: ");
840                 scanf("%d", &key1);
841                 pos = LocateVex(current_G, key1);
842                 if (pos != -1)
843                     printf("顶点 %d(%s) 存在, 位序为 %d\n", key1,
                            ↪  current_G->vertices[pos].data.name,
                            ↪ pos);
844                 else
845                     printf("顶点不存在! \n");
846                 break;
847             case 4:
848                 printf("请输入要赋值的顶点关键字: ");
849                 scanf("%d", &key1);
850                 printf("请输入新的顶点名称: ");
851                 scanf("%s", name);
```

```
852                 if (PutVex(current_G, key1, name) == OK)
853                     printf("顶点赋值成功! \n");
854                 else
855                     printf("顶点赋值失败! \n");
856                 break;
857             case 5:
858                 printf("请输入顶点位序: ");
859                 scanf("%d", &pos);
860                 int adj = FirstAdjVex(current_G, pos);
861                 if (adj != -1)
862                     printf("第一个邻接点位序为 %d, 关键字为 %d\n"
                        ↪ , adj, current_G->vertices[adj].data.
                        ↪ key);
863                 else
864                     printf("该顶点没有邻接点! \n");
865                 break;
866             case 6:
867                 int v_pos, w_pos;
868                 printf("请输入顶点v的位序: ");
869                 scanf("%d", &v_pos);
870                 printf("请输入邻接点w的位序: ");
871                 scanf("%d", &w_pos);
872                 int next = NextAdjVex(current_G, v_pos, w_pos);
873                 if (next != -1)
874                     printf("下一个邻接点位序为 %d, 关键字为 %d\n"
                        ↪ , next, current_G->vertices[next].data.
                        ↪ key);
875                 else
876                     printf("没有下一个邻接点! \n");
877                 break;
878             case 7:
879                 printf("请输入新顶点信息 (关键字 名称): ");
880                 scanf("%d%s", &v.key, v.name);
881                 if (InsertVex(current_G, v) == OK)
882                     printf("顶点插入成功! \n");
```

```
883                 else
884                     printf("顶点插入失败! \n");
885                 break;
886             case 8:
887                 printf("请输入要删除的顶点关键字: ");
888                 scanf("%d", &key1);
889                 if (DeleteVex(current_G, key1) == OK)
890                     printf("顶点删除成功! \n");
891                 else
892                     printf("顶点删除失败! \n");
893                 break;
894             case 9:
895                 printf("请输入边的两个顶点关键字: ");
896                 scanf("%d%d", &key1, &key2);
897                 if (InsertArc(current_G, key1, key2) == OK)
898                     printf("边插入成功! \n");
899                 else
900                     printf("边插入失败! \n");
901                 break;
902             case 10:
903                 printf("请输入要删除的边的两个顶点关键字: ");
904                 scanf("%d%d", &key1, &key2);
905                 if (DeleteArc(current_G, key1, key2) == OK)
906                     printf("边删除成功! \n");
907                 else
908                     printf("边删除失败! \n");
909                 break;
910             case 11:
911                 printf("深度优先遍历结果: ");
912                 DFSTraverse(current_G, Visit);
913                 printf("\n");
914                 break;
915             case 12:
916                 printf("广度优先遍历结果: ");
917                 BFSTraverse(current_G, Visit);
```

```
918                 printf("\n");
919                 break;
920             case 13:
921                 printf("请输入顶点关键字: ");
922                 scanf("%d", &key1);
923                 printf("请输入距离k: ");
924                 scanf("%d", &k);
925                 VerticesSetLessThanK(current_G, key1, k);
926                 break;
927             case 14:
928                 printf("请输入两个顶点关键字: ");
929                 scanf("%d%d", &key1, &key2);
930                 len = ShortestPathLength(current_G, key1, key2);
931                 if (len != -1)
932                     printf("顶点 %d 和 %d 之间的最短路径长度为 %d
                        ↪ \n", key1, key2, len);
933                 else
934                     printf("两个顶点不可达! \n");
935                 break;
936             case 15:
937                 printf("图的连通分量个数为: %d\n",
                    ↪ ConnectedComponentsNums(current_G));
938                 break;
939             case 16:
940                 printf("请输入文件名: ");
941                 scanf("%s", filename);
942                 if (SaveGraph(current_G, filename) == OK)
943                     printf("图保存成功! \n");
944                 else
945                     printf("图保存失败! \n");
946                 break;
947             case 17:
948                 printf("请输入文件名: ");
949                 scanf("%s", filename);
950                 if (LoadGraph(current_G, filename) == OK)
```

```
951                     printf("图加载成功！\n");
952                 else
953                     printf("图加载失败！\n");
954                 break;
955             case 18:
956                 DisplayGraph(current_G);
957                 break;
958             case 19:
959                 printf("请输入新图名称：");
960                 scanf("%s", name);
961                 AddGraph(&gm, name);
962                 break;
963             case 20:
964                 printf("请输入要删除的图名称：");
965                 scanf("%s", name);
966                 DeleteGraph(&gm, name);
967                 break;
968             case 21:
969                 printf("请输入要切换的图名称：");
970                 scanf("%s", name);
971                 SwitchGraph(&gm, name);
972                 break;
973             case 22:
974                 ListAllGraphs(&gm);
975                 break;
976             case 0:
977                 for (int i = 0; i < gm.graph_count; i++) {
978                     DestroyGraph(&gm.graphs[i]);
979                 }
980                 printf("程序退出！\n");
981                 return 0;
982             default:
983                 printf("输入错误，请重新输入！\n");
984         }
985     }
```

```
986
987         return 0;
988 }
```